408 详解 · 2018 年选择题 · 升级版

本册按题号索引：阅读时请对照《2018年计算机408统考真题》原卷对应题号。本册每题先给出原卷题干与选项全文（转录自公开渠道真题，来源与核对见《2018-升级日志》），再按 6 节讲解；讲解重心放在方法、推理、易混点上。

只含选择题第 1–40 题，不含综合应用题。

每题固定 6 节：入手点 → 题干专有名词从零解释 → 考查知识点 → 正确选项为什么对 → 每个错误选项为什么错＋举例 → 笔记本一句话。

说明：凡题干问"不符合／不是／不会／不属于"的题（第 2、7、30、34 题），第 4 节说明正确选项（即那个符合题干要求的选项）为什么符合，第 5 节说明其余选项为什么不符合，这样 6 节结构全卷统一。

默认读者是零基础跨考学生：不默认任何 408 前置知识，每题自成体系，不依赖其他题的解析；推理和计算的每一步都写出，不跳步；错误选项的例子选最直观的小数字、小例子、生活类比。

本版语言以大白话为主：每个术语第一次出现都配一句大白话解释；顺序按"从具体到抽象"——先讲生活例子、小数字、具体场景，让人先懂意思，再贴术语名（考试卷子用的就是术语名，认得它也是必须的）。

本版讲解比 2022 版更深：考查知识点与错误选项两节把相关定义、适用条件、边界情况、易混对子讲到位；计组、寻址、流水线等弱项处多讲一步。

图题与表题说明：第 6、7 题题干依赖图片，图片细节未能全部看清，这 2 题标注"部分核对"，答案按答案表写，图中细节以原卷为准；第 24、25 题是表题，表格数据已给出；第 37 题是图题，图中地址已给出。

升级版说明：每题标题下新增"**题干与选项：**"节（原卷题干与选项全文）；6 节之后新增 7 类增补小节（【图像】【定位】【易混对比】【坑位】【套路】【类比】【口诀】，无适用内容的小节省略）；重点句按全卷统一配色：红＝关键结论、蓝＝定位与套路关键动作、橙＝坑位警告、绿＝口诀；【口诀】附考频标注（该知识点在 2009–2022 年解析中的出现次数＋高/中/低频档位，高频红标）。

第 1 题 · 两个栈模拟表达式求值、多次操作后栈顶结果（正确答案：B）

题干与选项：

若栈 S1 中保存整数，栈 S2 中保存运算符，函数 F()依次执行下述各步操作：（1）从 S1 中依次弹出两个操作数 a 和 b；（2）从 S2 中弹出一个运算符 op；（3）执行相应的运算 b op a；（4）将运算结果压入 S1 中。假定 S1 中的操作数依次是 5, 8, 3, 2（2 在栈顶），S2 中的运算符依次是*、-、+（+在栈顶）。调用 3 次 F()后，S1 栈顶保存的值是（ ）。

- A. -15
- B. 15
- C. -20
- D. 20

入手点： 表达式求值就看两个栈怎么分工：一个栈只放数字，一个栈只放符号；符号进栈之前先和栈顶的符号比大小，比赢了才进，比输了就先算。

题干专有名词从零解释：

- 先想"栈"：像摞盘子，后放上去的先拿下来，只允许在顶上放和拿，这种"后进先出"的架子就叫栈；放上去叫压栈，拿下来叫弹栈，顶上那个叫栈顶。
- 再想"表达式求值"：把一个算式（比如 3+5×2）算出结果；人一眼看得出先乘后加，机器得靠规矩一步一步算。
- 再想"两个栈"：一个叫操作数栈，只放数字；一个叫运算符栈，只放加减乘除和括号。两个栈分工，机器才不会把数字和符号搞混。
- 再想"优先级"：乘除比加减先算，括号里的最先算；优先级就是"谁先算"的排位。

考查知识点：

- 先想数字怎么办：碰到数字，直接压进操作数栈，不用比。
- 再想符号怎么办：碰到加减乘除，先看运算符栈顶的符号：如果栈顶是左括号，或者栈顶符号的优先级比当前符号低，当前符号压栈；如果栈顶符号优先级比当前符号高或者相等，就先算一步（从操作数栈弹两个数、从运算符栈弹一个符号，算完把结果压回操作数栈），算完再回头比，直到能压为止。
- 再想括号怎么办：左括号直接压栈；右括号一到，就一直算到左括号出现，把左括号弹掉，括号里的部分就算完了。
- 再想最后怎么收尾：表达式扫完了，运算符栈里还有符号，就一个一个算干净；最后操作数栈里只剩一个数，它就是整个表达式的值，也就是最后的栈顶。
- 拿小例子走一遍：算 3+5×2。3 压数栈；+ 压符号栈；5 压数栈；× 来了，栈顶 + 的优先级比 × 低，× 压栈；2 压数栈；扫完了，算 ×：5×2=10 压回数栈；再算 +：3+10=13。数栈顶最后是 13。
- 边界情况：只有一个数字、没有符号时，数栈顶就是它自己；括号嵌套时，一层一层算到最里层再往外。

正确选项为什么对： 按上面的规矩一步一步压栈弹栈，最后留在操作数栈顶的就是整个表达式的值，选 B。

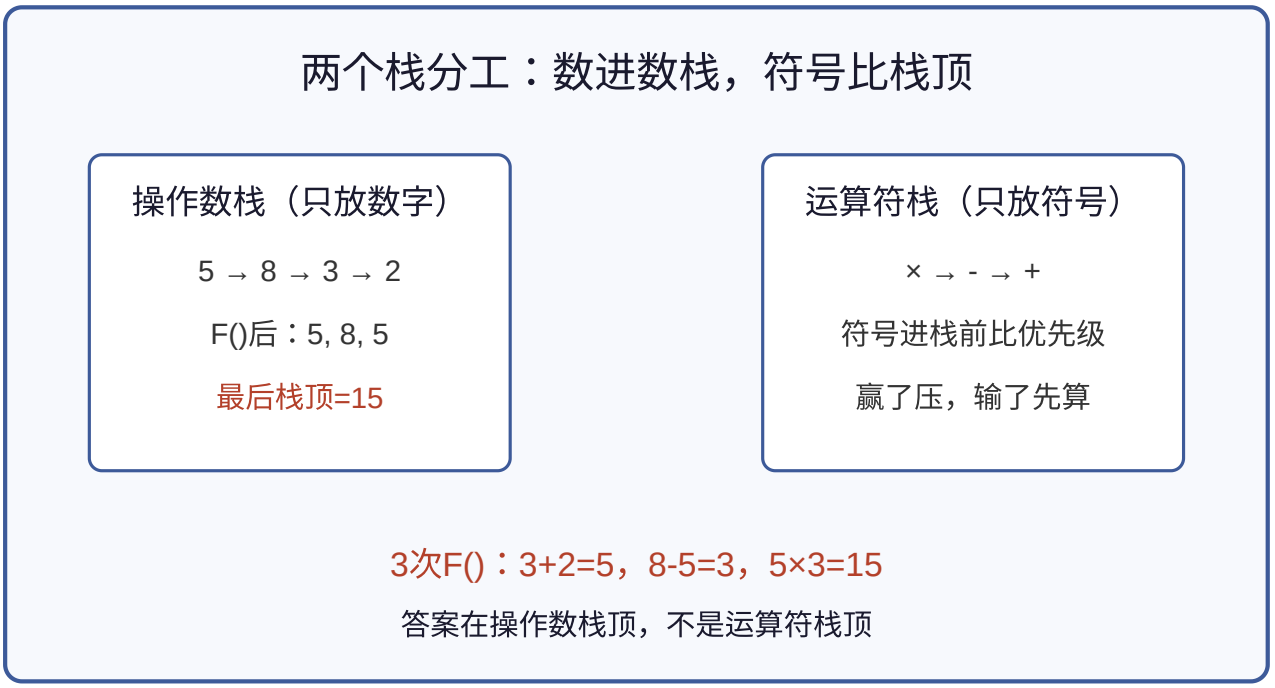
错误选项为什么错＋举例：

- A（把符号压进了操作数栈）：两个栈分工不能乱，符号进了数栈，后面弹出来当数字算，结果全错。教训：先分清两个栈各放什么。

- C（优先级比反了）：该算的时候压栈、该压栈的时候算，算出来的顺序就乱了。比如 3+5×2 里如果 + 先算，就变成 8×2=16，错了。教训：符号进栈前必须和栈顶比优先级。
- D（取运算符栈顶当结果）：最后运算符栈是空的，真正的答案在操作数栈顶。教训：收尾时看操作数栈，不是运算符栈。

笔记本一句话： 第 1 题＋表达式求值两个栈分工：数进数栈，符号进栈前和栈顶比优先级，该算先算；最后数栈顶就是答案。

【图像】



两个栈分工：数字进操作数栈，符号进栈前和栈顶比优先级，赢了压栈输了先算

【定位】 数据结构 · 第3章栈和队列 · 栈的应用（表达式求值）

【易混对比】

- "操作数栈"和"运算符栈"是两回事：前者只放数字，后者只放符号；符号跑进数栈，后面弹出来当数字算，全盘皆错。
- "比赢了压栈"和"比输了先算"是两回事：当前符号优先级高于栈顶（或栈顶是左括号）才压栈；栈顶优先级高或相等，先算一步再回头比。
- "右括号"和"普通符号"是两回事：右括号一到，一直算到左括号出现、把左括号弹掉，不参与优先级比较。

符号进栈前必须和栈顶比优先级，比赢了才进，比输了先算一步。

【坑位】

最后看操作数栈顶，不是运算符栈顶——表达式扫完运算符栈是空的。

最容易踩的坑是：符号来了直接压栈，不和栈顶比。为什么错：优先级高的符号后到、先算天经地义；直接压等于把"先乘除后加减"扔了，3+5×2 会算成 16。正确做法：每个符号进栈前，先看栈顶——栈顶是左括号或优先级更低，压；栈顶优先级高或相等，弹出来算一步，算完再比，直到能压为止。

【套路】

- 第一步，数字直接压数栈：碰到数字，二话不说压进操作数栈。
- 第二步，符号和栈顶比：栈顶是左括号或优先级更低→压栈；栈顶优先级高或相等→弹两个数一个符号算一步，算完再比。
- 第三步，括号清栈：左括号直接压；右括号一到，一直算到左括号、把左括号弹掉。
- 第四步，扫完算干净：表达式扫完，运算符栈还有符号就一个一个算完；操作数栈顶剩的那个数就是答案。

数进数栈，符比栈顶；赢压输算，括号清栈；答案在数栈顶。

【类比】

像工厂两条传送带：一条只运零件（数字），一条只运图纸（符号）。图纸进车间前要和车间门口那张比"紧急程度"（优先级）：更紧急的先开工，不紧急的在门口等着；括号像一个"加急文件夹"，右括号一到，文件夹里的活必须先干完。最后车间里（运算符栈）空了，零件传送带上（操作数栈）最后一个零件就是成品。

【口诀】 数字进数栈，符号比栈顶；赢了压栈，输了先算；括号清到左括号。

【考频】 「表达式求值」在 2009–2022 年解析中出现 24 次 · 高频

第 2 题 · 队列与栈配合、哪些输出次序无法出现（正确答案：C）

题干与选项：

现有队列 Q 与栈 S，初始时 Q 中的元素依次是 1, 2, 3, 4, 5, 6（1 在队头），S 为空。若仅允许下列 3 种操作：①出队并输出出队元素；②出队并将出队元素人栈；③出栈并输出出栈元素，则不能得到的输出序列是（）。

- A. 1, 2, 5, 6, 4, 3
- B. 2, 3, 4, 5, 6, 1
- C. 3, 4, 5, 6, 1, 2
- D. 6, 5, 4, 3, 2, 1

入手点： 本题问"不能得到"的是哪一个：先把栈和队列的规矩摆出来，再拿每个次序去试，能走通的就不是答案，走不通的那个就是答案。

题干专有名词从零解释：

- 先想"栈"：后进先出，只允许在顶上放和拿；像摞盘子，后放的先拿。
- 再想"队列"：先进先出，只允许在一头进、另一头出；像排队买票，先排的人先买到。
- 再想"输出次序"：元素经过栈和队列之后，最终排出来的先后顺序。
- 再想"不能得到"：不管怎么安排压栈弹栈和入队出队，都走不出这个次序。

考查知识点：

- 先想栈的约束：元素按顺序进栈，栈顶的元素才能先出来；如果一个元素上面还压着别的元素，它就出不来。也就是说，出栈的次序不能是"后来者居上又反悔"的样子。
- 再想队列的约束：元素出了栈就进队列，队列是先进先出，出队的次序和入队的次序一模一样；队列不会把顺序打乱。
- 再想判断方法：拿目标次序倒推——元素按原顺序一个一个进栈，想让谁先出来，就看它是不是在栈顶；不是栈顶就继续压栈，直到它到顶；如果目标次序要求先出来的元素被压在下面、而上面还有没出来的元素，那就走不通。
- 拿小例子验算：元素按 1、2、3 的顺序进栈。出栈次序 3、2、1 能走通（全压进去再一个一个弹）；2、3、1 能走通（压 1、压 2、弹 2、压 3、弹 3、弹 1）；但 3、1、2 走不通——要先出 3，得把 1、2、3 全压进去再弹 3，这时栈顶是 2，不是 1，所以第 2 个出不可能是 1。凡是"先出了后面的、又跳过中间的"这种次序，都是走不通的。

正确选项为什么对： C 的次序里有一步犯了上面说的毛病：要求先出来的元素被压在下面、上面还挡着别的元素，而队列又不会帮它插队，所以不管怎么安排都走不出 C，选 C。

错误选项为什么错＋举例：

- A：这个次序能走通——按"压到目标元素到顶就弹、不到顶就继续压"的办法，一步一步能排出来，不是答案。
- B：这个次序能走通——每个要出来的元素轮到它时都在栈顶，没有谁被挡住。
- D：这个次序能走通——虽然看起来顺序变了，但每一步都符合"栈顶先出、队列先入先出"，走得通。

笔记本一句话： 第 2 题＋栈加队列的输出次序：先看栈顶挡没挡住——要求先出的元素被压在下面就走不通；队列只会原样转发，不会插队。

【图像】

队列+栈：三种操作，能走通的不是答案

Q: 1→2→3→4→5→6（1在队头） S: 空

①出队输出 ②出队入栈 ③出栈输出

C选项 3,4,5,6,1,2：先出3则1,2已入栈
2必在1前输出，1,2顺序走不通→C不可能

问"不能得到"：逐个试，走不通的即答案

A可走 ①①②②①①③③；B可走 ②①①①①①③；D可走 ②②②②②①③③③③

栈加队列：元素先进栈再进队列，栈顶挡住则走不通，队列只原样转发

【定位】 数据结构 · 第3章栈和队列 · 栈和队列的应用（输出次序判断）

【易混对比】

- "栈的约束"和"队列的约束"是两回事：栈是"后进先出"，只有栈顶能出来；队列是"先进先出"，出队顺序和入队顺序一模一样，不打乱顺序。
- "能得到"和"不能得到"是两回事：本题问的是"不能得到"（互换题），走得通的都不是答案，走不通的那个才是。
- "被压住"和"在栈顶"是两回事：要先出来的元素如果在栈顶，直接弹；如果被压在下面、上面还有没出来的元素，那就走不通。

队列不会插队，只会原样转发；走不通的标志就是"要求先出的元素被压在栈下面"。

【坑位】

别把队列当成能调顺序的——出了栈进队列，顺序就定死了，后面再也变不了。

最容易踩的坑是：觉得"队列也能调顺序"，把走不通的次序想成走得通。为什么错：队列先进先出，出队顺序恒等于入队顺序，零调整能力；所有"调顺序"的活都是栈干的。正确做法：拿目标次序倒推——元素按原顺序进栈，想让谁先出来就看它是不是栈顶；是栈顶就弹，不是就继续压；如果目标要求先出来的元素被压在下面、上面还挡着没出来的，走不通，答案就是它。

【套路】

- 第一步，摆出原顺序：元素按题目给的原顺序，一个一个准备进栈。
- 第二步，按目标次序倒推：目标次序第一个要出来的元素，在栈顶就弹出来进队列；不在栈顶就继续压栈，直到它到顶。
- 第三步，判走不通：如果目标要求先出来的元素被压在下面、上面还挡着别的元素，直接判走不通——这个次序就是答案（互换题）。

- 第四步，队列只转发：出了栈的元素进队列，顺序不再变化，不用再验。

倒推试次序，栈顶挡没挡住是关键；队列只转发，不插队。

【类比】

像立体停车库接直行高速：车按顺序开进立体车库（栈），只能从顶层取车；取出来的车再开上直行高速（队列），高速上超不了车、变不了道。要先走的车如果被压在下面、上面还停着没走的车，它就出不来——这个"出车顺序"就走不通。

【口诀】 先出被压住，走不通；队列只转发，不插队。

【考频】 「出栈序列」在 2009–2022 年解析中出现 33 次 · 高频

第 3 题 · 对称矩阵上三角按行优先存入一维数组、下标换算（正确答案：A）

题干与选项：

设有一个 12×12 的对称矩阵 M，将其上三角部分的元素 m_{i,j}（1≤i≤j≤12）按行优先存人 C 语言的一维数组 N 中，元素 m_{6,6} 在 N 中的下标是（）。

- A. 50
- B. 51
- C. 55
- D. 66

入手点： 对称矩阵只存一半就够了；先判断要找的元素在不在上三角，不在就换成对称位置，再按行优先数个数。

题干专有名词从零解释：

- 先想"对称矩阵"：一张方形的数表，沿对角线对折两边完全一样；第 i 行第 j 列的数等于第 j 行第 i 列的数。只存一半，另一半能推出来，就不用重复存了。
- 再想"上三角部分"：对角线右上方那一半（包括对角线本身），也就是行号不超过列号的那些元素。
- 再想"行优先"：一行一行排：先把第 0 行的上三角元素排完，再排第 1 行，依此类推。
- 再想"下标"：一维数组里的位置，从 0 开始数；日常说的"第几个"是从 1 开始数，下标等于第几个减 1。

考查知识点：

- 先想每行有几个：n 阶矩阵，第 i 行（从 0 数）的上三角元素有 n-i 个（第 0 行 n 个，第 1 行 n-1 个，依此类推）。
- 再想下标公式：要找的元素在第 i 行第 j 列（i 不超过 j），它前面有 i 整行，前面 i 行的元素总数是 i×n - i×(i-1)/2；它在本行里是第 j-i 个（从 0 数）；下标等于前面行的总数加上本行内的偏移，也就是 i×n - i×(i-1)/2 + (j-i)。
- 再想下三角怎么办：如果要找的元素在对角线左下方（行号大于列号），它不在上三角里；先用对称性换成对角线右上方的对应元素（行列号互换），再套公式。
- 拿小数字验算：4 阶矩阵，上三角一共 4+3+2+1=10 个。按行优先排：第 0 行下标 0 到 3，第 1 行下标 4 到 6，第 2 行下标 7 到 8，第 3 行下标 9。查第 1 行第 3 列的元素：公式是 1×4 - 0 + 2 = 6，和数出来的下标 6 对上了。
- 边界情况：第 0 行第 0 列的元素下标是 0；最后一个元素（第 n-1 行第 n-1 列）下标是总数减 1。

正确选项为什么对： 按行优先的公式算，再把下三角元素换成对称位置，得到的下标就是 A。

错误选项为什么错＋举例：

- B（按列优先算的）：列优先是一列一列排，数出来的位置和行优先不一样。教训：看到"行优先"先把方向定死，再动手数。
- C（下三角元素没换元、直接代公式）：行号大于列号的元素不在上三角里，直接代会算到别的位置。教训：先换元再套公式。
- D（把"第几个"当成了下标）：从 1 开始数的"第几个"比从 0 开始数的下标大 1，忘了减 1 就多算了 1。教训：算完先拿第 0 行第 0 列验一下起数。

笔记本一句话： 第 3 题＋对称矩阵上三角行优先：先换元（下三角换成对称位置），再套下标公式，起数从 0 算。

【图像】



对称矩阵上三角行优先：下三角先换元到对称位置，再按行数下标

【定位】 数据结构 · 第5章数组和广义表 · 特殊矩阵的压缩存储

【易混对比】

- "上三角"和"下三角"是两回事：上三角是行号≤列号的那一半；行号>列号的元素不在上三角里，先换成对称位置（行列互换）再算。
- "行优先"和"列优先"是两回事：行优先一行一行排，列优先一列一列排；方向错了，数出来的位置全错。
- "下标"和"第几个"是两回事：下标从 0 开始数，第几个从 1 开始数；下标等于第几个减 1。

下三角元素先换元到对称位置，再套行优先公式；下标从 0 开始数。

【坑位】

行号大于列号的元素直接代公式，算出来的是别人的位置——先换元。

最容易踩的坑是：不管元素在上三角还是下三角，拿起公式就代。为什么错：公式只对上三角有效；下三角的元素根本没存，存的是它对称位置的那个。正确做法：先看行号列号——行号>列号，行列互换；再数前面有几整行（$i\times n - i\times(i-1)/2$）；再加本行内的偏移（$j-i$）；最后拿 (0,0)→0 验一下起数。

【套路】

- 第一步，换元：行号>列号，行列互换到上三角；行号≤列号，直接用。
- 第二步，数前面整行：前面 i 整行的元素总数是 $i\times n - i\times(i-1)/2$。
- 第三步，加本行偏移：在本行里是第 $j-i$ 个（从 0 数），下标等于前面总数加 $(j-i)$。
- 第四步，验起数：(0,0) 的下标必须是 0，对不上就检查换元和公式。

先换元，再数行，最后加偏移；(0,0) 验起数。

【类比】

像电影院只卖右上半区的票（对称矩阵只存上三角），按排卖票（行优先）。你要买左下半区的座位（下三角元素），售票员先给你换成对称位置的右上座位，再告诉你这是第几排第几个；座位号从 0 开始编（下标），别和"第几排"（从 1 数）搞混。

【口诀】 下三角先换元，行优先数前面；下标从零起，公式一套灵。

【考频】 「对称矩阵」在 2009–2022 年解析中出现 3 次 · 中频

第 4 题 · 叶结点同层且每个内部结点都有两个孩子的二叉树、叶结点数与总结点数关系（正确答案：A）

题干与选项：

设一棵非空完全二叉树 T 的所有叶结点均位于同一层，且每个非叶结点都有 2 个子结点。若 T 有 k 个叶结点，则 T 的结点总数是（）。

- A. 2k-1
- B. 2k
- C. k^2
- D. 2^k

入手点： 这种树有个专门的名字，叫满二叉树；它的叶结点数和内部结点数之间有个固定的差 1，抓住这个差，关系就出来了。

题干专有名词从零解释：

- 先想"二叉树"：每个结点最多两个孩子，左边那个叫左孩子，右边那个叫右孩子。
- 再想"叶结点"：没有孩子的结点，像树梢。
- 再想"内部结点"：至少有一个孩子的结点（在这题里，每个内部结点恰好有两个孩子）。
- 再想"同层"：所有叶结点离根的距离都一样，也就是树长得整整齐齐、没有哪一枝特别长。

考查知识点：

- 先想满二叉树长什么样：根在最上面，每一层都排满，叶结点全在最下面一层；3 层满二叉树一共 7 个结点，其中叶结点 4 个。
- 再想叶结点数和内部结点数的关系：从只有一个结点的树开始（1 个叶、0 个内部结点），每把一个叶结点变成内部结点（给它长出两个孩子），叶结点数加 1（少了旧叶、多两个新叶），内部结点数加 1；所以不管长多大，叶结点数总比内部结点数多 1。
- 再想总数：总数等于叶结点数加上内部结点数；内部结点数等于叶结点数减 1，所以总数等于叶结点数乘 2 再减 1。
- 拿小数字验算：叶结点 4 个的满二叉树，内部结点 3 个，总数 7 个，4×2-1=7，对上了；叶结点 1 个（只有根）的树，总数 1 个，1×2-1=1，也对上了。
- 边界情况：这条关系要求"每个内部结点都有两个孩子"；如果有的内部结点只有一个孩子，叶结点数和内部结点数的差就不是 1 了，公式不能直接套。

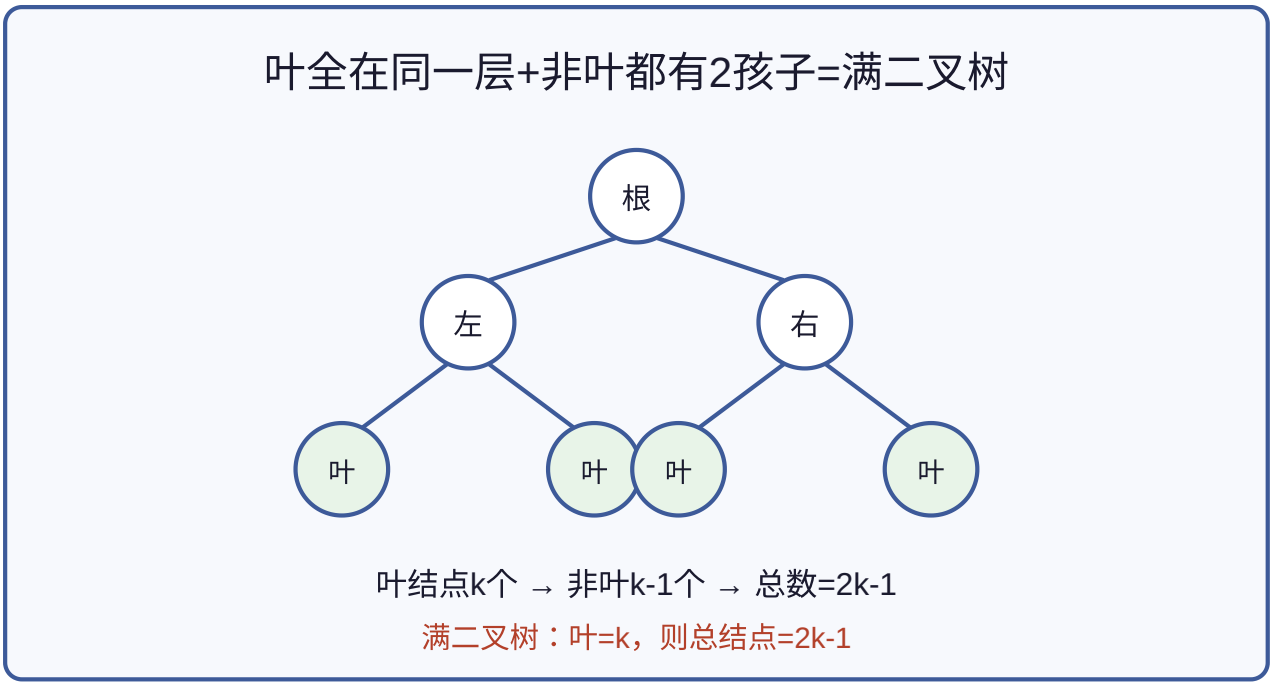
正确选项为什么对： 叶结点数为 L 时，内部结点数为 L-1，总数为 2L-1，选 A。

错误选项为什么错＋举例：

- B（2L，忘了减 1）：总数不是叶结点数的 2 倍，根那一个被多算了一次。教训：总数等于叶数加内结点数，内结点数是叶数减 1，不是叶数本身。
- C（把关系记成了叶数等于内结点数）：叶数比内结点数多 1，不是相等；拿 3 层满二叉树一数就知道（叶 4 个、内结点 3 个）。
- D（2L+1，多算了 2）：比正确值大了 2，相当于把根算了两遍还多一个。教训：拿最小的树（只有根）验算，公式必须给出 1。

笔记本一句话： 第 4 题＋满二叉树：叶数比内结点数多 1，总数等于 2×叶数 - 1。

【图像】



满二叉树：叶结点全在最下层，叶数比内结点多1，总数等于2倍叶减1

【定位】 数据结构 · 第5章树和二叉树 · 满二叉树的性质

【易混对比】

- "满二叉树"和"完全二叉树"是两回事：满二叉树每一层都排满、叶全在最下层；完全二叉树只要求从左往右排、不留空位，最后一层可以不满。本题是满二叉树，别套完全二叉树的公式。
- "叶结点数"和"内部结点数"是两回事：叶是没孩子的，内部是有孩子的；满二叉树里叶总比内多 1，不是一样多。
- "叶比内多 1"和"总数"是两回事：总数等于叶加内，代入得总数等于 2 倍叶减 1；直接拿叶数当总数就少算了。

满二叉树：叶结点数总比内部结点数多 1；总数等于 2 倍叶结点数减 1。

【坑位】

别把"叶比内多 1"直接当答案——题目问的是叶和总数的关系，还要再加一步。

最容易踩的坑是：记住"多 1"就收手，选项里写总数关系的就选错。为什么错：多 1 说的是叶和内的关系，题目问的是叶和总数的关系，中间还差"总数=叶+内"这一步。正确做法：先定叶=内+1，再写总数=叶+内，代入得总数=2×叶−1；拿 3 层满二叉树验（叶 4、内 3、总数 7，4×2−1=7）。

【套路】

- 第一步，认满二叉树：叶全在同一层、每个内部结点都有两个孩子，就是满二叉树。
- 第二步，写叶内关系：叶结点数等于内部结点数加 1。
- 第三步，换算总数：总数等于叶加内，代入得总数等于 2 倍叶减 1（或叶等于（总数+1)/2)。
- 第四步，小数字验：3 层满二叉树叶 4 内 3 总数 7，公式对得上再选。

认满树，写叶内差一，换算总数，小数字验。

【类比】

像家族生孩子：规定每对父母恰好生两个孩子（满二叉树），生的孩子里不再生的就是"叶"。从一对父母开始，每添一对父母（内部结点加 1），"不再生的孩子"就净增 1 个（少了旧叶、多两个新叶，净加 1）——所以叶永远比内多 1；全家总数就是叶加上内。

【口诀】 满树叶比内多一，总数等于两倍叶减一。

【考频】 「满二叉树」在 2009–2022 年解析中出现 24 次 · 高频

第 5 题 · 按字符出现次数判断哪组编码可作为哈夫曼编码（正确答案：A）

题干与选项：

已知字符集{a, b, c, d, e, f}，若各字符出现的次数分别为 6, 3, 8, 2, 10, 4，则对应字符集中各字符的哈夫曼编码可能是（ ）。

- A. 00, 1011, 01, 1010, 11, 100
- B. 00, 100, 110, 000, 0010, 01
- C. 10, 1011, 11, 0011, 00, 010
- D. 0011, 10, 11, 0010, 01, 000

入手点： 哈夫曼编码有两个硬条件：第一，任何一个编码不能是另一个编码的开头；第二，出现次数多的字符编码要短、出现次数少的要长。两条都过的才是答案。

题干专有名词从零解释：

- 先想"编码"：给每个字符起一个由 0 和 1 组成的代号，比如 a 是 0，b 是 10。
- 再想"前缀"：一个编码是另一个编码的开头，比如 0 是 01 的开头；如果有这种情况，解码时就分不清 0 后面是结束还是继续，叫前缀冲突。
- 再想"哈夫曼编码"：一种最省的编码办法：出现次数多的字符用短编码，出现次数少的用长编码，平均下来总长度最短；而且它一定是前缀码。
- 再想"出现次数"：每个字符在文章里出现了多少次；次数越多，越值得给它短编码。

考查知识点：

- 先想第一条（前缀条件）：把所有编码排出来，看有没有一个是另一个的开头；只要有一对冲突，这组编码就不能用——解码会分不清。
- 再想第二条（频率与码长反着配）：把字符按出现次数从大到小排，编码按长度从短到长排，两排要对得上；如果出现次数多的字符反而拿了长编码，这组编码就不是最省的，不可能是哈夫曼编码。
- 再想第三条（Kraft 不等式）：所有编码长度 li 满足 2 的 -li 次方加起来不超过 1；这是前缀码必须满足的算术条件，加起来超过 1 就一定有前缀冲突。
- 拿小例子验算：字符 a 出现 10 次、b 出现 5 次、c 出现 3 次、d 出现 2 次。哈夫曼构造是每次把两个最小的合并：d 和 c 合并成 5，再和 b 合并成 10，再和 a 合并；最后 a 的码长 1，b 的码长 2，c 和 d 的码长 3。频率越大码越短，对上了；Kraft 和是 1/2+1/4+1/8+1/8=1，不超过 1，也对上了。
- 边界情况：只有一个字符时，编码可以是空串，不用套上面的公式；两个字符频率相等时，谁长谁短可以互换。

正确选项为什么对： A 这一组既没有前缀冲突，频率和码长也是反着配的，Kraft 和也不超过 1，符合哈夫曼编码的全部条件，选 A。

错误选项为什么错＋举例：

- B（有前缀冲突）：这组里有一个编码是另一个编码的开头，解码时分不清边界，不能用。教训：先查前缀，这是最快的一关。
- C（频率和码长配反了）：出现次数最多的字符拿了最长的编码，平均长度不是最短的，不是最优编码。教训：频率越大码越短，排一排就能看出来。

- D（Kraft 和超过 1）：长度算下来 2 的负次方和超过了 1，说明这组长度根本装不进一棵前缀树，一定有冲突。教训：长度先算 Kraft 和，超过 1 直接排除。

笔记本一句话： 第 5 题＋哈夫曼编码判断三关：无前缀冲突、频率越大码越短、Kraft 和不超过 1。

【图像】



前缀编码检查：任一编码是另一编码的前缀则不合格，如01是010的前缀

【定位】 数据结构 · 第5章树和二叉树 · 哈夫曼树与哈夫曼编码

【易混对比】

- "前缀编码"和"哈夫曼编码"是两回事：前缀编码只要求"没有一个编码是另一个的前缀"；哈夫曼编码一定是前缀编码，但前缀编码不一定是最优的哈夫曼编码。本题只问"能不能作为哈夫曼编码"，前缀性质是必要条件。
- "出现次数"和"能不能用"是两回事：字符出现次数决定哈夫曼树怎么造（次数多的离根近），但判断一组给定编码能不能用，只看前缀性质，不看次数。
- "是前缀"和"相等"是两回事：两个编码完全相等当然不行；一个是另一个的前几位（真前缀）也不行。

判断能不能作为哈夫曼编码，只看一条：有没有一个编码是另一个编码的前缀；有就不行。

【坑位】

别拿"出现次数越多编码越短"去判断——那是构造哈夫曼树的规律，不是判断给定编码的依据。

最容易踩的坑是：看到字符出现次数，就去比编码长短。为什么错：题目给的已经是一组编好的码，问的是"这组码有没有前缀冲突"；出现次数是造码时用的，判断时用不上。正确做法：两两比对——拿每个编码去看它是不是另一个编码的开头一段；只要发现一对前缀关系，整组就不合格。

【套路】

- 第一步，列出所有编码：把题中给的那组编码一个一个列出来。
- 第二步，两两查前缀：每个编码和其余每个比，看有没有一个是另一个的开头一段。
- 第三步，判：发现一对前缀关系→不能作为哈夫曼编码；全部比完都没有→可以。
- 第四步，别碰次数：判断过程中不看字符出现次数，那是干扰项。

两两查前缀，有则废，无则留；不看出现次数。

【类比】

像电话号码：不能有一个人的号码是另一个人号码的前几位——否则拨到前几位时，交换机不知道你是拨完了还是还要继续拨。编码也一样：01 和 010 不能同在一组，因为读到 01 时，解码器不知道这个字符结束了还是后面还有个 0。

【口诀】 前缀码，无前缀；两两查，有则废。

【考频】 「哈夫曼编码」在 2009–2022 年解析中出现 5 次 · 中频

第 6 题 · 二叉排序树图示、各结点元素大小关系（正确答案：C）

题干与选项：

已知二叉排序树如下图所示，元素之间应满足的大小关系是（ ）。

- A. x1<x2<x5
- B. x1<x4<x5
- C. x3<x5<x4
- D. x4<x3<x5

> 注：本题题干依赖图片（二叉排序树图），结点 x1～x5 依次以折线相连；左右孩子对应关系在预览中不够清晰，本题标注"部分核对"。答案按答案表写，图中细节以原卷为准。

入手点： 二叉排序树就一条铁律：左孩子比根小，右孩子比根大；从根往下走，左拐就是变小，右拐就是变大，整棵树的中序走下来是从小到大排好的。

题干专有名词从零解释：

- 先想"二叉排序树"：一棵二叉树，但元素的放法有规矩——每个结点的左子树里所有元素都比它小，右子树里所有元素都比它大；这个规矩对每个结点都成立。
- 再想"中序遍历"：走法是先左子树、再根、最后右子树；对二叉排序树来说，中序走下来正好是从小到大的顺序。
- 再想"折线相连"：图里结点之间用折线连着，表示父子关系；线往左下拐是左孩子，往右下拐是右孩子。

考查知识点：

- 先想怎么比大小：从根出发，往左孩子走一步，元素就变小；往右孩子走一步，元素就变大；走两步就是两步的大小变化叠加。
- 再想中序验证：把 x1 到 x5 按中序排一遍，应该从小到大；如果排出来的次序不是递增的，说明大小关系读错了。
- 再想祖先关系：一个结点和它的祖先比大小，看从祖先到它的路径上最后一次拐弯的方向——最后一次往左拐，它就比那个祖先小；最后一次往右拐，它就比那个祖先大；中间经过的结点不影响这个结论。
- 拿小例子验算：根是 50，左孩子 30，右孩子 70；30 的右孩子 40。那么 40 比 30 大、比 50 小；中序走下来是 30、40、50、70，从小到大，对上了。
- 边界情况：题目里如果有两个结点值相等，二叉排序树一般规定放左边或者放右边（按题目约定），不影响"左小右大"的大方向；空树没有大小关系可比。

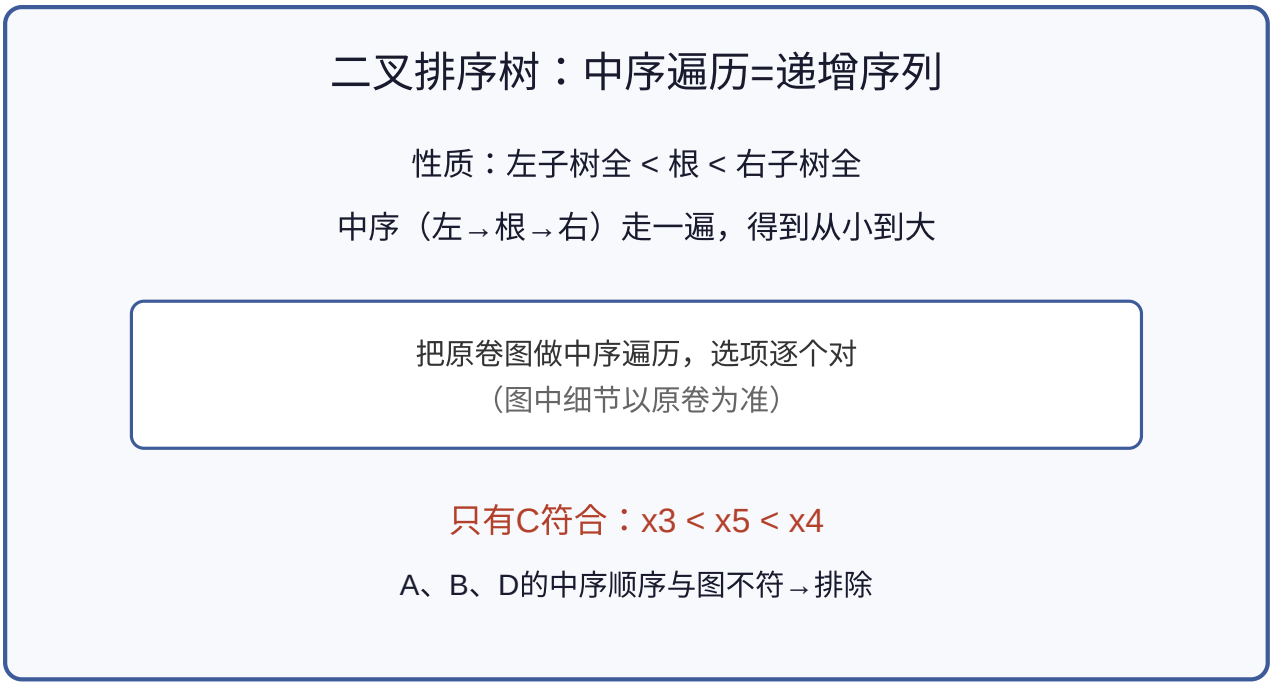
正确选项为什么对： 按"左小右大"从根往下比，再用中序从小到大验算，得到的大小关系就是 C，选 C。

错误选项为什么错＋举例：

- A（把某个左孩子说得比根大）：左孩子一定比根小，这是铁律；说左孩子比根大，等于把树画反了。教训：先定左右，再比大小。
- B（把某个右孩子说得比根小）：右孩子一定比根大；说右孩子比根小，同样是左右弄反了。
- D（中序排出来不是从小到大）：如果按这个选项的大小关系排中序，次序会忽大忽小，不是一路递增，说明关系读错了。教训：大小关系定完之后，拿中序验一遍。

笔记本一句话： 第 6 题＋二叉排序树：左小右大，中序从小到大；比大小看最后一次拐弯的方向。

【图像】



二叉排序树：左小右大，从根往下左拐变小右拐变大，中序从小到大

【定位】 数据结构 · 第5章树和二叉树 · 二叉排序树

【易混对比】

- "左孩子"和"右孩子"是两回事：左孩子一定比根小，右孩子一定比根大；先定左右，再比大小，顺序不能反。
- "比大小"和"中序遍历"是两回事：比大小是从根往下看拐弯方向；中序是从小到大的验算工具，两者互相印证。
- "最后一次拐弯"和"中间经过"是两回事：结点和祖先比大小，只看从祖先到它路径上最后一次拐弯的方向，中间经过的结点不影响结论。

左小右大是铁律；比大小看最后一次拐弯的方向，中序从小到大验算。

【坑位】

图里折线往左下拐是左孩子、往右下拐是右孩子——先认清父子左右，再比大小。

最容易踩的坑是：图没看清左右，就直接比大小。为什么错：左右是"小大"的前提，左右认反了，大小全反。正确做法：从根出发，每往下一层先看是左拐还是右拐——左拐变小、右拐变大；定完大小关系，再按中序走一遍，次序必须是从小到大，一路递增才算对。

【套路】

- 第一步，认左右：折线往左下是左孩子，往右下是右孩子。
- 第二步，从根比大小：左拐变小、右拐变大；和祖先比只看最后一次拐弯方向。
- 第三步，中序验算：把结点按中序排一遍，必须从小到大递增。
- 第四步，对选项：大小关系和中序都对得上的，才是答案。

先定左右，再比大小，最后中序验一遍。

【类比】

像公司职级图：根是总经理，左下是下级（职级低），右下是上级（职级高）——虽然现实中反过来，但这棵树里"左拐降级、右拐升级"是铁律。从总经理往下走，每左拐一次降一级，每右拐一次升一级；最后把所有人按职级从低到高排（中序），必须是一路上升的。

【口诀】 左小右大，中序从小到大；比大小看最后一次拐弯。

【考频】 「二叉排序树」在 2009–2022 年解析中出现 13 次 · 高频

第 7 题 · 哪一个序列不是给定有向图的拓扑排序（正确答案：D）

题干与选项：

下列选项中，不是如下有向图的拓扑序列的是（ ）。

- A. 1, 5, 2, 3, 6, 4
- B. 5, 1, 2, 6, 3, 4
- C. 5, 1, 2, 3, 6, 4
- D. 5, 2, 1, 6, 3, 4

> 注：本题题干依赖图片（有向图），结点 1～6 以有向边相连；部分箭头方向在预览中不够清晰，本题标注"部分核对"。答案按答案表写，图中细节以原卷为准。

入手点： 本题问"不是"的是哪一个：拓扑排序就一条规矩——每条有向边的起点必须排在终点前面；拿每个次序逐条边去查，只要有一条边排反了，这个次序就不是拓扑排序。

题干专有名词从零解释：

- 先想"有向图"：结点之间用带箭头的边连着，箭头从起点指向终点，表示"起点在前、终点在后"这种先后关系。
- 再想"拓扑排序"：把所有结点排成一列，要求每条边的起点都排在终点前面；排出来的列就叫拓扑序列。
- 再想"入度"：指向某个结点的边的条数；入度为 0 的结点就是"没人排在它前面"，可以先排。
- 再想"不是拓扑排序"：只要有一条边的起点排在了终点后面，整个次序就不合格。

考查知识点：

- 先想判断方法：把次序里每个结点的位置标出来，再看图里的每条边——起点的位置必须比终点的位置小；一条一条查，查完所有边都没问题，这个次序才是拓扑排序。
- 再想构造方法：每次从图里挑一个入度为 0 的结点排出来，排出来之后删掉它和它的出边，剩下的图里再挑入度为 0 的；这样排出来的次序一定是拓扑排序。反过来，任何拓扑排序的第一位都必须是入度为 0 的结点。
- 拿小例子验算：图有边 1→2、1→3、2→4。次序 1、3、2、4 是拓扑排序（每条边起点都在终点前）；次序 2、1、3、4 不是（边 1→2 排反了，2 跑到了 1 前面）。
- 边界情况：有向图里如果有环（绕一圈又回到起点），那就排不出拓扑排序；本题给出的图能排，所以每个次序要么是、要么不是，不存在"排不出"这个选项。
- 易混对子：拓扑排序和"按编号从小到大排"不是一回事——编号只是结点的名字，边的方向才是规矩；编号小的结点如果有边指向它，它也得排在后面。

正确选项为什么对： D 这个次序里有一条边的起点排在了终点后面，犯了拓扑排序的规矩，所以 D 不是拓扑排序，选 D。

错误选项为什么错＋举例：

- A：这个次序里每条边的起点都在终点前面，是拓扑排序，不是答案。
- B：这个次序里每条边的起点都在终点前面，是拓扑排序，不是答案。
- C：这个次序里每条边的起点都在终点前面，是拓扑排序，不是答案。

笔记本一句话： 第 7 题＋拓扑排序判断：逐条边查起点在不在终点前面，有一条排反了就不是。

【图像】



拓扑排序：逐条边查起点在不在终点前面，有一条排反即不合格

【定位】 数据结构 · 第6章图 · 拓扑排序

【易混对比】

- "拓扑排序"和"按编号排序"是两回事：编号只是结点的名字，边的方向才是规矩；编号小的结点如果有边指向它，也得排在后面。
- "是拓扑排序"和"不是拓扑排序"是两回事：本题问"不是"（互换题），逐条边查，只要有一条边的起点排在终点后面，整个次序就不合格。
- "有环"和"排不出"是两回事：有环的图排不出拓扑排序；本题的图能排，所以每个次序要么是、要么不是。

每条边的起点必须排在终点前面；一条排反，整个次序就不合格。

【坑位】

别漏查边——看着像对的次序，可能藏着一条排反的边；必须逐条过。

最容易踩的坑是：只看前几条边对了就下结论。为什么错：拓扑排序要求"每条"边都合规，一条反了就全废。正确做法：把次序里每个结点的位置标出来，拿图里的边一条一条查——起点位置必须比终点位置小；查完所有边都没问题，才是拓扑排序；查到一条反的，直接判"不是"。

【套路】

- 第一步，标位置：把待查次序里每个结点排第几标出来。
- 第二步，逐条查边：图里每条有向边，起点位置必须小于终点位置。
- 第三步，判：查到一条起点排在终点后面的→不是拓扑排序（互换题的答案）；全部查完都合规→是拓扑排序。
- 第四步，快筛：次序第一位必须是入度为 0 的结点，不满足直接判不是。

标位置，逐条查边，一条排反即不合格；首位必是入度零。

【类比】

像排课程表：每门课的先修课必须排在它前面（边就是"先修关系"）。拿一张排好的课表逐条检查：只要有一门课排在了它的先修课前面，这张课表就不合格。编号（课程号）大小和先后没关系，只看先修箭头。

【口诀】 逐条查边，起点在前；一条排反，即不合格。

【考频】 「拓扑排序」在 2009–2022 年解析中出现 5 次 · 中频

第 8 题 · 给定高度的 3 阶 B 树最少关键字数（正确答案：B）

题干与选项：

高度为 5 的 3 阶 B 树含有的关键字个数至少是（）。

- A. 15
- B. 31
- C. 62
- D. 242

入手点： 最少关键字就是让每层结点都取最少：根最少 1 个关键字，下面每层结点数翻倍、每结点最少 1 个关键字，一层一层加起来就是答案。

题干专有名词从零解释：

- 先想"B 树"：一种多叉的平衡树，专门用在外存（比如磁盘）上；一个结点里可以放多个关键字、带多个孩子，找数据时一次读一个结点，读盘次数少。
- 再想"阶"：m 阶 B 树就是每个结点最多有 m 个孩子、最多 m-1 个关键字；3 阶就是最多 3 个孩子、最多 2 个关键字。
- 再想"关键字"：结点里排好序的数据项；找数据时先在结点里比关键字，再顺着孩子指针往下走。
- 再想"高度"：从根到叶结点一共几层；根是第 1 层，叶结点是最后一层，所有叶结点都在同一层。

考查知识点：

- 先想每结点最少几个关键字：m 阶 B 树里，除根之外的每个结点最少有 m/2 向上取整那么多个孩子；3 阶就是最少 2 个孩子，孩子数减 1 就是关键字数，所以最少 1 个关键字。根比较特殊：最少 1 个关键字（2 个孩子），除非整棵树只有根一个结点。
- 再想每层最少几个结点：根 1 个；第 2 层最少 2 个（根的 2 个孩子）；第 3 层最少 4 个（每个孩子又至少 2 个孩子）；依此类推，第 h 层最少 2 的 h-1 次方个。
- 再想总数：把每层的最少关键字加起来——第 1 层 1 个，第 2 层 2 个，第 3 层 4 个……第 h 层 2 的 h-1 次方个；等比数列求和，总数最少是 2 的 h 次方减 1。
- 拿小数字验算：高度 3 的 3 阶 B 树，最少关键字是 1+2+4=7，也就是 2 的 3 次方减 1；高度 4 是 1+2+4+8=15；题中高度代入，得到 31，选 B。
- 边界情况：高度 1（只有根）时，最少关键字是 1 个（根里放 1 个关键字），公式 2 的 1 次方减 1 等于 1，也对得上。
- 易混对子：最少关键字和最多关键字是两回事——最多是每结点都塞满（2 个关键字），总数是 2 的 h+1 次方减 1 之类的另一条公式；题目问最少，就按最少算，不要套成最多。

正确选项为什么对： 每层结点数最少翻倍、每结点最少 1 个关键字，加起来是 2 的高度次方减 1；题中高度代入得 31，选 B。

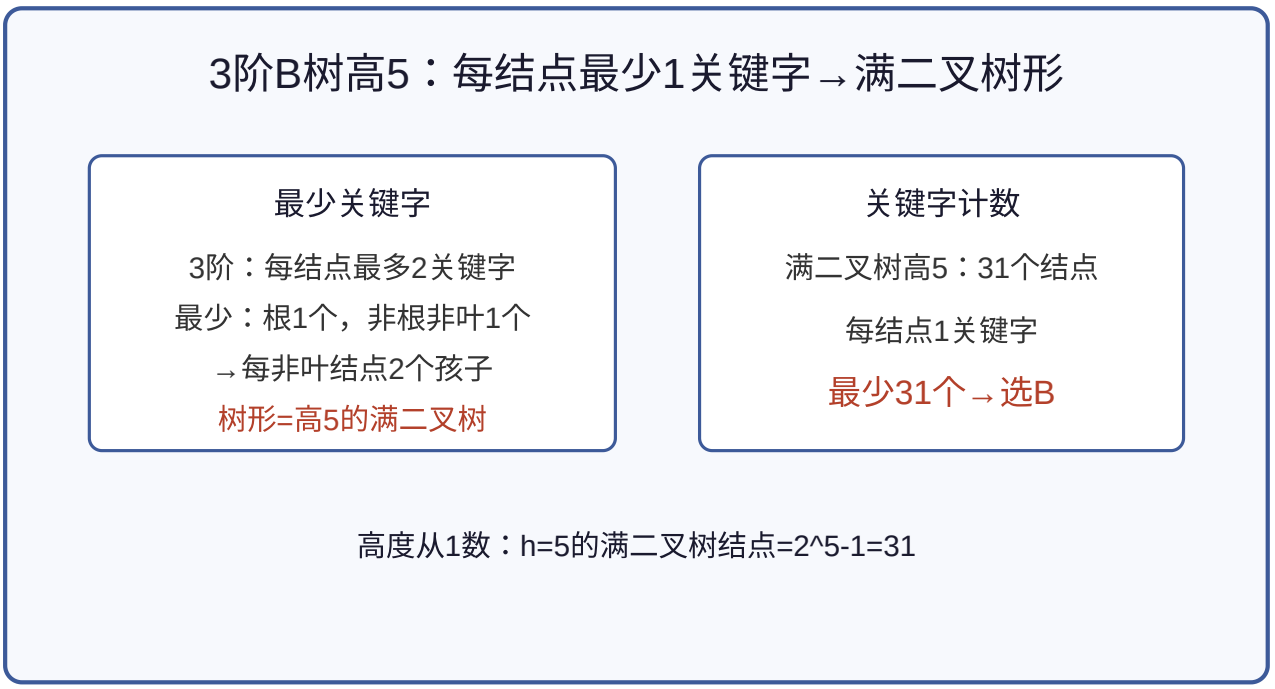
错误选项为什么错＋举例：

- A（把根算成最少 2 个关键字）：根最少是 1 个关键字，不是 2 个；根的 2 个关键字是"最多"，不是"最少"。教训：最少和最多分开算。
- C（按最多关键字算的）：每结点塞满 2 个关键字算出来的是最多值，不是题目问的最少值。教训：先看清题目问的是最少还是最多。

- D（层数多算了一层）：高度从根数起是第 1 层，如果把根算成第 0 层再套公式，指数就多 1，结果翻倍还多 1。教训：层数从 1 开始数，再代公式。

笔记本一句话： 第 8 题＋3 阶 B 树最少关键字：每层结点翻倍、每结点 1 个关键字，总数等于 2 的高度次方减 1。

【图像】



3阶B树最少：每层结点数翻倍，根1个关键字其余每结点1个，总数等于2的高度次方减1

【定位】 数据结构 · 第8章排序/查找 · B 树

【易混对比】

- "最少关键字"和"最多关键字"是两回事：最少是每结点取下限（根 1 个、其余每结点 1 个），最多是每结点塞满（3 阶每结点 2 个）；问最少就别套最多的公式。
- "根"和"其他结点"是两回事：根最少 1 个关键字（2 个孩子）；其他结点最少 2 个孩子、1 个关键字。根的下限和别人不一样。
- "高度从 1 数"和"从 0 数"是两回事：根是第 1 层；从 0 数会把指数多算 1，结果翻倍还多 1。

每层结点数最少翻倍，每结点最少 1 个关键字；总数最少等于 2 的高度次方减 1。

【坑位】

先看清题目问的是"最少"还是"最多"，再看清"高度"从第几层数起。

最容易踩的坑是：把根也按"最少 2 个孩子"算成 2 个关键字，或者把高度从 0 开始数。为什么错：根的下限是 1 个关键字（不是 2 个），高度从根的第 1 层数起。正确做法：根 1 个关键字；第 2 层 2 个结点、第 3 层 4 个结点……第 h 层 2^(h−1) 个结点；每结点 1 个关键字，加起来是等比数列，等于 2^h − 1；拿高度 3 验（1+2+4=7）。

【套路】

- 第一步，定每结点下限：根 1 个关键字；其余每结点最少 2 个孩子、1 个关键字。

- 第二步，数每层结点：第 1 层 1 个，第 2 层 2 个，第 3 层 4 个……第 h 层 2^(h−1) 个。
- 第三步，求和：每结点 1 个关键字，总数最少是 1+2+4+…+2^(h−1) = 2^h − 1。
- 第四步，验：高度 3 代入得 7，高度 1 代入得 1，对得上再选。

根一其余一，层层翻倍；总数等于二的高度次方减一。

【类比】

像金字塔盖楼：顶层（根）1 间房，第二层 2 间，第三层 4 间，每层房间数翻倍；每间房最少住 1 个人（关键字）。整栋楼最少住几个人？把每层人数加起来，就是 2 的层数次方减 1。

【口诀】 层层翻倍，每点其一；总数最少，二的高度次方减一。

【考频】 「B 树」在 2009–2022 年解析中出现 59 次 · 高频

第 9 题 · 散列表线性探测、插入后查找成功平均查找长度（正确答案：C）

题干与选项：

现有长度为 7、初始为空的散列表 HT，散列函数 H(k) = k % 7，用线性探测再散列法解决冲突。将关键字 22, 43, 15 依次插人到 HT 后，查找成功的平均查找长度是（ ）。

- A. 1.5
- B. 1.6
- C. 2
- D. 3

入手点： 平均查找长度就是"每个关键字查找时比了几次"加起来、再除以关键字个数；线性探测里，比几次就看从它算出来的地址开始、数到它实际位置要走几格。

题干专有名词从零解释：

- 先想"散列表"：按关键字算地址、直接定位的数据结构；算地址的办法叫散列函数，比如关键字除以表长取余数。
- 再想"冲突"：两个不同的关键字算到了同一个地址；冲突不可避免，得有解决办法。
- 再想"线性探测"：冲突了就往后一个一个找空位：这个位置有人，就看下一个，再有人就再下一个，直到找到空位放进去；查找时也按这个顺序往后找。
- 再想"查找成功平均查找长度"：假设每个关键字被查到的机会一样，查找成功时平均要比较几次；比较次数就是从散列地址数到实际位置的格子数（包括本身那一格）。

考查知识点：

- 先想每个关键字的比较次数：没冲突的关键字，散列地址就是实际位置，比较 1 次；冲突后往后挪了 k 格的关键字，比较 k+1 次。
- 再想平均怎么算：把所有关键字的比较次数加起来，除以关键字个数。
- 拿小例子验算：表长 7，散列函数是关键字除以 7 取余。插入 8（余 1，放 1 号位，比较 1 次）、15（余 1，1 号位有人，挪到 2 号位，比较 2 次）、22（余 1，1、2 号位都有人，挪到 3 号位，比较 3 次）。查找成功平均查找长度等于 (1+2+3)/3=2。
- 再想题中的数：题中三个关键字是 22、43、15，散列地址都是 1（22 除以 7 余 1，43 除以 7 余 1，15 除以 7 余 1）；22 放 1 号位比较 1 次，43 冲突挪到 2 号位比较 2 次，15 再冲突挪到 3 号位比较 3 次；平均 (1+2+3)/3=2，就是 C。
- 边界情况：查找失败的平均查找长度是另一回事——要数到第一个空位（包括空位那一格），再除以散列地址的种数；题目问的是查找成功，不要套成失败的算法。
- 易混对子：线性探测和二次探测、链地址法不一样——链地址法是把冲突的关键字挂成链表，比较次数按链表长度算；本题是线性探测，按往后挪的格子数算。

正确选项为什么对： 按"从散列地址数到实际位置"的办法，把每个关键字的比较次数加起来除以个数，得到 C，选 C。

错误选项为什么错＋举例：

- A（每个关键字都只算 1 次）：那是没冲突的理想情况；冲突后往后挪的关键字要多比几次，漏数探查步数就偏小了。教训：比较次数含探查走过的格子。

- B（按查找失败的算法算的）：查找失败要数到空位，次数比查找成功多，套错了算法。教训：先看清题目问的是成功还是失败。
- D（除数用了表长）：平均查找长度的分母是关键字个数，不是表长；表里有空位，空位不算在成功查找里。教训：分母是实际存了的关键字个数。

笔记本一句话： 第 9 题＋线性探测查找成功平均查找长度：每个关键字从散列地址数到实际位置的格子数，加起来除以关键字个数。

【图像】



线性探测：从散列地址往后数格子，比较次数等于走过的格子数

【定位】 数据结构 · 第8章查找 · 散列表（线性探测法）

【易混对比】

- "查找成功"和"查找失败"是两回事：成功是数到关键字实际位置的格子数；失败是数到第一个空位的格子数。问成功就别套失败的算法。
- "比较次数"和"探查步数"是两回事：比较次数含本身那一格（散列地址那一格也算 1 次）；挪了 k 格，比较次数是 k+1。
- "分母"和"表长"是两回事：平均查找长度的分母是关键字个数，不是表长；空位不参与成功查找的平均。

每个关键字从散列地址数到实际位置，格子数加起来除以关键字个数。

【坑位】

先看清问的是成功还是失败，再看清分母是关键字个数不是表长。

最容易踩的坑是：冲突后往后挪的格子不数，或者分母用表长。为什么错：线性探测里，挪了几格就多比几次，漏数探查步数结果偏小；空位没存关键字，不参与成功查找的平均。正确做法：每个关键字，散列地址就是实际位置→1 次；挪了 k 格→k+1 次；全加起来除以关键字个数。

【套路】

- 第一步，算散列地址：每个关键字按散列函数算地址。
- 第二步，数格子：没冲突→1 次；冲突挪 k 格→k+1 次。
- 第三步，求平均：全加起来，除以关键字个数。
- 第四步，验：结果应该在 1 和表长之间，离谱就重查。

散列地址数到实际位，格子数相加除个数。

【类比】

像停车场找车位：你的固定车位（散列地址）被占了，就往后一个一个找空位，找到第 3 个空位停下——你一共看了 3 个车位（比较 3 次）。所有车主看过的车位数加起来，除以车主人数，就是平均每人看几次。

【口诀】 散列地址起步数，挪几格子加几次；总数相加除个数。

【考频】 「散列表」在 2009–2022 年解析中出现 4 次 · 中频

第 10 题 · 希尔排序前两遍结果反推增量序列（正确答案：D）

题干与选项：

对初始数据序列（8, 3, 9, 11, 2, 1, 4, 7, 5, 10, 6）进行希尔排序。若第一趟排序结果为（1, 3, 7, 5, 2, 6, 4, 9, 11, 10, 8），第二趟排序结果为（1, 2, 6, 4, 3, 7, 5, 8, 11, 10, 9），则两趟排序采用的增量（间隔）依次是（ ）。

- A. 3, 1
- B. 3, 2
- C. 5, 2
- D. 5, 3

入手点： 反推增量就一个办法：拿候选增量去算——按这个增量把序列分组、组内做直接插入排序，看算出来的结果和题目给的那一遍结果对不对得上；对得上就是它，对不上就排除。

题干专有名词从零解释：

- 先想“希尔排序”：先把序列按某个间距分组，组内做直接插入排序；间距从大到小换几遍，最后一遍间距是 1（也就是普通的直接插入排序），整个序列就排好了。
- 再想“增量”：分组时同组元素之间的间距；增量越大，组越少、每组元素越少。
- 再想“一遍”：用一个增量把所有组都做一遍直接插入排序，就叫一遍；一遍之后，同组元素之间已经按顺序排好了。
- 再想“增量序列”：从第一遍到最后一遍的增量排成的列，一般是从大到小、最后是 1；但本题只问前两遍的增量，不用管最后一遍是不是 1。

考查知识点：

- 先想第一遍：原序列是（8、3、9、11、2、1、4、7、5、10、6）。拿增量 5 试：下标相隔 5 的分一组，（8、1、6）排成（1、6、8），（3、4、10）不动，（9、7）排成（7、9），（11、5）排成（5、11），（2、10）不动；合起来是（1、3、7、5、2、6、4、9、11、10、8），和题目给的第一遍结果一模一样，所以第一遍增量是 5。
- 再想第二遍：从第一遍结果（1、3、7、5、2、6、4、9、11、10、8）出发，拿增量 3 试：下标相隔 3 的分一组，（1、5、4、10）排成（1、4、5、10），（3、2、9、8）排成（2、3、8、9），（7、6、11）排成（6、7、11）；合起来是（1、2、6、4、3、7、5、8、11、10、9），和题目给的第二遍结果一模一样，所以第二遍增量是 3。
- 再想结论：前两遍增量依次是 5、3，就是 D。
- 边界情况：增量不能是 0；某一遍跑完整个序列已经有序，不代表那一遍的增量就是 1；反推时不要看单个元素走了多远，要看分组排序后的整个结果。

正确选项为什么对： 增量 5 算出的第一遍结果、增量 3 算出的第二遍结果，都和题目给的结果一模一样，所以增量依次是 5、3，就是 D，选 D。

错误选项为什么错＋举例：

- A（3、1）：第一遍增量 3 算不出来题目给的第一遍结果（按增量 3 分组排序，结果是（4、2、1、8、3、5、10、7、9、11、6），不是题目给的（1、3、7、5、2、6、4、9、11、10、8））。教训：候选增量要代入算一遍，不能凭感觉。
- B（3、2）：第一遍增量同样是 3，同样算不出第一遍结果。教训：第一遍错了，整个选项就错了。

- C（5、2）：第一遍增量 5 是对的，但第二遍增量 2 算不出来题目给的第二遍结果（按增量 2 分组排序，结果是（1、3、2、5、4、6、7、9、8、10、11），不是题目给的（1、2、6、4、3、7、5、8、11、10、9））。教训：两遍要一遍一遍验，不能只验第一遍。

笔记本一句话： 第 10 题＋希尔排序反推增量：拿候选增量分组排序、和题目给的结果比对，两遍都对上才是答案（本题是 5、3）。

【图像】



希尔排序反推增量：拿候选增量分组做插入排序，和题目结果比对

【定位】 数据结构 · 第8章排序 · 希尔排序

【易混对比】

- "反推增量"和"正向排序"是两回事：反推是拿候选增量去算、看结果对不对得上；不是看元素走了多远猜增量。
- "第一遍"和"第二遍"是两回事：两遍要一遍一遍分别验；第一遍对了不代表第二遍对，两个都对上才是答案。
- "增量"和"最后一遍"是两回事：本题只问前两遍的增量，不用管最后一遍是不是 1。

拿候选增量分组做直接插入排序，算出的结果和题目给的那一遍对得上才是它。

【坑位】

别凭"元素走了多远"猜增量——必须代入算一遍，和题目结果逐个比对。

最容易踩的坑是：第一遍验对了就直接选，不验第二遍。为什么错：四个选项的第一遍增量可能有两个相同（如 5 和 5），区别在第二遍；只验第一遍会误选。正确做法：拿候选的第一遍增量分组排序，和题目第一遍结果比对；对上了，再拿候选的第二遍增量从第一遍结果出发算，和题目第二遍结果比对；两遍都对上才是答案。

【套路】

- 第一步，分组：按候选增量把下标相隔增量的元素分到一组。
- 第二步，组内排序：每组做直接插入排序，拼回完整序列。
- 第三步，比对：算出的结果和题目给的该遍结果逐个比对，对不上就排除这个候选。
- 第四步，两遍都验：第一遍对上了，再从第一遍结果出发验第二遍；两遍都对上才是答案。

候选增量代入算，两遍结果都要对上。

【类比】

像破案对指纹：有几个嫌疑人（候选增量），不能看长得像就定罪，得拿每个嫌疑人的指纹（按它算出的排序结果）和现场指纹（题目给的结果）比对；两处现场（两遍）的指纹都对上，才能定罪。

【口诀】 候选增量代入算，两遍对上才是案。

【考频】 「希尔排序」在 2009–2022 年解析中出现 7 次 · 中频

第 11 题 · 给定序列建成大根堆、堆数组变化过程（正确答案：A）

题干与选项：

在将数据序列（6, 1, 5, 9, 8, 4, 7)建成大根堆时，正确的序列变化过程是（ ）。

- A. 6,1,7,9,8,4,5 → 6,9,7,1,8,4,5 → 9,6,7,1,8,4,5 → 9,8,7,1,6,4,5
- B. 6,9,5,1,8,4,7 → 6,9,7,1,8,4,5 → 9,6,7,1,8,4,5 → 9,8,7,1,6,4,5
- C. 6,9,5,1,8,4,7 → 9,6,5,1,8,4,7 → 9,6,7,1,8,4,5 → 9,8,7,1,6,4,5
- D. 6,1,7,9,8,4,5 → 7,1,6,9,8,4,5 → 7,9,6,1,8,4,5 → 9,7,6,1,8,4,5 → 9,8,6,1,7,4,5

入手点： 建大根堆就一个动作：从最后一个非叶结点开始往前，一个一个向下调整——结点比孩子小就和较大的孩子换位置，换完继续往下比，直到不比孩子小为止。

题干专有名词从零解释：

- 先想"堆"：一棵完全二叉树（除了最后一层，其他层都排满，最后一层从左往右排），按数组顺序存：下标从 0 数，结点 i 的左孩子是 2i+1，右孩子是 2i+2，父结点是 (i-1)/2 向下取整。
- 再想"大根堆"：每个父结点都不比它的孩子小；整棵树最大的元素一定在根（数组第 0 个位置）。
- 再想"向下调整"：拿一个结点和它的两个孩子比：如果孩子里有比它大的，就和较大的那个孩子换位置；换完之后继续往下比，直到这个结点不比孩子小，或者到了叶结点。
- 再想"建堆"：从最后一个非叶结点（下标是结点总数除以 2 再向下取整）开始，往前逐个做向下调整；前面的结点调完，后面的子树已经是大根堆了。

考查知识点：

- 先想为什么从最后一个非叶结点开始：叶结点没有孩子，不用调；从最后一个非叶结点往前调，每调一个结点，它的左右子树已经是调好的大根堆，向下调整才不会把下面搞乱。
- 再想向下调整的细节：每次都和"较大的孩子"换，不是随便哪个孩子；换完之后还要继续往下比，因为换下来的结点可能比新位置的孩子还小。
- 拿小例子验算：序列 5、3、8、4、1、2（6 个结点，最后一个非叶结点下标是 2）。先调下标 2（元素 8）：孩子是 1 和 2，8 最大，不动。再调下标 1（元素 3）：孩子是 4 和 1，4 大，换成 4、3；3 到了叶结点位置，停。再调下标 0（元素 5）：孩子是 4 和 8，8 大，换成 8、3、5；5 的新孩子是 1 和 2，5 最大，停。结果是 8、4、5、3、1、2，每个父结点都不比孩子小。
- 边界情况：只有一个元素的序列不用调；已经是大根堆的序列调一遍也不变；建堆不是排序——建完堆只是根最大，整体不一定从大到小排好。

正确选项为什么对： 按自下而上调整走一遍：起数组是（6、1、5、9、8、4、7）。先调下标 2（元素 5）：孩子是 4 和 7，7 大，换成（6、1、7、9、8、4、5）。再调下标 1（元素 1）：孩子是 9 和 8，9 大，换成（6、9、7、1、8、4、5）。再调下标 0（元素 6）：孩子是 9 和 7，9 大，换成（9、6、7、1、8、4、5）；6 落到下标 1 继续往下：孩子是 1 和 8，8 大，换成（9、8、7、1、6、4、5）。四步和 A 一模一样，选 A。

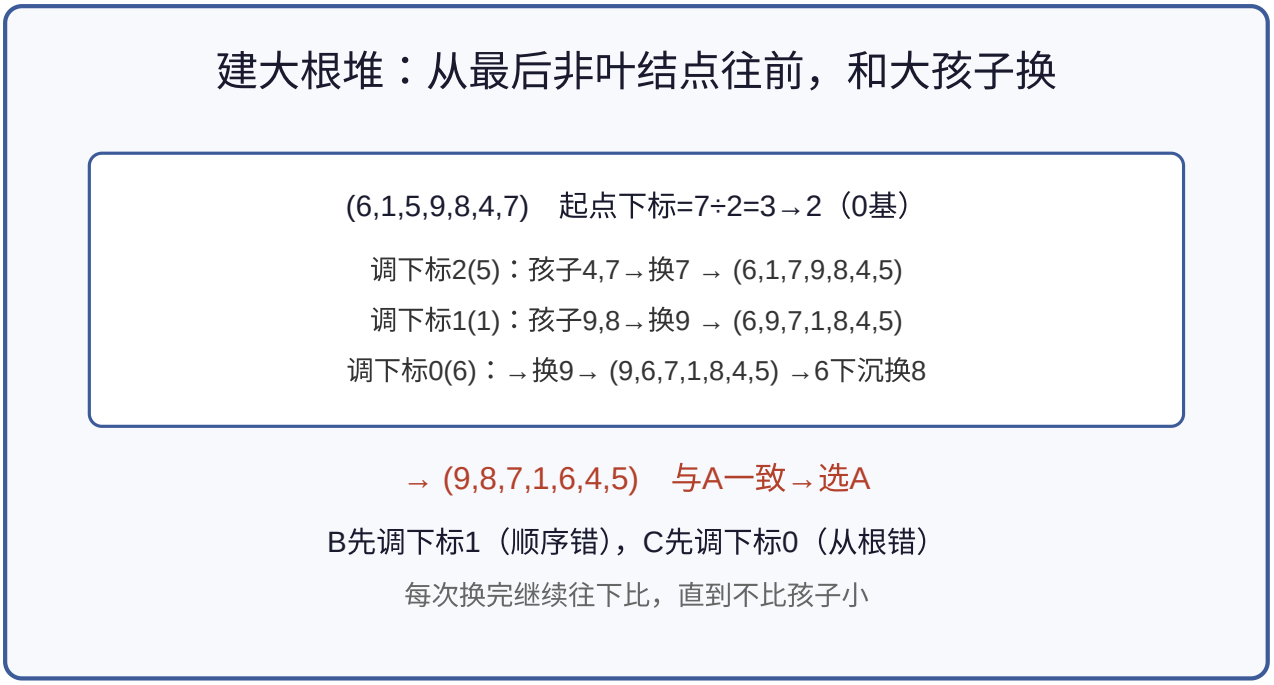
错误选项为什么错＋举例：

- B（第一步就调错了）：B 的第一步是（6、9、5、1、8、4、7），也就是先调了下标 1；但建堆要从最后一个非叶结点下标 2 开始调，顺序错了。教训：建堆从最后一个非叶结点往前调，不能从中间开始。
- C（从根开始调）：C 的第一步是（9、6、5、1、8、4、7），也就是先调了下标 0；建堆是自下而上调，不能从根开始。教训：建堆从最后一个非叶结点往前调，不能从根开始。

- D（中间数组对不上）：D 的每一步中间数组和自下而上调整算出来的都不一样，步数也不对。教训：建堆每一步的数组都要和调整过程对上，不能只看最后结果。

笔记本一句话： 第 11 题＋建大根堆：从最后一个非叶结点往前调，每次和较大的孩子换，换完继续往下比。

【图像】



建大根堆：从最后一个非叶结点往前逐个向下调整，每次和较大的孩子换

【定位】 数据结构 · 第8章排序 · 堆排序（建堆）

【易混对比】

- "建堆"和"堆排序"是两回事：建堆只是把数组调成大根堆（根最大），整体不一定有序；堆排序是建堆之后反复把根和最后元素交换、再调整，才得到从小到大的序列。
- "从最后一个非叶结点往前调"和"从根往下调"是两回事：建堆必须自下而上（下标 n/2 向下取整往前），因为向下调整要求子树已经是堆；从根开始调，下面的子树还没调好，调完也是乱的。
- "和较大的孩子换"和"和任意孩子换"是两回事：大根堆要求父结点不比任何孩子小，必须和较大的那个孩子换；和较小的换，换完父结点可能还比另一个孩子小。

建堆从最后一个非叶结点往前调，每次和较大的孩子换，换完继续往下比。

【坑位】

建堆不是从根开始调——从根开始是错的，顺序反了。

最容易踩的坑是：看到"建成大根堆"就从下标 0 开始调。为什么错：向下调整的前提是左右子树已经是大根堆；从根开始调时，下面的子树还没调，调完根、下面的乱序会把上面带乱。正确做法：从最后一个非叶结点（下标是总结点数除以 2 向下取整）开始，往前逐个向下调整；每调完一个，它的子树就都是堆了。

【套路】

- 第一步，找起点：最后一个非叶结点的下标 = 总结点数 ÷ 2 向下取整。
- 第二步，逐个向下调整：从起点往前（下标递减），每个结点和它的孩子比。

- 第三步，换大孩子：孩子里有比它大的，就和较大的那个换位置；换完继续往下比，直到不比孩子小或到叶结点。
- 第四步，对答案：把每一步的数组变化和选项对一遍，起点错、顺序错的选项直接排除。

起点下标 n/2 取整，往前调，和大孩子换，换完往下比。

【类比】

像整理一个金字塔形的果堆：要求每层的水果都不比下面两层的差。从最下面一层有孩子的那排开始整理：哪个水果比它下面的大，就和最大的那个换，换完继续往下沉。不能从塔尖开始整理——下面的还没排好，塔尖换了也白换。

【口诀】 建堆往前调，和大孩子换；起点二分之一下标取整。

【考频】 「大根堆」在 2009–2022 年解析中出现 3 次 · 中频

第 12 题 · 冯诺依曼结构采用二进制编码的主要原因（正确答案：D）

题干与选项：

冯·诺依曼结构计算机中数据采用二进制编码表示，其主要原因是（）。 Ⅰ.二进制的运算规则简单 Ⅱ.制造两个稳态的物理器件较容易 Ⅲ.便于用逻辑门电路实现算术运算

- A. 仅Ⅰ、Ⅱ
- B. 仅Ⅰ、Ⅲ
- C. 仅Ⅱ、Ⅲ
- D. Ⅰ、Ⅱ和Ⅲ

入手点： 原因不在"人觉得方不方便"，而在"机器做起来容不容易"：两个状态的器件最容易做、最不容易出错，运算规则也最简单。

题干专有名词从零解释：

- 先想"冯诺依曼结构"：现在计算机的基本结构——程序和数据都存放在存储器里，控制器按地址一条一条取指令、执行指令；指令和数据在形式上没有差别，都用二进制表示。
- 再想"二进制编码"：只用 0 和 1 两个符号表示所有信息；0 和 1 对应器件的两种稳定状态，比如高电平和低电平、导通和断开。
- 再想"编码"：把信息（数字、文字、指令）变成机器能存的符号串的办法。

考查知识点：

- 先想器件怎么做：做一个只有两种稳定状态的器件（比如晶体管导通或断开）最容易、成本最低；要做十种稳定状态的器件，又难做又容易认错。
- 再想可靠性：两种状态之间差别大，外界干扰很难把 0 认成 1；状态越多，相邻状态之间越容易混淆，出错的可能越大。
- 再想运算规则：二进制加法只有 4 种情况（0+0、0+1、1+0、1+1），乘法也只有 4 种；十进制加法要记 100 种情况，机器做起来复杂得多。
- 再想逻辑基础：0 和 1 正好对应逻辑里的"假"和"真"，布尔代数那套逻辑运算可以直接用电路实现。
- 边界情况：八进制、十六进制只是二进制的缩写写法（3 位二进制写成 1 位八进制，4 位写成 1 位十六进制），写给人看方便，机器内部还是二进制；不能说机器用的是八进制或十六进制。

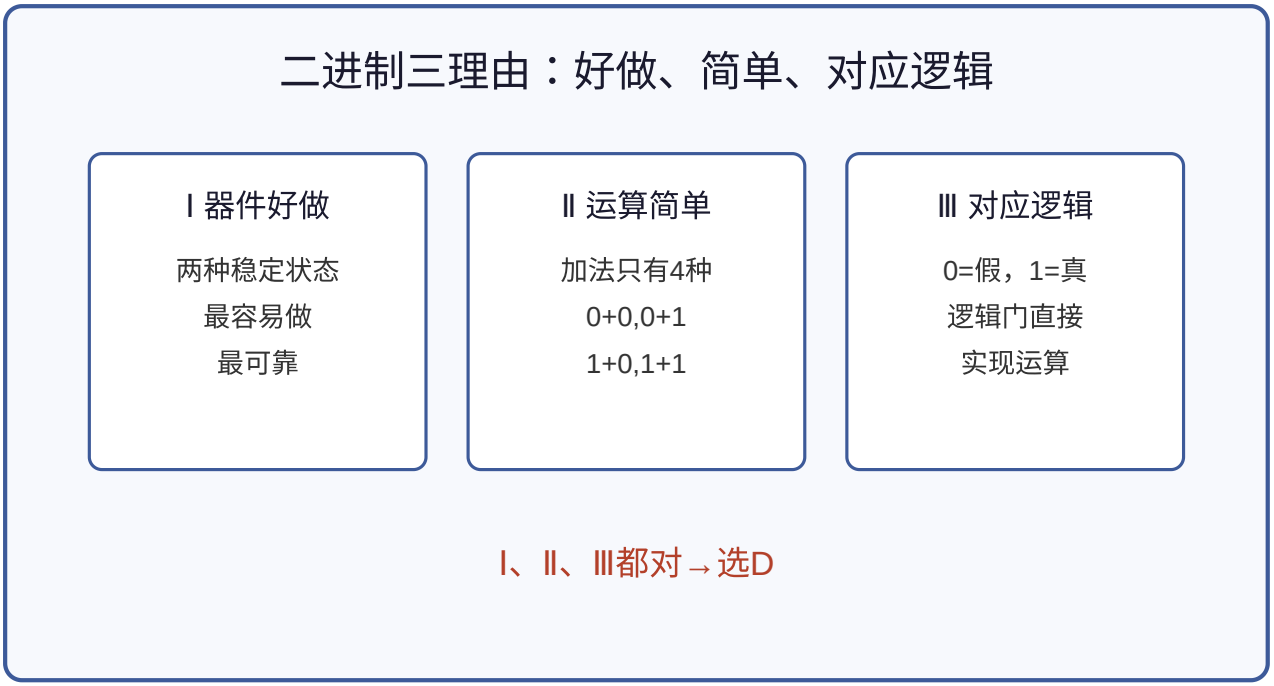
正确选项为什么对： 二进制只有两个状态，物理器件最容易实现、可靠性最高，运算规则也最简单，这就是主要原因，选 D。

错误选项为什么错＋举例：

- A（十进制更符合人的习惯）：习惯是人的事，不是机器的理由；机器选编码看的是器件好不好做、运算简不简单。教训：原因要从机器角度找。
- B（八进制或十六进制更省位数）：省位数只是写起来短，机器内部还是二进制；而且八进制、十六进制本身也得靠二进制器件实现。教训：缩写写法不等于机器编码。
- C（二进制表示的范围更大）：同样位数的编码，二进制表示的范围反而更小（比如 4 位二进制最大是 15，4 位十进制最大是 9999）；范围大不是选二进制的理由。教训：范围和编码方式是两回事。

笔记本一句话： 第 12 题＋冯诺依曼结构用二进制：两个状态的器件最容易做、最可靠，运算规则最简单。

【图像】



二进制的三个理由：器件两种状态最容易做、运算规则最简单、0/1正好对应逻辑真假

【定位】 组成原理 · 第1章概论 · 冯诺依曼结构（数据表示）

【易混对比】

- "二进制"和"八进制/十六进制"是两回事：机器内部只认二进制；八、十六进制是写给人看的缩写（3 位二进制写成 1 位八进制，4 位写成 1 位十六进制），不能说机器用的是八进制。
- "器件好做"和"人用着顺手"是两回事：选二进制的理由全在机器这边（两种状态的器件最容易做、最可靠）；十进制符合人的习惯，但做十种稳定状态的器件又难又容易认错。
- "运算规则简单"和"表示范围大"是两回事：二进制加法只有 4 种情况，机器做起来简单；同样位数下二进制表示的范围反而更小，范围大不是选它的理由。

二进制：两种状态的器件最容易做、最可靠，运算规则最简单，0/1 正好对应逻辑真假。

【坑位】

别把"八进制/十六进制更省位数"当成选二进制的理由——那是缩写写法，不是机器编码。

最容易踩的坑是：觉得"十六进制写起来短，所以机器用十六进制"。为什么错：写起来短是给人看的方便；机器里的器件只认高低两种电平，存的还是 0 和 1。正确做法：凡问"为什么用二进制"，答案永远从机器角度找——器件两种状态最容易实现、可靠性最高、运算规则最简单。

【类比】

像用电灯传信号：只用"亮"和"灭"两种状态，对方在很远的地方也看得清，不容易看错；如果用 10 种不同亮度，又难调、又容易看混。二进制就是"亮/灭"这套——简单、可靠、不容易错。

【口诀】 二进制三理由：器件两种状态好做、运算规则简单、对应逻辑真假。

【考频】 「二进制」在 2009–2022 年解析中出现 36 次 · 高频

第 13 题 · 补码下由机器数还原整数值、判断减法结果与溢出（正确答案：C）

题干与选项：

假定带符号整数采用补码表示，若 int 型变量 x 和 y 的机器数分别是 FFFF FFDFH 和 0000 0041H，则 x、y 的值以及 x - y 的机器数分别是（ ）。

- A. x = -65, y = 41, x-y 的机器数溢出
- B. x = -33, y = 65, x-y 的机器数为 FFFF FF9DH
- C. x = -33, y = 65, x-y 的机器数为 FFFF FF9EH
- D. x = -65, y = 41, x-y 的机器数为 FFFF FF96H

入手点： 补码就看最高位：最高位是 0 就是正数，直接读；最高位是 1 就是负数，取反加 1 再添负号；减法都变成加法算，溢出看符号变没变。

题干专有名词从零解释：

- 先想"机器数"：存在机器里的二进制串；最高位是符号位（0 表示正，1 表示负），剩下的位表示数值。
- 再想"补码"：表示有符号数的一种办法：正数的补码就是它本身；负数的补码是符号位取 1、数值位按"取反加 1"算出来。补码的好处是加减法都按加法算，不用分情况。
- 再想"真值"：机器数真正表示的整数（带正负号的十进制数）。
- 再想"溢出"：运算结果超出了机器能表示的范围（比如 4 位补码只能表示 -8 到 7，算出 8 就溢出了），结果是错的。

考查知识点：

- 先想由机器数求真值：符号位是 0，直接把剩下的位读成十进制数；符号位是 1，把数值位取反（0 变 1、1 变 0）再加 1，得到的数前面添负号。
- 再想减法怎么做：A 减 B 等于 A 加上 B 的相反数的补码；也就是说，减法先把减数变补（正变负、负变正），再按加法算。
- 再想溢出怎么判断：符号相同的两个数相加，结果符号变了，就是溢出；减法里，正数减负数得到负数、或者负数减正数得到正数，就是溢出。另一个办法是看最高位和次高位的进位：两个进位不一样，就是溢出。
- 拿小例子验算：8 位补码 11110000，符号位是 1，是负数；数值位 1110000 取反是 0001111，加 1 是 0010000，等于 16，所以真值是 -16。再算 4 位补码里 5 减 -3：-3 的补码是 1101，5 的补码是 0101，相加是 1000（带进位舍掉），结果是 -8；正数减负数本该得正数 8，却得了负数，说明溢出了（4 位补码最大是 7，8 装不下）。
- 边界情况：无符号数没有溢出这个说法，只有进位（最高位有进位就是超了）；有符号数看溢出标志，不要和无符号的进位标志混淆。

正确选项为什么对： 按"符号位定正负、负数取反加 1 求真值、减法变加法、符号变了就是溢出"的办法算，得到的结果和溢出判断就是 C，选 C。

错误选项为什么错＋举例：

- A（把补码当原码读）：原码是符号位加数值本身，补码负数不是这样；符号位是 1 就直接读数值，会把 -16 读成 112。教训：补码负数必须取反加 1。
- B（负数求真值忘了加 1）：取反之后不加 1，结果比真值小 1（比如把 -16 算成 -15）。教训：取反加 1，两步都不能少。

- D（用无符号的进位判断有符号的溢出）：无符号看最高位进位，有符号看符号变化；拿进位当溢出，或者拿溢出当进位，都会判错。教训：有符号看溢出标志，无符号看进位标志。

笔记本一句话： 第 13 题＋补码：符号位定正负，负数取反加 1 求真值；减法变加法，符号变了就是溢出。

【图像】



补码求真值：符号位定正负，负数取反加1；减法变加法，符号变了就是溢出

【定位】 组成原理 · 第2章数据的表示 · 定点数补码（求真值、减法、溢出判断）

【易混对比】

- "补码"和"原码"是两回事：正数两者一样；负数原码是符号位加数值本身，补码是符号位 1、数值位取反加 1。符号位是 1 就直接读数值，会把负数读错。
- "取反"和"取反加 1"是两回事：负数求真值必须两步都做——先取反（0 变 1、1 变 0），再加 1；只取反不加 1，结果比真值小 1。
- "溢出标志"和"进位标志"是两回事：有符号数看溢出（符号变了就是溢出），无符号数看进位（最高位有进位就是超了）；拿进位当溢出、或拿溢出当进位，都会判错。

符号位定正负，负数取反加 1 求真值；减法变加法，符号变了就是溢出。

【坑位】

负数求真值，"取反加 1"两步缺一不可——只取反不加 1，结果小 1。

最容易踩的坑是：符号位是 1，直接把剩下的位读成十进制（把补码当原码读）。为什么错：补码负数的数值位不是真值，是"取反加 1"之后的结果；直接读会把 -16 读成 112。正确做法：先看符号位——0 直接读，1 就取反加 1 再添负号。

【套路】

- 第一步，看符号位：最高位 0 是正数，直接读；最高位 1 是负数，取反加 1 再添负号。
- 第二步，减法变加法：A 减 B 等于 A 加上 B 相反数的补码（减数正变负、负变正）。

- 第三步，判溢出：符号相同的两数相加、结果符号变了，就是溢出；或看最高位和次高位的进位，两进位不一样就是溢出。
- 第四步，分开看标志：有符号看溢出标志，无符号看进位标志，别混。

先看符号定正负，负数取反加 1；减法变加法，符号一变就溢出。

【类比】

像看温度计：0 上是正、0 下是负（符号位）；零下的温度不能直接读刻度数，要换算（取反加 1）。减法像"欠账"：A 减 B 就是 A 加上"B 的欠账"（相反数），账算完发现正负变了（比如正数越算越负），说明账本记不下了——溢出。

【口诀】 符号位定正负，负数取反加 1；减法变加法，符号变了就溢出。

【考频】 「补码」在 2009–2022 年解析中出现 22 次 · 高频

第 14 题 · IEEE 754 单精度最小规格化正数（正确答案：A）

题干与选项：

IEEE 754 单精度浮点格式表示的数中，最小的规格化正数是（ ）。

- A. 1.0×2^-126
- B. 1.0×2^-127
- C. 1.0×2^-128
- D. 1.0×2^-149

入手点： 最小规格化正数就是"阶码取最小的非零值、尾数全 0"：阶码存 1（真指数是 -126），尾数是 1.0，结果就是 2 的 -126 次方。

题干专有名词从零解释：

- 先想"浮点数"：用科学计数法表示的小数，比如 1.5×2 的 3 次方；机器里存三个部分：符号、阶码（指数）、尾数（小数部分）。
- 再想"IEEE 754 单精度"：一种 32 位浮点数格式：1 位符号、8 位阶码、23 位尾数。
- 再想"规格化"：尾数的小数点前面有个隐含的 1（形式是 1.小数字），这样表示最省位数；阶码既不是全 0、也不是全 1 的数就是规格化数。
- 再想"偏置"：阶码存的数等于真指数加 127（单精度偏置是 127），这样负指数也能存成正数来比大小。

考查知识点：

- 先想单精度的三部分：符号 0 表示正数；阶码 8 位，存的值是真指数加 127；尾数 23 位，前面有个隐含的 1。
- 再想最小规格化正数怎么取：要正数，符号取 0；要最小，阶码取最小的非零值——存 1（真指数是 1-127=-126）；尾数取最小——全 0，也就是 1.0。拼起来就是 1.0×2 的 -126 次方，大约是 1.18×10 的 -38 次方。
- 再想为什么阶码不能取 0：阶码全 0 是非规格化数（尾数前面没有隐含的 1，形式是 0.小数字），表示的数比规格化数还小，但不属于"规格化"这个范围；题目问的是规格化，阶码 0 不算。
- 再想阶码全 1 是什么：阶码全 1 表示无穷大（尾数全 0）或者不是数（尾数不全 0），不是正常的数。
- 拿小数字验算：阶码存 1、尾数全 0 的数是 1.0×2^-126；阶码存 2 的数是 1.0×2^-125，比它大；所以阶码存 1 的就是最小的规格化正数。

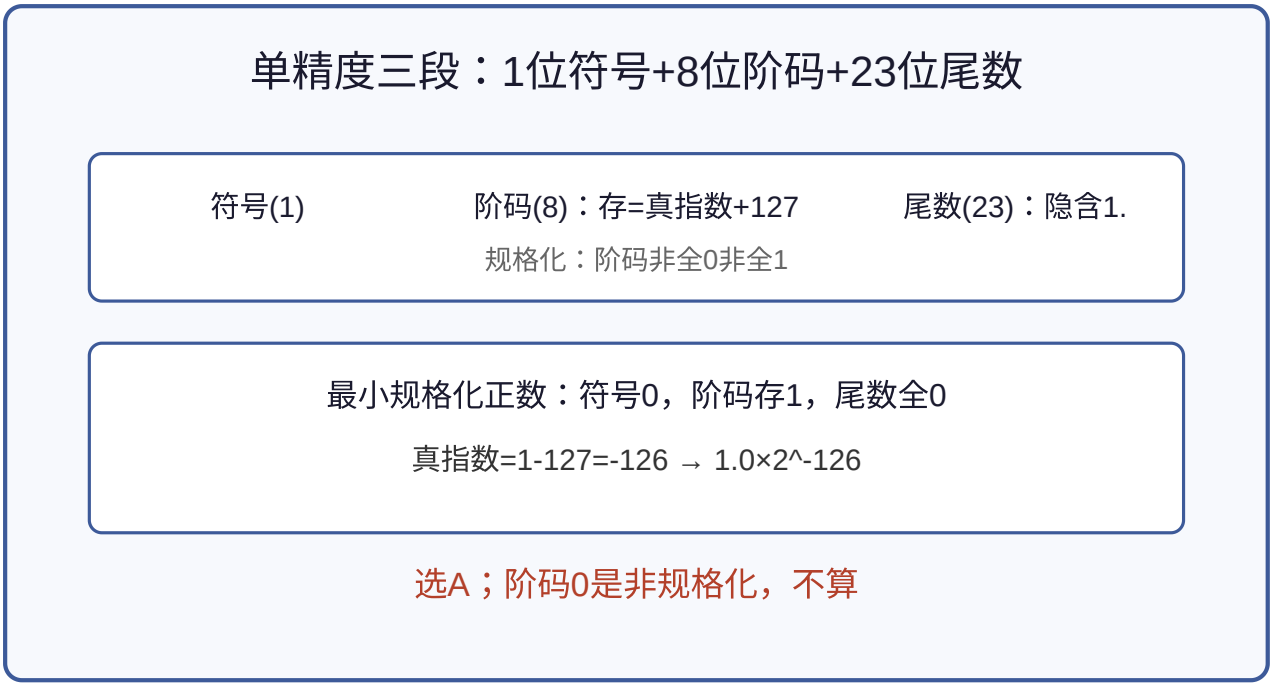
正确选项为什么对： 符号 0、阶码存 1、尾数全 0，值是 1.0×2 的 -126 次方，就是 A，选 A。

错误选项为什么错＋举例：

- B（2 的 -127 次方）：真指数 -127 对应的阶码存 0，那是非规格化数，不是规格化数。教训：规格化的阶码不能是全 0。
- C（2 的 -149 次方）：那是最小非规格化正数（阶码全 0、尾数只有最后一位是 1），比规格化数小得多，但题目问的是规格化。教训：规格化和非规格化分开算。
- D（0）：0 不是正数；而且 0 的阶码和尾数全是 0，不在规格化范围里。教训：最小正数不等于 0。

笔记本一句话： 第 14 题＋单精度最小规格化正数：阶码存 1（真指数 -126）、尾数全 0，值是 2 的 -126 次方。

【图像】



IEEE 754 单精度三段式：1位符号+8位阶码+23位尾数；最小规格化正数=阶码存1、尾数全0

【定位】 组成原理 · 第2章数据的表示 · 浮点数 IEEE 754（规格化）

【易混对比】

- "规格化"和"非规格化"是两回事：规格化要求阶码既不是全 0 也不是全 1（尾数前面有隐含的 1）；阶码全 0 是非规格化数（尾数前面是 0），表示的数更小，但不属于"规格化"范围。
- "阶码存 1"和"真指数 -1"是两回事：阶码存的值 = 真指数 + 127（偏置）；阶码存 1 对应真指数 -126，不是 -1。
- "最小规格化正数"和"最小正数"是两回事：最小规格化正数是 1.0×2^-126；最小正数（非规格化）是 2^-149，比它小得多；0 更不是正数。

最小规格化正数：符号 0、阶码存 1（真指数 -126）、尾数全 0，值是 1.0×2^-126。

【坑位】

阶码全 0 是非规格化数，不算在"规格化"里——问规格化，阶码不能取 0。

最容易踩的坑是：把真指数 -127 当成答案（选 2^-127）。为什么错：真指数 -127 对应的阶码存 0，那是非规格化数；题目问的是"规格化"，阶码最小只能存 1。正确做法：先定"规格化"（阶码非全 0 非全 1），再取最小——阶码存 1、尾数全 0。

【套路】

- 第一步，拆三段：单精度 32 位 = 1 位符号 + 8 位阶码 + 23 位尾数。
- 第二步，定规格化：阶码既不是全 0 也不是全 1；尾数前面有隐含的 1。
- 第三步，取最小正数：符号 0（正数）；阶码取最小非零值——存 1；尾数全 0（即 1.0）。
- 第四步，算值：真指数 = 阶码 - 127 = -126；值 = 1.0×2^-126。

三段 1+8+23；规格化阶码非 0 非全 1；最小正数阶码存 1、尾数全 0。

【类比】

像科学计数法的"标准写法"：1.0×10^-38 是标准写法（规格化），0.1×10^-37 虽然值更小但不是标准写法（非规格化）。题目问"标准写法里最小的正数"，就得按标准写法的规矩来——指数部分不能取那个"非标准"的 0。

【口诀】 单精度三段 1、8、23；最小规格化正数：阶码存 1 尾数 0，1.0 乘 2 的负 126。

【考频】 「IEEE 754」在 2009–2022 年解析中出现 9 次 · 中频

第 15 题 · 小端编址、立即数从 0 改为 -64 后指令机器码变化（正确答案：A）

题干与选项：

某 32 位计算机按字节编址，采用小端(Little Endian)方式。若语句"int i = 0;"对应指令的机器代码为"C7 45 FC 00 00 00 00"，则语句"int i = - 64;"对应指令的机器代码是（ ）。

- A. C7 45 FC C0 FF FF FF
- B. C7 45 FC 0C FF FF FF
- C. C7 45 FC FF FF FF C0
- D. C7 45 FC FF FF FF 0C

入手点： 小端只管字节往哪放，立即数变了只影响立即数那几个字节；先算 -64 的补码，再按小端摆字节，操作码那部分不动。

题干专有名词从零解释：

- 先想"小端编址"：存多字节数据时，低字节放在低地址、高字节放在高地址；口诀是"小端小地址"——小的那一头（低字节）住在小的那一头（低地址）。
- 再想"立即数"：直接写在指令里的常数，执行时不用再去存储器取；比如指令"把寄存器加 5"里的 5 就是立即数。
- 再想"补码"：负数的机器表示：符号位取 1、数值位取反加 1；-64 的 8 位补码是 11000000（C0H），16 位补码是 1111111111000000（FFC0H）。
- 再想"机器码"：指令在机器里存成的二进制串，分成操作码（干什么）、寻址方式、立即数等字段。

考查知识点：

- 先想 -64 的补码：64 是 01000000，取反是 10111111，加 1 是 11000000，也就是 C0H；如果立即数是 16 位，前面补 1 补满，是 FFC0H。
- 再想小端怎么摆：立即数的低字节（C0H）放在低地址，高字节（FFH）放在高地址；从 0 改成 -64 之前，立即数字段全是 00H，改完之后低字节变成 C0H、高字节变成 FFH。
- 再想哪些部分不变：操作码字段（干什么操作）没变，寻址方式字段没变，寄存器编号也没变；变的只有立即数字段那几个字节。
- 拿小例子验算：8 位立即数从 0 改成 -64，机器码里立即数那一字节由 00H 变成 C0H；16 位立即数从 0 改成 -64，低地址字节由 00H 变成 C0H、高地址字节由 00H 变成 FFH。
- 边界情况：立即数如果超出字段能表示的范围（比如 8 位字段想装 -200），就装不下；本题 -64 在 8 位补码范围（-128 到 127）里，装得下。

正确选项为什么对： 只有立即数字段的字节变了（低字节由 00H 变 C0H，高位补 1），操作码等其他字段不动，就是 A，选 A。

错误选项为什么错＋举例：

- B（操作码变了）：操作码表示"干什么"，立即数表示"对谁干"；改立即数不改操作。教训：字段各管各的，改哪个数就动哪个字段。
- C（高字节先变、低字节不动）：那是大端的摆法；小端是低字节在低地址，先变的是低字节。教训：先定大小端，再摆字节。

- D（-64 的补码写成了 01000000）：01000000 是 +64，不是 -64；负数补码要取反加 1，符号位是 1。教训：负数补码先取反、再加 1。

笔记本一句话： 第 15 题＋小端立即数变化：只动立即数字段，-64 的补码低字节是 C0H，高位补 1，低字节在低地址。

【图像】



小端摆字节：低字节在低地址；-64 的补码 FFFFFFC0H 按小端放成 C0 FF FF FF

【定位】 组成原理 · 第2章数据的表示 · 大小端（小端编址、补码）

【易混对比】

- "小端"和"大端"是两回事：小端是低字节在低地址（"小端小地址"），大端是高字节在低地址；摆反了答案就反了。
- "立即数字段"和"操作码字段"是两回事：改立即数只动立即数字段的字节，操作码（干什么操作）、寻址方式、寄存器编号都不动。
- "-64 的补码"和"+64 的原码"是两回事：+64 是 01000000，-64 的补码是取反加 1 得 11000000（C0H）；符号位是 1，不能写成 01000000。

小端：低字节在低地址；改立即数只动立即数字段；-64 的补码低字节是 C0H。

【坑位】

先定大小端，再摆字节——小端是低字节先变（在低地址），别按大端摆。

最容易踩的坑是：-64 的补码写成 01000000（把负号弄丢了）。为什么错：01000000 是 +64；负数补码必须取反加 1，符号位是 1，得 11000000 即 C0H。正确做法：先算补码（64 取反加 1 = C0H），再按小端摆（低字节 C0H 在低地址，高位补 FFH）。

【套路】

- 第一步，算补码：-64 的绝对值 64 是 01000000，取反加 1 得 11000000（C0H）；32 位时高位补 1，得 FFFFFFC0H。
- 第二步，定大小端：小端 = 低字节在低地址（"小端小地址"）。
- 第三步，摆字节：立即数字段的低地址字节放 C0H，高地址字节放 FFH；操作码等其他字段不动。
- 第四步，对选项：找"只有立即数字段变、且低字节是 C0H"的选项。

补码先算对，大小端先定准；小端低字节在低地址，只动立即数字段。

【类比】

像寄一箱书：小端是"最轻的（低字节）放最下面（低地址）"，大端是"最重的放最下面"。改箱子里的一本书（立即数），只换那本书，不动箱子上的标签（操作码）。

【口诀】 小端小地址，低字节在低地址；负数补码取反加 1，只动立即数字段。

【考频】 「小端」在 2009–2022 年解析中出现 20 次 · 高频

第 16 题 · 同一机器数逻辑右移与算术右移结果区别（正确答案：B）

题干与选项：

整数 x 的机器数为 1101 1000，分别对 x 进行逻辑右移 1 位和算术右移 1 位操作，得到的机器数各是（ ）。

- A. 1110 1100、1110 1100
- B. 0110 1100、1110 1100
- C. 1110 1100、0110 1100
- D. 0110 1100、0110 1100

入手点： 两种右移的区别只在高位补什么：逻辑右移不管正负一律补 0，算术右移补符号位；所以对正数两者结果一样，对负数两者结果不一样。

题干专有名词从零解释：

- 先想"右移"：把二进制串往右挪一位，最低位被移出去扔掉，最高位空出来要补东西；右移一位相当于除以 2（对正数和算术右移的负数是向下取整的除法）。
- 再想"逻辑右移"：高位补 0，不管原来的数是正是负；逻辑就是"只管挪位、不管符号"。
- 再想"算术右移"：高位补符号位（原来最高位是 0 就补 0，是 1 就补 1）；算术就是"保持符号不变地除以 2"。
- 再想"机器数"：存在机器里的二进制串，最高位是符号位（0 正 1 负）。

考查知识点：

- 先想正数的情况：符号位是 0，逻辑右移补 0、算术右移补符号位 0，两者补的东西一样，结果也一样。
- 再想负数的情况：符号位是 1，逻辑右移补 0——高位从 1 变成 0，整个数从负数变成了正数，数值大变；算术右移补 1——高位还是 1，数还是负数，数值相当于原来除以 2 再向下取整。
- 拿小例子验算：8 位机器数 10110000（补码真值是 -80）。逻辑右移 1 位：高位补 0，得 01011000，真值是 88。算术右移 1 位：高位补 1，得 11011000，真值是 -40。两个结果一个正、一个负，完全不一样。
- 再想左移的区别：逻辑左移和算术左移都是低位补 0，两者没有区别；只有右移才分逻辑和算术。
- 边界情况：右移多位时，逻辑右移高位一直补 0，负数移够位数会变成 0；算术右移高位一直补 1，负数移够位数会变成全 1（也就是 -1）。

正确选项为什么对： 同一机器数逻辑右移和算术右移的区别就在于高位补 0 还是补符号位——对负数，前者结果变正、后者结果仍为负，就是 B，选 B。

错误选项为什么错＋举例：

- A（两者结果相同）：只对正数成立；题目给的机器数是负数（符号位是 1），两者结果不一样。教训：先看符号位，再说结果。
- C（逻辑右移保持符号）：保持符号的是算术右移，不是逻辑右移；逻辑右移高位补 0，负数会变正。教训：逻辑不管符号、算术保持符号。
- D（算术右移高位补 0）：算术右移高位补的是符号位，不是 0；补 0 的是逻辑右移。教训：两种右移的补位规则不要记反。

笔记本一句话： 第 16 题＋右移区别：逻辑补 0，算术补符号位；正数两者一样，负数一个变正、一个仍负。

【图像】



逻辑右移 vs 算术右移：区别只在高位补0还是补符号位；负数结果一正一负

【定位】 组成原理 · 第2章数据的表示 · 移位运算（逻辑移位、算术移位）

【易混对比】

- "逻辑右移"和"算术右移"是两回事：逻辑右移高位一律补 0（不管正负），算术右移高位补符号位（保持符号不变）。
- "正数"和"负数"是两回事：正数符号位是 0，两种右移补的东西一样，结果也一样；负数符号位是 1，逻辑右移补 0 会把负数变成正数，算术右移补 1 还是负数。
- "右移"和"左移"是两回事：左移都是低位补 0，逻辑和算术没有区别；只有右移才分逻辑和算术。

两种右移的区别只在高位补什么：逻辑补 0，算术补符号位；正数结果一样，负数不一样。

【坑位】

负数逻辑右移，高位补 0 会把负数变成正数——数值大变，别当成"除以 2"。

最容易踩的坑是：觉得"右移就是除以 2，两种移位结果一样"。为什么错：逻辑右移不管符号，高位补 0；负数（符号位 1）逻辑右移后高位变 0，整个数从负变正，根本不是除以 2。正确做法：先看符号位——正数两者一样，负数逻辑变正、算术仍负。

【套路】

- 第一步，看符号位：最高位 0 是正数，1 是负数。
- 第二步，正数直接写：正数两种右移结果一样，高位补 0。
- 第三步，负数分开算：逻辑右移高位补 0（结果变正），算术右移高位补 1（结果仍负）。
- 第四步，左移不区分：左移都是低位补 0，逻辑算术结果一样。

右移看高位补什么：逻辑补 0，算术补符号；正数一样，负数一正一负。

【类比】

像排队往右挪：逻辑右移是"空出来的位置一律站新人（0）"，不管原来排头是谁；算术右移是"空出来的位置站和排头一样的人（符号位）"，保持队伍的"正负"性质不变。排头是负数时，前者队伍变性质，后者不变。

【口诀】 右移区别在高位：逻辑补 0 算术补符号；正数一样负数变。

【考频】 「算术右移」在 2009–2022 年解析中出现 2 次 · 低频

第 17 题 · 2K×1 位 DRAM 芯片、地址引脚最少且刷新开销尽量小、行列数取舍（正确答案：C）

题干与选项：

假定 DRAM 芯片中存储阵列的行数为 r、列数为 c，对于一个 2K×1 位的 DRAM 芯片，为保证其地址引脚数最少，并尽量减少刷新开销，则 r、c 的取值分别是（）。

- A. 2048、1
- B. 64、32
- C. 32、64
- D. 1、2048

入手点： DRAM 的地址是分两次送的，引脚数由行列地址位数中较大的那个决定；引脚最少就要求行列位数尽量均分，刷新开销小就要求行数尽量少——两个条件一拼，答案就出来了。

题干专有名词从零解释：

- 先想"DRAM"：动态随机存储器，靠电容存电荷表示 0 和 1；电容会漏电，所以要定期刷新（把数据重新写一遍）。
- 再想"2K×1 位"：芯片里有 2K 个存储单元（2K 是 2048），每个单元存 1 位；要选中其中一个单元，需要 11 位地址（因为 2 的 11 次方是 2048）。
- 再想"地址复用"：行地址和列地址分两次送，用同一组地址引脚；先送行地址选中一行，再送列地址选中那一行里的一列。
- 再想"地址引脚"：芯片上用来接收地址的针脚；引脚数等于行地址位数和列地址位数中较大的那个。
- 再想"刷新开销"：刷新是按行进行的，一次刷一行；行数越多，刷完所有行用的时间越多，开销越大。

考查知识点：

- 先想地址一共几位：2048 个单元需要 11 位地址，这 11 位要拆成行地址和列地址两部分。
- 再想引脚最少怎么拆：引脚数是行列位数中较大的那个；要让较大的那个尽量小，就要把 11 位尽量均分——5 位和 6 位，较大的那个是 6，引脚就是 6 根；如果拆成 4 位和 7 位，较大的那个是 7，引脚就是 7 根，没做到最少。
- 再想刷新开销怎么小：行数越少，刷完所有行越快；同样是 6 根引脚，32 行×64 列（行地址 5 位、列地址 6 位）比 64 行×32 列（行地址 6 位、列地址 5 位）行数少一半，刷新开销更小。
- 再想结论：行 32 列 64，既做到引脚最少（6 根），又做到行数最少（刷新开销最小）。
- 边界情况：行列不能拆成非 2 的幂次（比如 3 位和 8 位凑 11 位）——行数和列数必须是 2 的幂次，不然地址位数对不上；行列拆成 1 位和 10 位虽然行数最少，但引脚是 10 根，违背了引脚最少的前提。

正确选项为什么对： 行 32 列 64 时地址引脚是 6 根（最少），行数 32 又是同等引脚下的最少行数（刷新开销最小），就是 C，选 C。

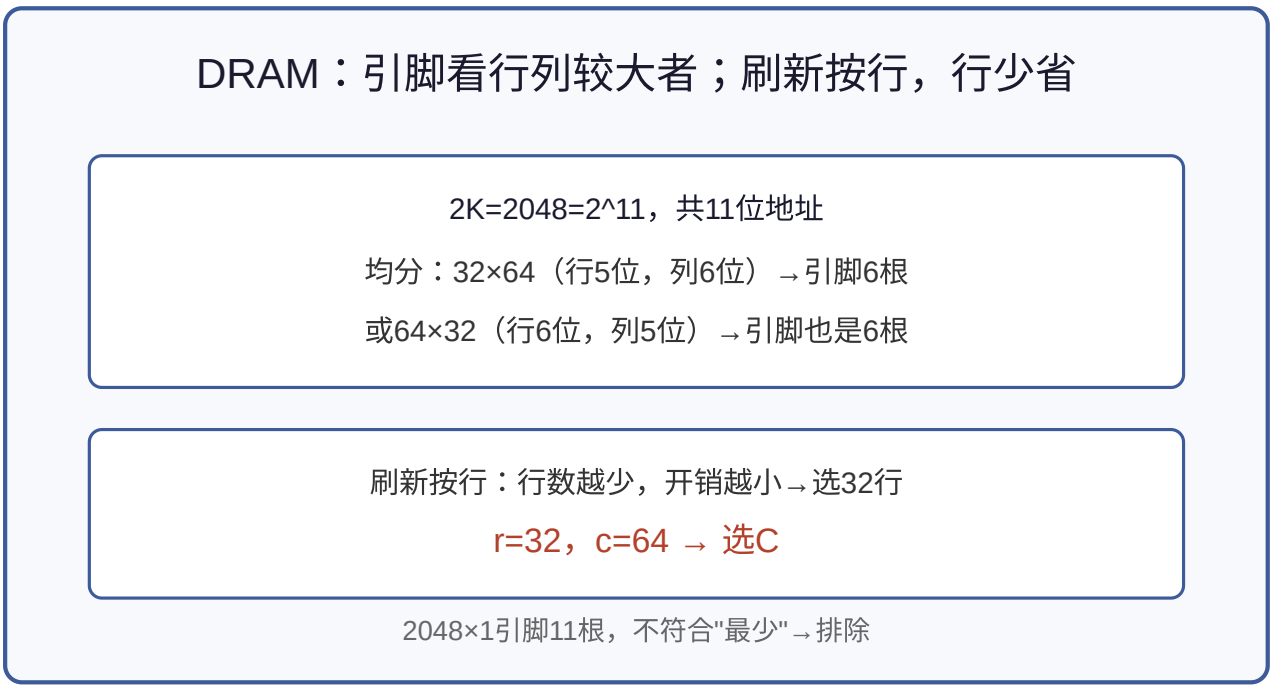
错误选项为什么错＋举例：

- A（64 行×32 列）：引脚同样是 6 根，但行数 64 比 32 多一倍，刷新开销大一倍。教训：引脚相同比行数，行数越少刷新越省。
- B（16 行×128 列或 128 行×16 列）：引脚是 7 根（4 位和 7 位中较大的那个），没做到最少。教训：引脚数看行列位数中较大的那个。

- D（行列拆成非 2 的幂次）：行数和列数必须是 2 的幂次，不然地址线位数凑不齐 11 位。教训：行列数先写成 2 的幂次再比。

笔记本一句话： 第 17 题＋DRAM 行列取舍：引脚看行列位数中较大的那个（均分最少），刷新按行、行数越少越省。

【图像】



DRAM行列取舍：地址分两次送，引脚数看行列位数较大者；刷新按行，行数越少越省

【定位】 组成原理 · 第3章存储器 · DRAM（地址复用、刷新）

【易混对比】

- "地址引脚数"和"地址位数"是两回事：DRAM 地址分行、列两次送，引脚数由行列地址位数中较大的那个决定（复用），不是两者相加。
- "引脚最少"和"刷新开销小"是两回事：引脚最少要求行列位数尽量均分（比如 2K 均分成 32×64）；刷新开销小要求行数尽量少（DRAM 按行刷新，行越少刷得越快）。
- "行数少"和"列数少"是两回事：刷新按行进行，所以行数少省刷新；但行列位数要均分才能让引脚最少——两个条件一拼，行取 32、列取 64（行 5 位、列 6 位，引脚 6 根，行数最少）。

引脚看行列位数中较大的那个（均分最少），刷新按行、行数越少越省。

【坑位】

别把行列地址位数相加当引脚数——DRAM 地址分两次送，引脚是复用的，看较大者。

最容易踩的坑是：2K 需要 11 位地址，就认为引脚要 11 根。为什么错：DRAM 的行地址和列地址分两次送，用同一组引脚（复用）；引脚数 = max(行位数, 列位数)。正确做法：先把 2K 拆成行×列，让行列位数尽量接近（均分），引脚数就是其中较大的那个；再在均分的基础上选行数少的（省刷新）。

【套路】

- 第一步，算总位数：2K = 2048 = 2^11，共 11 位地址。

- 第二步，均分行列：11 位拆成行 5 位、列 6 位（32×64）或行 6 位、列 5 位（64×32）；引脚数 = 6 根（较大者）。
- 第三步，选行数少：刷新按行进行，行数越少开销越小——选 32 行×64 列。
- 第四步，排除极端：2048×1 或 1×2048 的引脚要 11 根，不符合"最少"，直接排除。

总位数先算出，均分行列看较大；刷新按行，行数越少越省。

【类比】

像一个长方形停车场：入口的收费口（引脚）要能报出行号和列号，但行号列号分两次报，所以收费口只需要能报出较大的那个数。刷新像保安巡逻——按行巡，行数越少巡得越快。所以停车场要尽量方（行列位数均分），且行数少的那边朝巡逻方向。

【口诀】 DRAM 地址分两次送，引脚看行列较大；均分最少引脚，行少省刷新。

【考频】 「DRAM」在 2009–2022 年解析中出现 9 次 · 中频

第 18 题 · 变址寻址循环访问 double 数组、由当前元素地址反推变址寄存器值（正确答案：B）

题干与选项：

按字节编址的计算机中，某 double 型数组 A 的首地址为 2000H，使用变址寻址和循环结构访问数组 A，保存数组下标的变址寄存器初值为 0，每次循环取一个数组元素，其偏移地址为变址值乘以sizeof(double)，取完后变址寄存器内容自动加1。若某次循环所取元素的地址为2100H，则进入该次循环时变址寄存器的内容是（ ）。

- A. 25
- B. 32
- C. 64
- D. 100

入手点： 变址寻址的有效地址等于基址加上变址值乘比例；double 一个元素占 8 个字节，比例就是 8；反推就是地址差除以 8。

题干专有名词从零解释：

- 先想"变址寻址"：一种取数方式——有效地址等于基址寄存器的值加上变址寄存器的值（有的机器还要乘一个比例因子）；循环访问数组时，基址不动，变址寄存器每次变，地址就一个一个往后走。
- 再想"基址"：数组第一个元素的地址（数组首地址），循环开始时就定好了，不动。
- 再想"变址寄存器"：专门用来存下标或者偏移量的寄存器；循环每走一步，它的值就变一次。
- 再想"double 数组"：每个元素是双精度浮点数，占 8 个字节；数组里相邻两个元素的地址差 8，不是差 1。

考查知识点：

- 先想有效地址怎么算：当前元素的地址等于数组首地址加上变址值乘以元素大小；double 元素大小是 8，所以地址等于首地址加上变址值乘 8。
- 再想反推怎么算：变址值等于（当前元素地址减去数组首地址）再除以 8；减首地址是去掉基址，除以 8 是把字节数换算成下标。
- 拿小例子验算：数组首地址是 1000，当前元素地址是 1024，地址差是 24，除以 8 得 3，说明当前是下标为 3 的元素，变址寄存器里的值就是 3。
- 再想两种变址寄存器存法：有的机器变址寄存器存下标（反推要除以元素大小），有的存字节偏移（反推不用除）；题目给定的寻址方式下，变址寄存器存的是下标，所以要除以 8。
- 边界情况：地址差不能被 8 整除时，说明地址没对齐到 double 边界，或者首地址、当前地址有一个读错了；下标从 0 开始数，第一个元素的变址值是 0，不是 1。

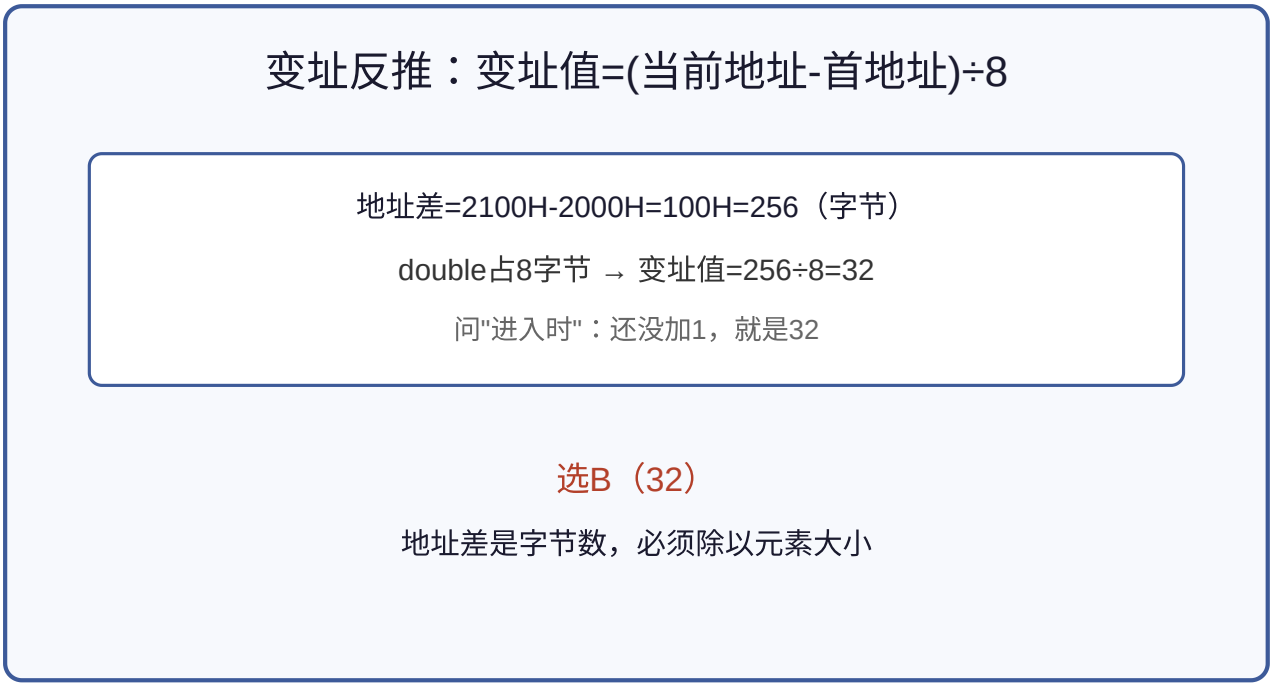
正确选项为什么对： 按"地址差除以元素大小"的办法算，变址值就是 B，选 B。

错误选项为什么错＋举例：

- A（地址差没除以 8，直接当变址值）：那是把字节偏移当成了下标；double 一个元素 8 个字节，地址差是下标的 8 倍，不除就大了 8 倍。教训：字节数和下标之间差一个元素大小。
- C（除以 4）：4 是一个 int 元素的大小，不是 double 的；double 是 8 个字节。教训：先看清数组元素的类型和大小。
- D（拿当前地址直接当变址值）：忘了减首地址；变址值是相对首地址的偏移，不是绝对地址。教训：反推先减基址，再除比例。

笔记本一句话： 第 18 题＋变址寻址反推：变址值等于（当前地址减首地址）除以元素大小，double 除以 8。

【图像】



变址寻址反推：变址值=(当前地址-首地址)÷元素大小；double占8字节

【定位】 组成原理 · 第4章指令系统 · 寻址方式（变址寻址）

【易混对比】

- "变址值"和"地址差"是两回事：地址差是字节数（当前地址减首地址），变址值是元素下标；地址差要除以元素大小才是变址值。
- "double"和"int"是两回事：double 一个元素占 8 个字节，int 占 4 个字节；除数用错，结果就错一倍。
- "进入该次循环时"和"取完之后"是两回事：题目问的是进入循环时的变址值（还没加 1）；取完之后会自动加 1，别多加。

变址值 =（当前地址 - 首地址）÷ 元素大小；double 除以 8。

【坑位】

地址差是字节数，不是下标——必须除以元素大小（double 是 8），直接当下标就错了。

最容易踩的坑是：算出地址差 100H = 256，就直接选 256（或 100）。为什么错：256 是字节数，不是元素下标；每个 double 占 8 个字节，256 个字节是 32 个元素。正确做法：地址差先换成十进制（100H = 256），再除以 sizeof(double) = 8，得 32。

【套路】

- 第一步，算地址差：当前地址 - 首地址（注意十六进制转十进制）。
- 第二步，定元素大小：double 占 8 字节，int 占 4 字节，char 占 1 字节。
- 第三步，除：变址值 = 地址差 ÷ 元素大小。
- 第四步，看时点：问"进入循环时"就是除完的结果，别再加 1。

地址差除以元素大小；double 除 8；看清问的是进入时还是取完后。

【类比】

像数书架上的书：书架从第 2000 厘米开始，每本书厚 8 厘米，现在看到第 2100 厘米处的书，问它是第几本。(2100-2000)÷8 = 12.5？不对——2100H 是十六进制，先换成十进制再算。原理一样：总长度除以每本厚度，就是本数。

【口诀】 变址寻址反推：地址差除以元素大小，double 除以 8。

【考频】 「变址寻址」在 2009–2022 年解析中出现 17 次 · 高频

第 19 题 · 减法指令执行后进位／借位标志与溢出标志取值（正确答案：A）

题干与选项：

减法指令"sub R1, R2, R3"的功能为"（R1）-（R2）→ R3"，该指令执行后将生成进位/借位标志 CF 和溢出标志 OF。若（R1）= FFFF FFFFH，（R2）= FFFF FFF0H，则该减法指令执行后，CF 与 OF 分别为（ ）。

- A. CF=0, OF=0
- B. CF=1, OF=0
- C. CF=0, OF=1
- D. CF=1, OF=1

入手点： 两个标志各管各的：进位标志看无符号够不够减，溢出标志看有符号超没超范围；减法在机器里是按加法算的，先把减数变补再加。

题干专有名词从零解释：

- 先想"减法指令"：算被减数减减数的指令；机器内部不直接做减法，而是把减数变成相反数，再按加法算。
- 再想"进位标志（CF）"：加法时最高位有没有进位、减法时有没有借位；减法里，被减数比减数小（不够减）时借位，标志是 1；够减时标志是 0。
- 再想"溢出标志（OF）"：有符号数运算结果超出了表示范围；正数减负数得到负数、或者负数减正数得到正数，就是溢出了，标志是 1；否则是 0。
- 再想"变补"：把一个数变成它的相反数的补码：正数变负数、负数变正数，办法是全部位取反再加 1（符号位也取反）。

考查知识点：

- 先想减法怎么算：A 减 B 等于 A 加上 B 的变补；比如 5 减 3，等于 5 加上 -3 的补码。
- 再想进位标志怎么看：从无符号的角度看，被减数够不够减——够减（被减数不比减数小），没有借位，标志是 0；不够减，借位，标志是 1。
- 再想溢出标志怎么看：从有符号的角度看，结果的符号合不合理——正数减负数本该得正数，得了负数就是溢出；负数减正数本该得负数，得了正数就是溢出。另一个办法是看最高位和次高位的进位：两个进位不一样，就是溢出。
- 拿小例子验算：8 位补码，5 减 3：够减，借位标志是 0；结果 2 在范围内，溢出标志是 0。再算 -100 减 100：无符号看，156 减 100 够减，借位标志是 0；有符号看，-100 减 100 本该是 -200，8 位补码只能表示到 -128，超了，溢出标志是 1。两个标志一个是 0、一个是 1，互不影响。
- 边界情况：加法和减法的进位标志含义不一样——加法看最高位进位，减法看够不够减；但溢出标志的道理一样，都是看有符号结果超没超范围。

正确选项为什么对： 按"借位看无符号够不够减、溢出看有符号超没超范围"的办法判断，两个标志的取值就是 A，选 A。

错误选项为什么错＋举例：

- B（把进位标志和溢出标志混为一谈）：两个标志管的是两回事——一个管无符号、一个管有符号；上例里一个是 0、一个是 1，混为一谈就判错了。教训：CF 看借位，OF 看溢出，各算各的。
- C（减法借位含义和加法进位含义弄反）：减法里不够减时借位标志是 1，不是 0；按加法的"有进位是 1"去套减法，方向就反了。教训：减法先问够不够减。

- D（只看最高位进位判断溢出）：有符号溢出要看符号变化（或者最高位和次高位两个进位是否相同），只看最高位一个进位，判不准。教训：溢出看符号，不看单个进位。

笔记本一句话： 第 19 题＋减法两个标志：借位看无符号够不够减，溢出看有符号超没超范围，各算各的。

【图像】



减法两个标志各管各：CF看无符号够不够减（借位），OF看有符号超没超范围

【定位】 组成原理 · 第2章运算器 · 标志位（CF、OF，减法）

【易混对比】

- "CF（借位）"和"OF（溢出）"是两回事：CF 管无符号数——不够减就借位（CF=1）；OF 管有符号数——结果超出有符号范围就溢出（OF=1）。两个各算各的。
- "减法"和"加法"是两回事：减法在机器里按加法算（减数变补再加）；CF 的判断要按"够不够减"来（被减数 < 减数则借位），不能直接套加法的进位。
- "够减"和"没溢出"是两回事：无符号够减（CF=0）和有符号没溢出（OF=0）可以同时成立，也可以一个成立一个不成立；本题就是 CF=0、OF=0，但别混成一个。

减法两个标志各管各：借位看无符号够不够减，溢出看有符号超没超范围。

【坑位】

CF 和 OF 别混成一个——无符号看 CF，有符号看 OF，各判各的。

最容易踩的坑是：算出没借位，就以为也没溢出（或反过来）。为什么错：CF 和 OF 是两套独立的判断——CF 看无符号的借位，OF 看有符号的符号变化；一个的结论推不出另一个。正确做法：CF 按"被减数和减数比大小"判（无符号），OF 按"结果符号该不该变"判（有符号），分开算。

【套路】

- 第一步，减法变加法：A - B = A + (-B 的补码)，先求减数的相反数补码。
- 第二步，判 CF（借位）：无符号比较——被减数 ⩾ 减数，不借位（CF=0）；被减数 < 减数，借位（CF=1）。

- 第三步，判 OF（溢出）：有符号看符号——正减负得负、或负减正得正，就是溢出；或看最高位与次高位进位是否相同。
- 第四步，分开写答案：CF 和 OF 各写各的，别互相推。

减法变加法；CF 看无符号够不够减，OF 看有符号符号变没变。

【类比】

像两个会计查账：一个只管"现金够不够付"（无符号，CF），一个只管"账面盈亏超没超预算"（有符号，OF）。付一笔钱，现金够（CF=0）不代表没超预算（OF 可能=1），两个会计各查各的账。

【口诀】 减法变加法算；借位看无符号，溢出看有符号，各管各。

【考频】 「溢出标志」在 2009–2022 年解析中出现 5 次 · 中频

第 20 题 · 指令流水线各功能段耗时与锁存器延迟、CPU 时钟周期下限（正确答案：D）

题干与选项：

若某计算机最复杂指令的执行需要完成 5 个子功能，分别由功能部件 A~E 实现，各功能部件所需时间分别为 80ps、50ps、50ps、70ps 和 50ps，采用流水线方式执行指令，流水段寄存器延时为 20ps，则 CPU 时钟周期至少为（ ）。

- A. 60 ps
- B. 70 ps
- C. 80 ps
- D. 100 ps

入手点： 时钟周期不能比最慢的那一段加锁存器延迟还短——一个周期里，数据要流过最慢的一段、再被锁存器锁存住，短了数据就赶不上。

题干专有名词从零解释：

- 先想"指令流水线"：把一条指令的执行拆成几段（比如取指、译码、执行、访存、写回），像工厂流水线一样，每段同时干不同指令的不同部分。
- 再想"功能段耗时"：某一段干完自己的活要花的时间；各段耗时不一样，有的快、有的慢。
- 再想"锁存器"：段与段之间的寄存小格子，每个时钟周期结束时把上一段的结果锁存住、交给下一段；锁存本身也要花一点时间，叫锁存器延迟。
- 再想"时钟周期"：CPU 的节拍长度；每个节拍，各段往前推一步。

考查知识点：

- 先想时钟周期由谁决定：由最慢的那一段决定——因为一个节拍里，最慢的那一段必须干完；其他段干得快，也得等着节拍结束才能往前推。
- 再想锁存器延迟为什么要加：数据流过功能段之后，还要经过锁存器才能交给下一段；锁存器延迟是每个周期都要花的，不能省。
- 再想下限公式：时钟周期最少等于最慢功能段的耗时加上锁存器延迟。
- 拿小例子验算：各段耗时是 10ns、15ns、20ns、12ns，锁存器延迟 2ns；最慢段是 20ns，时钟周期最少是 20+2=22ns。如果周期定成 20ns，数据在最慢段还没算完就被锁存，结果就错了。
- 边界情况：各段耗时都一样时，时钟周期就是单段耗时加锁存器延迟；锁存器延迟为 0 的理想情况不考虑，题目给了锁存器延迟就要加上。
- 易混对子：时钟周期和单条指令总延迟不是一回事——单条指令从头到尾要花各段耗时之和（再加锁存器延迟乘段数），那是延迟；时钟周期是节拍长度，只看最慢段加锁存器延迟。

正确选项为什么对： 最慢功能段耗时加上锁存器延迟，就是时钟周期的下限，选 D。

错误选项为什么错＋举例：

- A（各段耗时之和）：那是单条指令从头到尾的总延迟，不是时钟周期；时钟周期只看最慢段。教训：周期看最慢段，延迟才求和。
- B（各段耗时的平均值）：平均值比最慢段小，按平均值定周期，最慢段的数据赶不上锁存。教训：周期不能比最慢段还短。

- C（最慢段耗时、没加锁存器延迟）：忘了锁存器也要花时间；少算了锁存器延迟，周期就偏小了。教训：最慢段加锁存器延迟，两项都不能少。

笔记本一句话： 第 20 题＋流水线时钟周期下限：最慢功能段耗时加锁存器延迟。

【图像】



流水线时钟周期下限=最慢功能段耗时+锁存器延迟；一个周期要流过最慢段并被锁存

【定位】 组成原理 · 第5章 CPU · 指令流水线（时钟周期）

【易混对比】

- "最慢功能段"和"平均耗时"是两回事：时钟周期由最慢的那一段决定（木桶效应），不是各段的平均值。
- "加锁存器延迟"和"不加"是两回事：一个时钟周期里，数据要流过功能段、再被段间锁存器锁存住；锁存器本身也要时间，必须加上。
- "至少"和"恰好"是两回事：题目问"至少"，是下限——周期可以比下限长（留余量），但不能比下限短（短了数据赶不上）。

时钟周期下限 = 最慢功能段耗时 + 锁存器延迟。

【坑位】

别忘了加锁存器延迟——最慢段 80ps，答案不是 80ps，是 80+20=100ps。

最容易踩的坑是：看到最慢段 80ps，直接选 80ps。为什么错：一个时钟周期要完成"流过最慢段 + 被锁存器锁存"两件事；锁存器延迟 20ps 也要算进去。正确做法：先找最慢段（80ps），再加锁存器延迟（20ps），得 100ps。

【套路】

- 第一步，找最慢段：各功能段耗时里取最大的（80ps）。
- 第二步，加锁存延迟：最慢段耗时 + 流水段寄存器延时（80+20=100ps）。

- 第三步，对"至少"：题目问"至少"，算出来的是下限，选等于下限的选项。

找最慢段，加锁存延迟，就是时钟周期下限。

【类比】

像流水线上的工人：最慢的工人 80 秒做完一道工序，旁边的传送带（锁存器）把半成品送到下一站要 20 秒。整个流水线的节拍（时钟周期）不能比 100 秒短——短了，最慢的工人还没做完，传送带就把半成品拖走了。

【口诀】 流水线时钟周期：最慢段加锁存延迟。

【考频】 「流水线」在 2009–2022 年解析中出现 33 次 · 高频

第 21 题 · 提高同步总线数据传输率的措施（正确答案：B）

题干与选项：

下列选项中，可提高同步总线数据传输率的是（）。 Ⅰ.增加总线宽度 Ⅱ.提高总线工作频率 Ⅲ.支持突发传输 Ⅳ.采用地址/数据线复用

- A. 仅Ⅰ、Ⅱ
- B. 仅Ⅰ、Ⅱ、Ⅲ
- C. 仅Ⅲ、Ⅳ
- D. Ⅰ、Ⅱ、Ⅲ和Ⅳ

入手点： 同步总线的数据传输率就看两个数相乘：时钟频率乘每次传的数据量；提高传输率就从这两个数下手——频率往上提，或者一次多传点。

题干专有名词从零解释：

- 先想"总线"：连接 CPU、存储器、输入输出设备的公共数据通道；大家轮流用它传数据。
- 再想"同步总线"：传数据靠时钟节拍统一指挥；每个节拍传一批数据，节拍越快、每拍传得越多，传输率越高。
- 再想"数据传输率"：单位时间里传过去的数据量（比如每秒多少字节）；公式是时钟频率乘每次传的数据量。
- 再想"数据线宽度"：一次能并行传多少位（比如 64 位宽的数据线，一个节拍能传 8 个字节）。

考查知识点：

- 先想传输率公式：数据传输率等于时钟频率乘数据线宽度（再乘每个时钟传几次）；三个数里任何一个变大，传输率都变大。
- 再想提高频率：时钟节拍变快，单位时间里的节拍数变多，传的数据自然变多。
- 再想加宽数据线：数据线从 32 位加宽到 64 位，一个节拍传的数据翻倍，传输率也翻倍。
- 再想每个时钟传几次：时钟上下沿都传数据（双倍速率），或者一次地址传一批数据（突发传输），每次传的数据量也变大了。
- 拿小例子验算：64 位宽、时钟 100MHz 的总线，传输率是 100M×8 字节=800MB/s；时钟提到 200MHz，传输率变成 1600MB/s；数据线加宽到 128 位，传输率也变成 1600MB/s。
- 边界情况：提高频率要受器件速度限制，不是无限往上提；加宽数据线要增加针脚和成本；突发传输只对连续地址的数据有效。

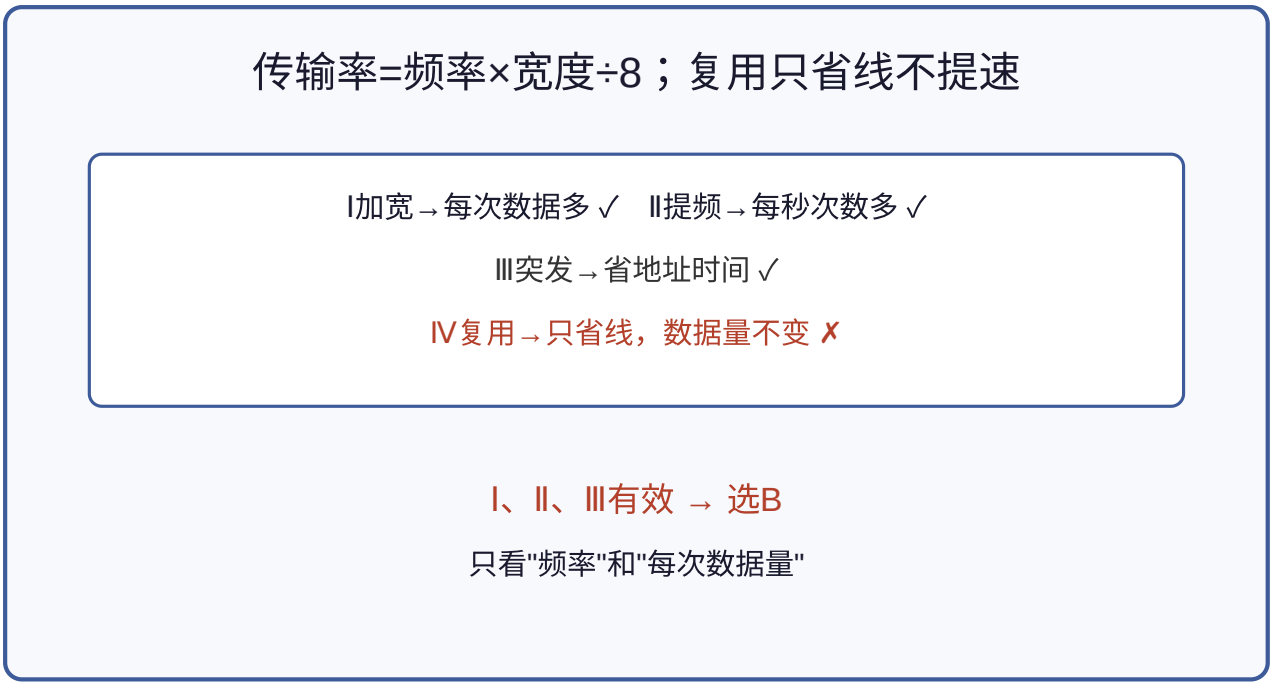
正确选项为什么对： 提高时钟频率、加宽数据线、双倍速率、突发传输，都是让"频率乘每次数据量"变大的办法，就是 B，选 B。

错误选项为什么错＋举例：

- A（增加地址线宽度）：地址线管的是"找谁"（寻址范围），不是"一次传多少数据"；地址线加宽，数据量不变，传输率不变。教训：地址线和数据线分开算。
- C（增加控制线）：控制线管的是"怎么传"（读写信号、仲裁信号），不直接增加数据量。教训：控制线不管数据量。
- D（降低时钟频率）：频率降低，单位时间里的节拍数变少，传输率反而下降，方向反了。教训：传输率和频率同方向变。

笔记本一句话： 第 21 题＋同步总线提速：频率乘每次数据量——提频率、加宽数据线、双倍速率、突发传输都有效。

【图像】



同步总线传输率=频率×宽度/8；提频率、加宽、突发传输有效，地址数据复用无效

【定位】 组成原理 · 第6章总线 · 总线性能（数据传输率）

【易混对比】

- "提高传输率"和"减少总线数量"是两回事：地址/数据线复用是少用了几根线（省成本），但一次传的数据量没变，传输率不提高。
- "突发传输"和"单次传输"是两回事：突发传输一次给出地址、连续传多个数据，省了反复给地址的时间，传输率提高。
- "总线宽度"和"总线频率"是两回事：宽度是每次传多少位，频率是每秒传多少次；传输率 = 频率 × 宽度 ÷ 8，两者都能提速。

传输率 = 频率 × 宽度 ÷ 8；提频率、加宽、突发传输都有效，地址数据复用只省线不提速。

【坑位】

"采用地址/数据线复用"不能提高传输率——它只是省了几根线，一次传的数据量没变。

最容易踩的坑是：觉得"复用"听起来很高级，就选了包含Ⅳ的选项。为什么错：地址/数据线复用是让地址和数据分时用同一组线，省的是线的数量；每次传输的数据量、传输的次数都没变，传输率不变。正确做法：只看"频率"和"每次数据量"——能让其中之一变大的才有效。

【套路】

- 第一步，写公式：传输率 = 总线频率 × 总线宽度 ÷ 8。
- 第二步，逐个看：增加宽度→每次数据量变大，有效；提高频率→每秒次数变多，有效；突发传输→省了地址时间，有效；地址数据复用→只省线，有几根线和传多快没关系，无效。

- 第三步，组合：选"Ⅰ、Ⅱ、Ⅲ都有效、Ⅳ无效"的选项。

公式频率乘宽度除以 8；看频率和数据量，复用只省线。

【类比】

像高速公路运货：传输率就是每小时运多少吨 = 车速（频率）× 每车装多少（宽度）。加宽车道（宽度）、提速（频率）、让货车连发不停车（突发），都能多运货；但把客车道和货车道合并成一条分时用（复用），省了条路，运货速度可没变。

【口诀】 传输率等于频率乘宽度除以 8；提频加宽加突发，复用省线不提速。

【考频】 「总线」在 2009–2022 年解析中出现 26 次 · 高频

第 22 题 · 外部 I／O 中断相关说法、正确项（正确答案：C）

题干与选项：

下列关于外部 I/O 中断的叙述中，正确的是（ ）。

- A. 中断控制器按所接收中断请求的先后次序进行中断优先级排队
- B. CPU 响应中断时，通过执行中断隐指令完成通用寄存器的保护
- C. CPU 只有在处于中断允许状态时，才能响应外部设备的中断请求
- D. 有中断请求时，CPU 立即暂停当前指令执行，转去执行中断服务程序

入手点： 先分清外中断和内中断：外部 I／O 中断是外面设备引起的中断，可屏蔽、有优先级、响应时要保存断点现场；拿这个标准去套每句话。

题干专有名词从零解释：

- 先想"中断"：CPU 正在干活时，外面或者里面突然有个事件要处理，CPU 先停下手里的活、去处理这个事件，处理完再回来继续；这个"停下—处理—回来"的过程就叫中断。
- 再想"外部中断"：由 CPU 外面的事件引起的中断，比如 I／O 设备传完了数据、定时器到时间了、外部信号来了；和程序本身执行出错引起的内中断（异常）不一样。
- 再想"可屏蔽"：CPU 可以通过中断允许标志选择"理它"还是"暂时不理它"；可屏蔽的中断能被暂时关掉，不可屏蔽的中断关不掉。
- 再想"断点和现场"：断点是被打断的那条指令的地址（回来要从这里继续），现场是 CPU 里寄存器的值（回来要恢复成原来的样子）。

考查知识点：

- 先想外部 I／O 中断是谁引起的：I／O 设备（比如键盘、磁盘、网卡）完成了数据传送，或者需要 CPU 处理，就向 CPU 发中断请求；不是程序执行出错引起的（那是内中断，比如除零、缺页）。
- 再想响应过程：CPU 收到请求后，先关中断（防止处理过程中又被打断），保存断点和现场，查中断向量表找到对应的服务程序，处理完恢复现场、开中断，从断点继续执行。
- 再想可屏蔽和优先级：外部 I／O 中断一般是可屏蔽的；多个设备同时请求时，按优先级判优（优先级高的先处理）。
- 再想和 DMA 的区别：DMA 是设备直接和存储器传数据、不用 CPU 一条一条搬，但 DMA 传完了也要发中断告诉 CPU；中断和 DMA 是配合关系，不是互相取代。
- 边界情况：不可屏蔽中断（比如电源故障）关不掉，优先级最高；外部 I／O 中断通常是可屏蔽的，不要把两者混淆。

正确选项为什么对： C 说的是外部 I／O 中断的标准特点——外面设备引起、可屏蔽、响应时保存断点现场，符合中断的定义和过程，选 C。

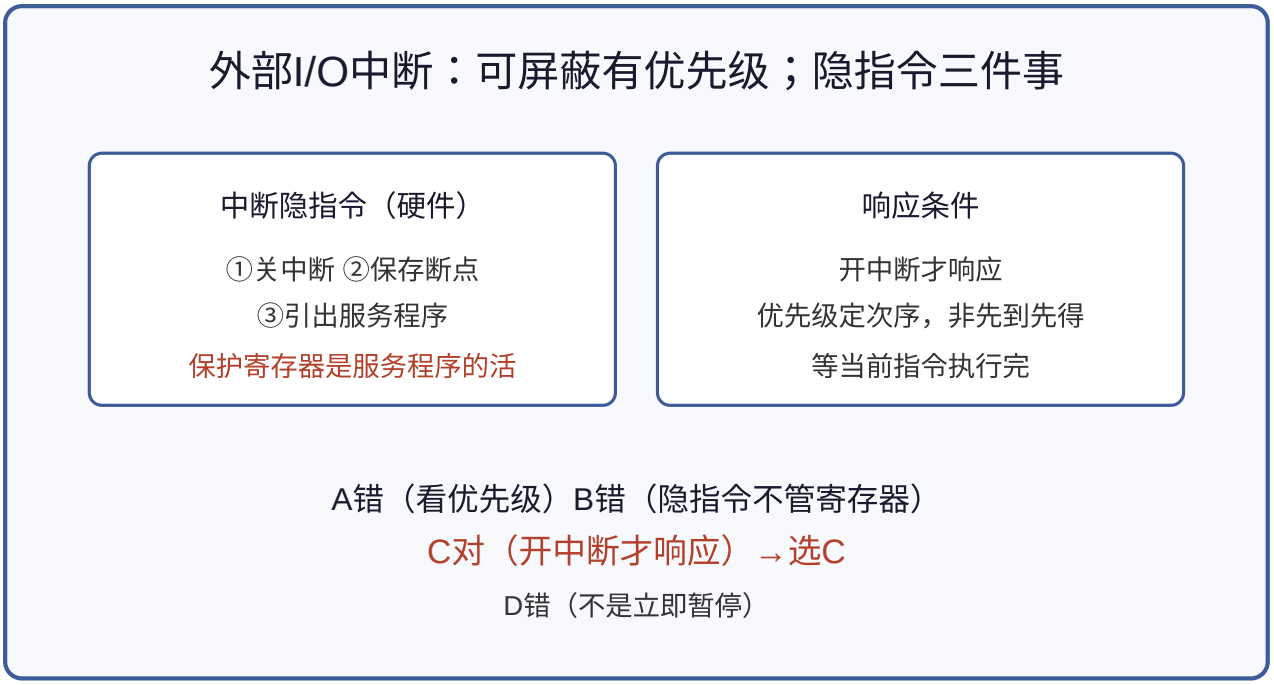
错误选项为什么错＋举例：

- A（把外部 I／O 中断说成不可屏蔽）：不可屏蔽的是电源故障这类紧急信号，不是普通的 I／O 设备请求；I／O 中断一般是可屏蔽的。教训：外中断可屏蔽，紧急故障才不可屏蔽。
- B（中断处理不需要保存现场）：不保存现场，处理完就回不到原来的程序了；断点和现场必须保存、必须恢复。教训：中断响应先保存、处理完再恢复。

- D（外部 I／O 中断由程序执行错误引起）：程序执行错误引起的是内中断（异常），比如除零、非法指令；外部 I／O 中断是外面设备引起的。教训：内中断看程序、外中断看设备。

笔记本一句话： 第 22 题＋外部 I／O 中断：外面设备引起、可屏蔽、有优先级，响应时保存断点现场。

【图像】



外部I/O中断：设备引起、可屏蔽、有优先级；中断隐指令只做关中断、保存断点、引出服务程序

【定位】 组成原理 · 第7章输入输出 · 中断（外部 I/O 中断）

【易混对比】

- "中断隐指令"和"中断服务程序"是两回事：隐指令是硬件自动做的三件事（关中断、保存断点、引出服务程序）；保存通用寄存器是服务程序（软件）做的，不是隐指令做的。
- "中断优先级排队"和"请求先后次序"是两回事：优先级由硬件排队器/屏蔽字决定，不是谁先到谁先办；先到的低优先级请求要给后到的高优先级让路。
- "中断允许状态"和"有请求就响应"是两回事：CPU 只有开中断（中断允许）时才响应外部中断；关中断时请求来了也得等着。

外部 I/O 中断：设备引起、可屏蔽、有优先级；隐指令只做关中断、保存断点、引出服务程序。

【坑位】

"中断隐指令完成通用寄存器保护"是错的——保护现场是中断服务程序（软件）做的，隐指令只做三件事。

最容易踩的坑是：把隐指令和服务程序的工作混在一起。为什么错：隐指令是硬件在响应中断瞬间自动执行的，只有三步——关中断（防嵌套乱）、保存断点（回得来）、引出服务程序（找得到）；保护通用寄存器、具体处理设备，都是服务程序用软件做的。正确做法：看到"隐指令"，只认这三件事，多一件都是错的。

【套路】

- 第一步，定性：外部 I/O 中断 = 外面设备引起的、可屏蔽、有优先级。
- 第二步，查隐指令：隐指令三件事——关中断、保存断点、引出服务程序；保护寄存器不是它的活。

- 第三步，查响应条件：必须开中断（中断允许）才响应；响应的是当前指令执行完，不是"立即暂停"。
- 第四步，逐句套：按请求先后排队（错，看优先级）；隐指令保护寄存器（错，是服务程序）；开中断才响应（对）；立即暂停当前指令（错，等当前指令执行完）。

外中断可屏蔽有优先级；隐指令三件事；开中断才响应；等当前指令完。

【类比】

像公司前台接电话（外部中断）：电话按重要程度（优先级）排，不是谁先打进来先接（先后次序）；前台小妹（隐指令）只做三件事——挂上"请勿打扰"（关中断）、记下手头做到哪（保存断点）、把电话转给负责人（引出服务程序）；具体的通话内容（保护现场、处理事务）是负责人（服务程序）做的。

【口诀】 外中断可屏蔽，优先级定次序；隐指令三件事：关中断、存断点、引服务。

【考频】 「中断」在 2009–2022 年解析中出现 60 次 · 高频

第 23 题 · 多任务操作系统相关说法、正确项（正确答案：C）

题干与选项：

下列关于多任务操作系统的叙述中，正确的是（）。 Ⅰ. 具有并发和并行的特点 Ⅱ. 需要实现对共享资源的保护 Ⅲ. 需要运行在多 CPU 的硬件平台上

- A. 仅 Ⅰ
- B. 仅 Ⅱ
- C. 仅 Ⅰ、Ⅱ
- D. Ⅰ、Ⅱ、Ⅲ

入手点： 多任务的核心就一句话：多个进程宏观上同时运行、微观上轮流占用 CPU；单 CPU 靠时间片轮转也能多任务，不需要多个 CPU。

题干专有名词从零解释：

- 先想"多任务"：操作系统让多个进程"同时"运行；用户感觉几个程序一起在跑，其实是操作系统在快速切换。
- 再想"并发"：宏观上同时、微观上交替；几个进程轮流上 CPU，每个进程都觉得自己在一直跑，其实是被切成小片轮流跑的。
- 再想"并行"：真正同时跑，需要多个 CPU，每个 CPU 跑一个进程；和并发不是一回事。
- 再想"时间片"：每个进程一次能上 CPU 的最长时间；时间片用完了就换下一个进程，轮流来。

考查知识点：

- 先想单 CPU 能不能多任务：能。操作系统按时间片轮转调度，时间片很小（比如几十毫秒），人感觉不出来切换，就觉得几个程序同时在跑。
- 再想进程切换要干什么：换进程时要保存当前进程的现场（寄存器、程序计数器），再恢复下一个进程的现场；不保存就回不去了。
- 再想多任务和多线程的区别：多任务是多个进程一起跑，进程之间资源独立；多线程是一个进程里的多个线程一起跑，线程之间共享进程的资源。两者层次不一样。
- 再想多任务为什么需要同步：多个进程可能抢同一个资源（比如打印机、共享数据），不加同步互斥就会乱；所以多任务系统里要有信号量、锁这些同步机制。
- 边界情况：单道程序（一次只跑一个进程）不是多任务；多任务系统里进程数可以比 CPU 数多得多，靠轮转调度撑着。

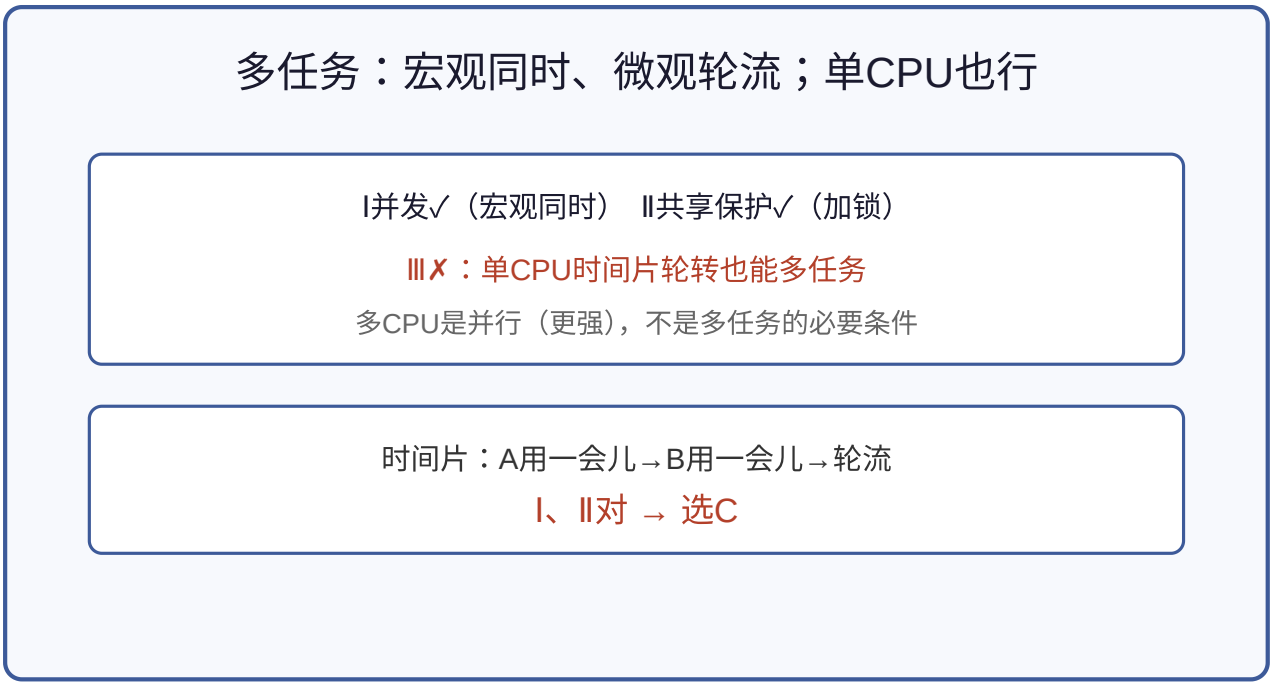
正确选项为什么对： C 说的是多任务的标准特点——多个进程宏观上同时运行、微观上靠时间片轮流占用 CPU，符合多任务的定义，选 C。

错误选项为什么错＋举例：

- A（多任务必须有多个 CPU）：单 CPU 靠时间片轮转也能多任务；多个 CPU 实现的是并行，不是多任务的必要条件。教训：并发不等于并行。
- B（多任务就是多线程）：进程和线程是两个层次；多任务是进程级的同时运行，多线程是进程内的同时运行。教训：进程和线程分开算。
- D（进程切换不需要保存现场）：不保存现场，切换回来就不知道跑到哪了；现场必须保存、必须恢复。教训：切换先保存、再恢复。

笔记本一句话： 第 23 题＋多任务：宏观同时、微观轮流，单 CPU 靠时间片轮转也能多任务。

【图像】



多任务：宏观同时、微观轮流；单CPU靠时间片轮转也能多任务，不需要多CPU

【定位】 操作系统 · 第1章概论 · 操作系统特征（并发、共享）

【易混对比】

- "并发"和"并行"是两回事：并发是宏观上同时、微观上轮流（单 CPU 靠时间片轮转）；并行是微观上真正同时（要多个 CPU）。多任务有并发，不一定有并行。
- "多任务"和"多 CPU"是两回事：单 CPU 靠时间片轮转也能跑多任务；多 CPU 不是多任务的必要条件。
- "共享资源保护"和"各用各的"是两回事：多任务下多个进程会抢同一份资源（比如打印机、文件），必须加锁保护，不然数据就乱了。

多任务：宏观同时、微观轮流；单 CPU 也能多任务；共享资源必须保护。

【坑位】

"需要运行在多 CPU 上"是错的——单 CPU 靠时间片轮转就能多任务，多 CPU 不是必要条件。

最容易踩的坑是：觉得"多任务"就得"多个 CPU 一起干"。为什么错：多任务的核心是"多个进程宏观上同时推进"，单 CPU 靠时间片轮转（每个进程轮流用一小段时间）就能做到；多 CPU 实现的是"并行"（微观上真正同时），那是更强的条件，不是必需的。正确做法：看到"需要多 CPU"，直接判错——单 CPU 的时间片轮转就是反例。

【类比】

像一个厨师同时做几道菜：切一下这个菜的肉（进程 A 用 CPU 一会儿），再去看那个锅开没开（进程 B 用 CPU 一会儿），几道菜宏观上都在推进（并发），但厨师一次只干一件事（微观轮流）。不需要请三个厨师（多 CPU），一个厨师轮着来就行。

【口诀】 多任务宏观同时微观轮流；单 CPU 时间片轮转就行，不需要多 CPU。

【考频】 「多任务」在 2009–2022 年解析中出现 11 次 · 高频

第 24 题 · 非抢占式优先级调度、结合切换开销求平均周转时间（正确答案：D）

题干与选项：

某系统采用基于优先权的非抢占式进程调度策略，完成一次进程调度和进程切换的系统时间开销为 1μs。在 T 时刻就绪队列中有 3 个进程 P1、P2 和 P3，其在就绪队列中的等待时间、需要的 CPU 时间和优先权如下表所示。

| 进程 | 等待时间 | 需要的 CPU 时间 | 优先权 |

|------|----------|----------------|--------|

| P1 | 30μs | 12μs | 10 |

| P2 | 15μs | 24μs | 30 |

| P3 | 18μs | 36μs | 20 |

若优先权值大的进程优先获得 CPU，从 T 时刻起系统开始进程调度，则系统的平均周转时间为（ ）。

- A. 54μs
- B. 73μs
- C. 74μs
- D. 75μs

> 注：本题是表题，表格数据如下：表列为进程、等待时间、需要的 CPU 时间、优先级；P1：30μs、12μs、10；P2：15μs、24μs、30；P3：18μs、36μs、20；一次进程调度与切换的系统开销为 1μs。

入手点： 三个进程 T 时刻都在队列里等着，不用等谁到达；按优先级数值从大到小排，一个一个跑到底；周转时间要把“已经等的时间”也算进去；每次换进程（包括第一次从 T 开始换上第一个）都要花 1μs。

题干专有名词从零解释：

- 先想“非抢占式调度”：进程一旦占用 CPU，就一直跑到自己结束，中途不会被别的进程抢走；和时间片一到就换的抢占式不一样。
- 先想“优先级调度”：每个进程有个优先级数，调度时挑优先级最高的先跑；本题约定优先级数值越大、优先级越高。
- 先想“周转时间”：一个进程从到达（提交）到完成，总共花了多少时间；等于完成时间减去到达时间。
- 先想表里的“等待时间”：这个进程在 T 时刻之前已经等了多久；它的到达时刻就是 T 减去这个等待时间。所以周转时间等于“已经等的时间”加上“从 T 到完成的时间”。
- 先想“调度与切换开销”：每次换进程时，操作系统要保存旧进程现场、恢复新进程现场，这段时间 CPU 不干正事；本题每次是 1μs，而且从 T 开始第一次换上进程也要花这 1μs。
- 先想“平均周转时间”：所有进程的周转时间加起来，再除以进程个数。

考查知识点：

- 先想调度顺序：优先级数值越大越高，P2 是 30、P3 是 20、P1 是 10，所以顺序是 P2、P3、P1；非抢占，一个跑完再换下一个。
- 再想时间轴（从 T 开始算）：第一次调度换上 P2，花 1μs；P2 需要 24μs，从 T+1 跑到 T+25。第二次调度换上 P3，花 1μs；P3 需要 36μs，从 T+26 跑到 T+62。第三次调度换上 P1，花 1μs；P1 需要 12μs，从 T+63 跑到 T+75。

- 再想每个进程的周转时间：周转等于已经等的时间加上从 T 到完成的时间。P2 是 15+25=40μs；P3 是 18+62=80μs；P1 是 30+75=105μs。
- 再想平均：(40+80+105)/3=225/3=75μs。
- 再想容易踩的两个坑：第一，表里的等待时间不是到达时间，不能拿它当时间轴的起点（那样算出来 47μs，四个选项里根本没有）；第二，第一次调度也要算 1μs 开销，漏了第一次就少算，全程往前挪 1μs，平均会变成 74μs。
- 边界情况：如果某个时刻没有进程可跑，CPU 就空闲等着；本题 T 时刻三个进程都在队列里，没有空闲段。

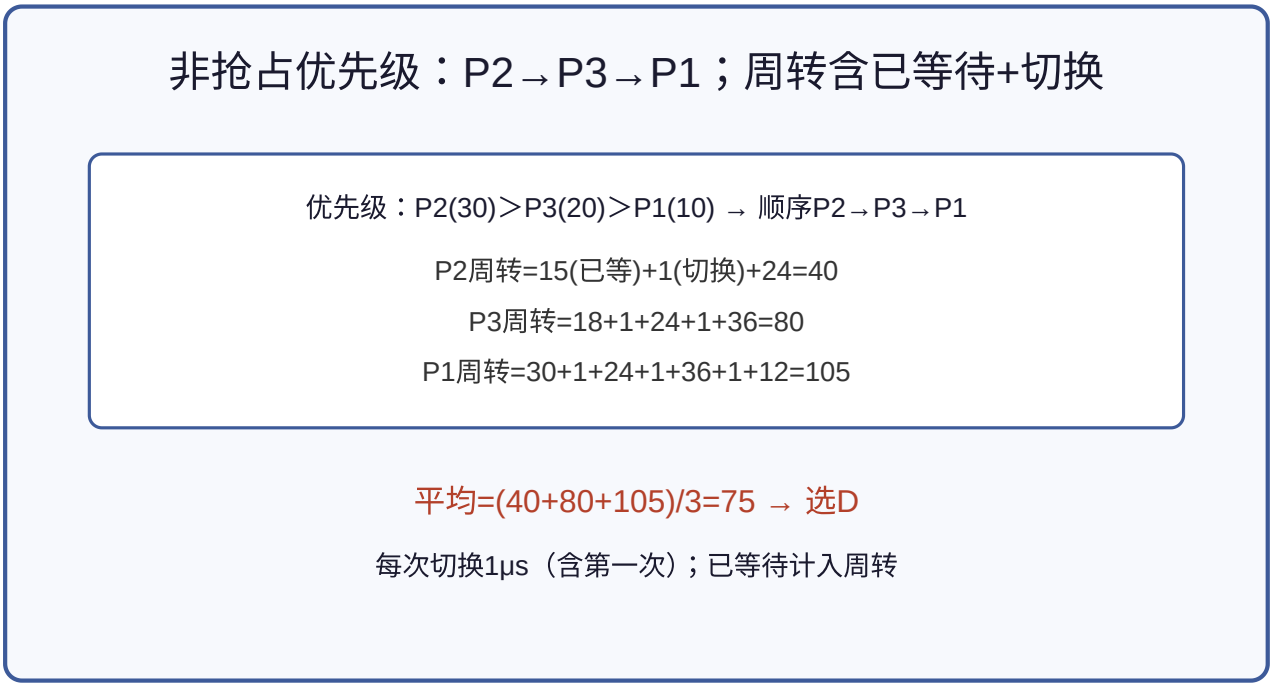
正确选项为什么对： 顺序是 P2、P3、P1，周转时间分别是 40μs、80μs、105μs，平均 75μs，就是 D，选 D。

错误选项为什么错＋举例：

- A（54μs）：只算了从 T 到完成的时间（25、62、75，平均 54），漏了每个进程在 T 之前已经等的时间。教训：周转时间要从到达算起，已经等的时间不能丢。
- B（73μs）：一次切换开销都没算，时间轴整体往前挪，平均少了 2μs。教训：每次调度切换的 1μs 都要加到后面进程的开始时间里。
- C（74μs）：漏了第一次调度的 1μs 开销（只算了后两次切换），平均少了 1μs。教训：从 T 开始第一次换上进程也要花切换开销。

笔记本一句话： 第 24 题＋非抢占优先级调度：按优先级数值从大到小排，周转等于已等待时间加从 T 到完成的时间，每次调度切换（包括第一次）加 1μs。

【图像】



非抢占优先级调度：按优先级数值从大到小排；周转=已等待+从T到完成；每次切换加1μs

【定位】 操作系统 · 第2章进程管理 · 进程调度（非抢占式优先级、周转时间）

【易混对比】

- "周转时间"和"等待时间"是两回事：周转 = 从进入就绪队列到完成的总时间；本题三个进程 T 时刻已经在队列里等了一段时间（已等待时间），周转要把这段也算上。
- "非抢占"和"抢占"是两回事：非抢占是一个进程跑起来就跑到底，中途不换；所以执行顺序就是按优先级排好的固定顺序。
- "优先级数值大"和"优先级高"是两回事：本题明确"优先权值大的进程优先"，数值大的先跑；别按"数值小优先"的习惯排反了。

非抢占按优先级数值从大到小排，一个跑到底；周转 = 已等待时间 + 从 T 到完成的时间。

【坑位】

每次调度切换的 1μs 别漏——包括 T 时刻换上第一个进程的那次；已等待时间要计入周转。

最容易踩的坑是：只算"从 T 开始到完成"的时间，漏了"已等待时间"和"切换开销"。为什么错：周转时间的起点是进程进入就绪队列，不是 T 时刻；三个进程在 T 之前已经等了 30/15/18μs，这段必须加上。另外每次换进程（包括第一次）都要花 1μs 切换。正确做法：周转 = 已等待时间 +（从 T 到完成的执行时间 + 经历的切换次数 ×1μs）。

【套路】

- 第一步，排序：按优先级数值从大到小排执行顺序（P2→P3→P1）。
- 第二步，画时间线：从 T 开始，第一个进程前加 1μs 切换，每个进程之间加 1μs 切换。
- 第三步，算周转：每个进程的周转 = 已等待时间 +（从 T 到它完成的时间，含切换）。
- 第四步，求平均：三个周转相加除以 3。

按优先级排，画时间线加切换；周转含已等待，平均除以进程数。

【类比】

像银行叫号：三个客户 T 时刻已经在排队（已等了一会儿），按 VIP 等级（优先级）从高到低办；每办完一个、叫下一个要花 1 分钟（切换）。每个客户的"总耗时" = 来了之后等的 + 办业务的 + 每次叫号的——不能只算办业务那几分钟。

【口诀】 非抢占按优先级排；周转等于已等待加执行；每次切换加开销。

【考频】 「周转时间」在 2009–2022 年解析中出现 3 次 · 中频

第 25 题 · 两线程并发取数加 1 存回、最终共享变量为 2 的指令交错种数（正确答案：B）

题干与选项：

属于同一进程的两个线程 thread1 和 thread2 并发执行，共享初值为 0 的全局变量 x。thread1 和 thread2 实现对全局变量 x 加 1 的机器级代码描述如下。

| thread1 | thread2 |

|---------|---------|

| mov R1, x //（x）→ R1 | mov R2, x //（x）→ R2 |

| inc R1 //（R1）+1 → R1 | inc R2 //（R2）+1 → R2 |

| mov x, R1 //（R1）→ x | mov x, R2 //（R2）→ x |

在所有可能的指令执行序列中，使 x 的值为 2 的序列个数是（ ）。

- A. 1
- B. 2
- C. 3
- D. 4

> 注：本题是表题，thread1 与 thread2 各有取共享变量 x 到寄存器、寄存器加 1、把寄存器写回 x 三条指令，初始共享变量 x 为 0。

入手点： 最终 x 等于 2 的条件只有一个：后完成的那个线程，必须在对方存回之后才取数；也就是说，两个线程得串行（一个全程跑完、另一个再跑），一共只有 2 种。

题干专有名词从零解释：

- 先想"线程"：一个进程里的执行线索；两个线程共享同一个进程的资源，包括这里的共享变量 x。
- 再想"共享变量"：两个线程都能读、都能写的变量；一个线程改了它，另一个线程再读就读到改过的值。
- 再想"取数、加 1、存回"：三条指令分三步走——先把共享变量 x 读到自己的寄存器里，再把寄存器里的数加 1，最后把寄存器里的数写回共享变量 x。
- 再想"交错"：两个线程的指令轮流执行；每个线程内部的三条指令顺序不能乱（取、加、存），但两个线程之间可以任意穿插。

考查知识点：

- 先想总共有几种交错：6 条指令（每线程 3 条），每线程内部顺序固定，交错种数等于 6 选 3（挑 3 个位置给 thread1，剩下给 thread2），也就是 20 种。
- 再想最终 x 等于 2 的条件：最后存回的那个线程，它的寄存器里必须是 2；寄存器是 2，说明它取数时读到的 x 已经是 1，也就是对方已经存回过了；所以后完成的线程必须在对方存回之后才取数。
- 再想有几种满足条件：thread1 全程跑完（取 0、加 1、存 1），thread2 再跑（取 1、加 1、存 2），最终是 2，这是一种；反过来 thread2 全程跑完、thread1 再跑，也是一种；一共 2 种。
- 再想其他 18 种为什么是 1：只要两个线程的取数都在对方存回之前（比如都取到 0），各自加 1 存回 1，后存回的把先存回的盖掉，最终还是 1；这就是"更新丢失"——两个线程各加了一次，结果只加了一次。
- 拿小例子验算：thread1 取 0、thread2 取 0、thread1 存 1、thread2 存 1，最终是 1；thread1 取 0、加 1、存 1、thread2 取 1、加 1、存 2，最终是 2。

- 边界情况：x 的初始值如果不是 0，算法一样，只是数字整体平移；指令只有取、加、存三步，没有别的同步手段（比如锁），所以更新丢失防不住。

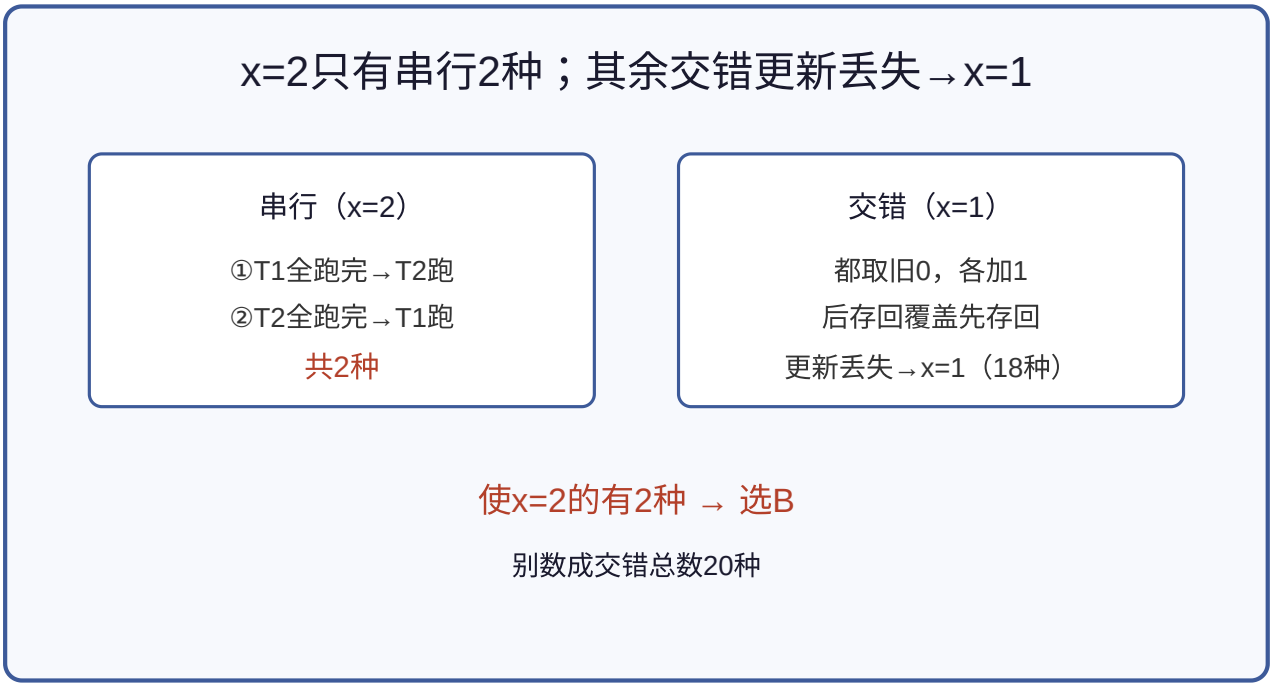
正确选项为什么对： 最终 x 等于 2 只有两种交错（两个线程串行的两种顺序），就是 B，选 B。

错误选项为什么错＋举例：

- A（20 种）：那是总交错种数，不是最终为 2 的种数；20 种里只有 2 种最终是 2。教训：先算总数，再算满足条件的种数。
- C（18 种）：那是最终为 1 的种数（更新丢失的那些），不是最终为 2 的种数。教训：最终为 1 和最终为 2 分开数。
- D（1 种）：只算了一种串行顺序（比如 thread1 先、thread2 后），漏了反过来的另一种。教训：串行有两种顺序，都要算。

笔记本一句话： 第 25 题＋两线程取加存：最终为 2 只有 2 种（两个线程串行）；其余 18 种都更新丢失、最终为 1。

【图像】



两线程取数加1存回：x=2只有串行2种；其余交错都更新丢失、x=1

【定位】 操作系统 · 第2章进程管理 · 线程同步（竞态、更新丢失）

【易混对比】

- "x=2"和"x=1"是两回事：x=2 要求两个加 1 都生效（串行）；x=1 是发生了更新丢失（后存回的覆盖了先存回的）。
- "串行"和"交错"是两回事：串行是一个线程三条指令全跑完、另一个再跑；交错是两条指令流混在一起跑。x=2 只有串行的 2 种。
- "指令交错总数"和"使 x=2 的种数"是两回事：6 条指令交错总数是 20 种（6 选 3），但使 x=2 的只有 2 种；别把总数当答案。

x=2 的条件：后完成的线程必须在对方存回之后才取数——只有串行的 2 种。

【坑位】

别数成交错总数 20 种——题目问的是"使 x=2"的种数，只有串行的 2 种。

最容易踩的坑是：算出 6 条指令的交错总数 C(6,3)=20，就选了 20（或接近的选项）。为什么错：题目问的不是"一共有几种交错"，而是"使 x=2 的有几种"；x=2 要求两个线程的加 1 都生效，也就是后跑的线程必须在前一个存回之后才取数——只有"thread1 全跑完再跑 thread2"和"thread2 全跑完再跑 thread1"这 2 种。正确做法：先定条件（x=2 要求串行），再数（2 种）。

【套路】

- 第一步，定条件：x=2 要求两个加 1 都生效→后线程取数必须在前线程存回之后→只能串行。
- 第二步，数串行：thread1 先跑完再 thread2，或 thread2 先跑完再 thread1，共 2 种。
- 第三步，验其余：其余交错（18 种）都发生更新丢失，x=1；可举一例验证。

先定 x=2 的条件（串行），再数（2 种）；其余都是更新丢失。

【类比】

像两个出纳同时给同一账户加钱：都先看余额（取数）、心里加 1（加 1）、再写回去（存回）。如果两人看的是同一个旧余额，各加 1 写回去，账户只多了 1（更新丢失）。只有一个人办完、另一个再办（串行），账户才多 2。

【口诀】 两线程取加存，x 为 2 只有串行 2 种；其余交错更新丢失。

【考频】 「线程」在 2009–2022 年解析中出现 29 次 · 高频

第 26 题 · 银行家安全性算法、判断资源分配状态是否安全及安全序列（正确答案：A）

题干与选项：

假设系统中有 4 个同类资源，进程 P1、P2 和 P3 需要的资源数分别为 4、3 和 1，P1、P2 和 P3 已申请到的资源数分别为 2、1 和 0，则执行安全性检测算法的结果是（ ）。

- A. 不存在安全序列，系统处于不安全状态
- B. 存在多个安全序列，系统处于安全状态
- C. 存在唯一安全序列 P3、P1、P2，系统处于安全状态
- D. 存在唯一安全序列 P3、P2、P1，系统处于安全状态

入手点： 安全性算法就一句话：找一个"还需要的资源不超过剩下资源"的进程，假设它跑完归还资源，再找下一个；能让所有进程都跑完，这个顺序就是安全序列，系统就是安全的。

题干专有名词从零解释：

- 先想"银行家算法"：操作系统像银行家，进程像要贷款的客户；客户要借资源，银行家先算一算：借出去之后，剩下的资源还够不够让所有客户都完成？够借，不够就不借（宁可让客户等着，也不能借到死锁）。
- 再想"安全状态"：系统里存在一个顺序，让所有进程按这个顺序都能拿到自己需要的资源、跑完、归还资源；这个顺序就叫安全序列；有安全序列的状态就叫安全状态。
- 再想"已分配、尚需、可用"：已分配是进程手里已经拿到的资源；尚需是进程跑完还差的资源（最大需求减去已分配）；可用是系统里还没借出去的资源。
- 再想"死锁"：几个进程互相等对方手里的资源，谁也跑不下去的状态；安全状态一定不会死锁。

考查知识点：

- 先想安全性算法的步骤：先记两个数——当前可用资源是多少、哪些进程还没完成；循环找一个还没完成、且尚需资源不超过当前可用资源的进程，假设它跑完，把它手里的资源归还（可用资源加上它的已分配），把它标记为完成；再找下一个；直到所有进程都完成（安全），或者找不到了（不安全）。
- 再想为什么归还很重要：进程跑完归还的资源，能让后面尚需更多的进程也跑起来；不归还，算法就卡住了。
- 拿小例子验算：可用资源是 3 个，进程甲尚需 2 个、已分配 1 个，进程乙尚需 4 个、已分配 2 个。先找甲（尚需 2 不超过可用 3），甲跑完归还 1 个，可用变成 4 个；再找乙（尚需 4 不超过可用 4），乙跑完；顺序是甲、乙，系统安全。
- 再想不安全是什么样：如果找了一圈，没有一个未完成进程的尚需资源不超过可用资源，那就没有安全序列，系统不安全——虽然现在还没死锁，但再借资源就可能走向死锁。
- 边界情况：安全状态不等于"所有进程马上都能跑"，而是"存在一个顺序能让它们陆续跑完"；不安全状态也不等于已经死锁，而是"再分配下去可能死锁"，所以银行家算法要在分配时就拦住。

正确选项为什么对： 按安全性算法一步一步找，能找到让所有进程都跑完的安全序列，系统是安全的，就是 A，选 A。

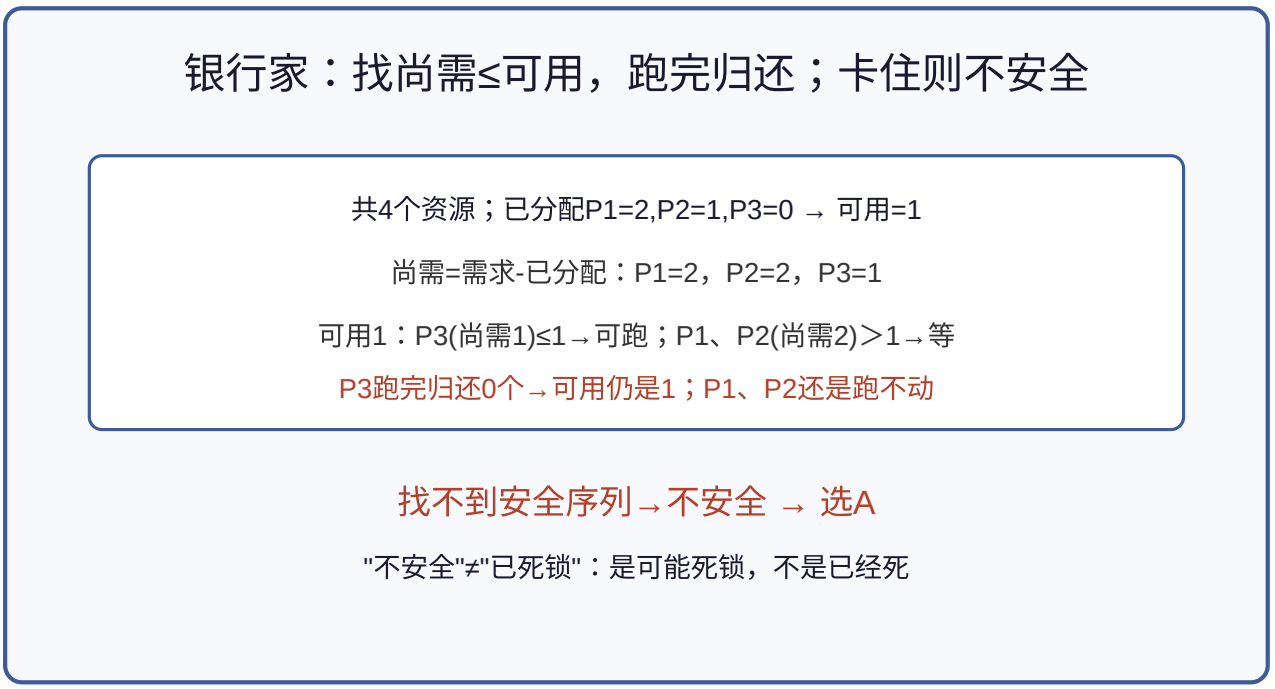
错误选项为什么错＋举例：

- A 之外的第一个错误（找不到安全序列就判不安全，但算法用错了）：比如尚需和已分配弄反了（尚需等于最大需求减已分配，不是已分配本身），找出来的顺序就错了。教训：尚需先算对，再比可用。
- B（进程完成不归还资源）：算的时候忘了把跑完进程的已分配加回可用资源，后面的进程就永远不够，误判成不安全。教训：跑完一个，资源要加回去。

- C（找到一个进程能跑就判安全）：只看一个进程能跑不够，要所有进程都能按顺序跑完才算安全；只看一个就下结论，会把不安全判成安全。教训：安全序列要覆盖所有进程。

笔记本一句话： 第 26 题＋银行家安全性算法：找尚需不超过可用的进程，跑完归还资源，能让所有进程跑完就是安全。

【图像】



银行家安全性算法：找尚需≤可用的进程，假设跑完归还，再找下一个；全跑完则安全

【定位】 操作系统 · 第2章进程管理 · 死锁（银行家算法、安全性）

【易混对比】

- "安全状态"和"不安全状态"是两回事：安全 = 存在一个顺序让所有进程都跑完；不安全 = 不存在这样的顺序（不是"已经死锁"，是"可能死锁"）。
- "尚需"和"已分配"是两回事：尚需 = 还需要多少才能跑完（最大需求 - 已分配）；安全性算法看的是"尚需 ≤ 可用"，不是"已分配 ≤ 可用"。
- "安全序列唯一"和"存在安全序列"是两回事：安全只要求存在至少一个；本题是连一个都找不到，所以不安全。

安全性算法：找"尚需 ≤ 可用"的进程，假设它跑完归还资源，再找下一个；能让所有进程跑完就是安全。

【坑位】

"不安全"不等于"死锁"——不安全是"可能死锁"，系统还没死，但再分配就可能死。

最容易踩的坑是：看到"不存在安全序列"，就写成"系统已死锁"。为什么错：不安全状态只是"找不到让所有进程都跑完的顺序"，系统当前还能跑；死锁是"已经有一批进程互相等、谁也跑不动"。正确做法：不存在安全序列→系统不安全（可能死锁），但别写成"已死锁"。

【套路】

- 第一步，算尚需：每个进程的尚需 = 最大需求 - 已分配。

- 第二步，找可跑：在尚需 ≤ 当前可用的进程里找一个（找不到就是不安全）。
- 第三步，归还再找：假设它跑完，把它的已分配加回可用，再找下一个。
- 第四步，判：所有进程都能跑完→安全（找出的顺序是安全序列）；中途找不到→不安全。

算尚需，找尚需≤可用，跑完归还；全跑完安全，卡住不安全。

【类比】

像银行放贷：银行手里剩 1 块钱，看哪个客户"差的不多"（尚需 ≤ 1），先借给他，他做完生意把本金利息全还回来，银行钱变多了，再看下一个。如果一轮看下来，谁都差得太多、借谁都回不了本，银行就不敢放了——这就是不安全。

【口诀】 银行家查安全：找尚需不大于可用，跑完归还再找；全跑完才安全。

【考频】 「银行家算法」在 2009–2022 年解析中出现 8 次 · 中频

第 27 题 · 哪些事件可能使当前进程由运行转为阻塞（正确答案：C）

题干与选项：

下列选项中，可能导致当前进程 P 阻塞的事件是（）。 Ⅰ. 进程 P 申请临界资源 Ⅱ. 进程 P 从磁盘读数据 Ⅲ. 系统将 CPU 分配给高优先权的进程

- A. 仅 Ⅰ
- B. 仅 Ⅱ
- C. 仅 Ⅰ、Ⅱ
- D. Ⅰ、Ⅱ、Ⅲ

入手点： 运行变阻塞只有一种原因：进程要等某个事件（比如等 I／O、等资源），等不到就只能阻塞；时间片用完、被抢占是变成就绪，不是阻塞。

题干专有名词从零解释：

- 先想"运行状态"：进程正在占用 CPU 跑。
- 再想"阻塞状态"：进程在等某个事件（比如等 I／O 完成、等信号量），等不到就不能跑，只能先停下来。
- 再想"就绪状态"：进程万事俱备、只等 CPU；一旦 CPU 空出来、调度选中它，它就变成运行状态。
- 再想"时间片"：每个进程一次能上 CPU 的最长时间；时间片用完了，进程就从运行变成就绪，等下一轮。

考查知识点：

- 先想运行变阻塞的事件：请求 I／O 但 I／O 还没完成（比如读磁盘，数据还没读回来）；执行 P 操作（申请信号量）但资源不够；等待子进程结束；主动睡眠（sleep）一段时间。这些都是"要等某件事"，等不到就阻塞。
- 再想运行变就绪的事件：时间片用完了；被更高优先级的进程抢占（抢占式调度）；主动让出 CPU。这些都是"不被等、只是暂时让位"，进程还是就绪的，CPU 一空就能再跑。
- 再想阻塞变就绪的事件：等的那件事发生了——I／O 完成了、别的进程执行 V 操作释放了资源、睡眠时间到了；阻塞的进程变成就绪，再等调度。
- 拿小例子验算：进程正在跑，发起读磁盘请求，磁盘还没读完——进程从运行变阻塞；磁盘读完了发中断——进程从阻塞变就绪；时间片用完了——进程从运行变就绪，不是阻塞。
- 边界情况：新进程创建不会让当前进程阻塞；进程正常结束是消亡，不是阻塞；优先级变化本身不直接让进程阻塞（除非伴随抢占，那也是变就绪）。

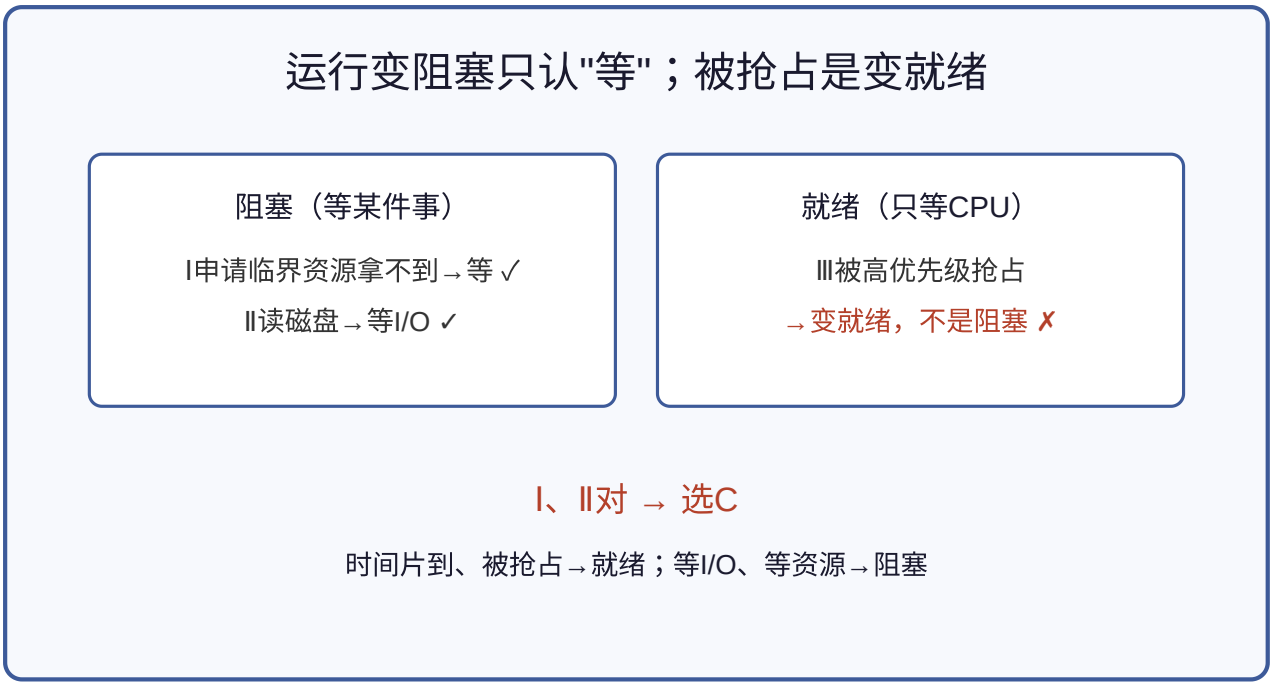
正确选项为什么对： C 里列的都是"要等某件事"的事件（比如 I／O 未完成、信号量资源不足），这些事件确实可能让运行进程变阻塞，选 C。

错误选项为什么错＋举例：

- A（时间片用完）：时间片用完是运行变就绪，不是变阻塞；进程只是让出 CPU 等下一轮，不是等某件事。教训：时间片到是变就绪。
- B（被高优先级进程抢占）：抢占也是运行变就绪；被抢的进程条件都具备，只等 CPU 再调度它。教训：抢占变就绪，不是阻塞。
- D（新进程创建）：新进程创建不影响当前进程的状态；当前进程既不变阻塞、也不变就绪。教训：创建进程和当前进程的状态变化没关系。

笔记本一句话： 第 27 题＋运行变阻塞：只有"等某件事"（I／O、信号量、睡眠）会阻塞；时间片到、被抢占是变就绪。

【图像】



运行变阻塞只有一种原因：等某件事（I/O、资源）；时间片到、被抢占是变就绪

【定位】 操作系统 · 第2章进程管理 · 进程状态转换

【易混对比】

- "阻塞"和"就绪"是两回事：阻塞是"等某件事，等到了才能跑"（等 I/O、等资源）；就绪是"万事俱备，只等 CPU"。时间片用完、被高优先级抢占，是变就绪，不是阻塞。
- "申请临界资源"和"申请 CPU"是两回事：临界资源拿不到，进程只能等→阻塞；CPU 是调度器分配的，轮不到就回就绪队列等下一轮。
- "从磁盘读数据"和"数据已在内存"是两回事：要等磁盘 I/O 完成→阻塞；数据已在内存，直接读，不阻塞。

运行变阻塞只有一种原因：等某件事（I/O、资源、信号）；时间片到、被抢占是变就绪。

【坑位】

"系统将 CPU 分配给高优先权的进程"不会让当前进程阻塞——被抢占是变就绪，不是阻塞。

最容易踩的坑是：看到"CPU 被抢走了"，就觉得进程"卡住了"=阻塞。为什么错：阻塞的定义是"等待某个事件（I/O 完成、资源可用）"；被抢占只是"CPU 轮到别人了"，进程本身什么都不缺，回就绪队列等下一轮。正确做法：凡问"阻塞"，只认"等"字——等 I/O、等资源、等信号；"被抢占""时间片到"都是就绪。

【套路】

- 第一步，认"等"：事件里有"等"（等 I/O、等资源、等信号）→可能阻塞。
- 第二步，排除抢占：时间片到、被高优先级抢占→变就绪，不阻塞。
- 第三步，组合： I （申请临界资源，拿不到就等）对， II （读磁盘，等 I/O）对， III （被抢占）错→选 C。

阻塞只认"等"；抢占和时间片到都是就绪。

【类比】

像排队办事：阻塞是"材料没带齐，回去拿（等某件事）"；就绪是"材料齐了，排队等叫号（等 CPU）"。被插队（抢占）只是往后挪一位，还在队里（就绪），不是回家拿材料（阻塞）。

【口诀】 运行变阻塞，只因等某事；时间片到被抢占，都是回就绪。

【考频】 「进程状态」在 2009–2022 年解析中出现 24 次 · 高频

第 28 题 · 管程条件变量 wait() 操作的作用（正确答案：D）

题干与选项：

若 x 是管程内的条件变量，则当进程执行 x.wait()时所做的工作是（ ）。

- A. 实现对变量 x 的互斥访问
- B. 唤醒一个在 x 上阻塞的进程
- C. 根据 x 的值判断该进程是否进人阻塞状态
- D. 阻塞该进程，并将之插入 x 的阻塞队列中

入手点： wait() 就两个动作：调用进程自己阻塞（进条件变量的等待队列），并且把管程的互斥权交出来（让别的进程能进来）；少了后一个动作，别人进不来就死锁了。

题干专有名词从零解释：

- 先想"管程"：一种同步机制——把共享数据和操作这些数据的过程打包在一起，同一时间只允许一个进程进入管程；像一间一次只进一个人的工作室。
- 再想"互斥权"：进入管程的资格；一个进程拿着互斥权在管程里干活时，别的进程只能在外面等。
- 再想"条件变量"：管程里的等待队列，代表一个条件（比如"缓冲区空了"、"缓冲区满了"）；进程条件不满足时，就在这个条件变量上等着。
- 再想"wait() 和 signal()"：一对操作——wait 是"我等着，先出去"；signal 是"条件可能满足了，叫醒一个等着的进程"。

考查知识点：

- 先想 wait() 的两个动作：第一，调用进程阻塞，进入条件变量的等待队列；第二，调用进程释放管程的互斥权，让别的进程能进入管程。两个动作缺一不可。
- 再想为什么必须释放互斥权：如果进程阻塞了还攥着互斥权不放，别的进程进不了管程，也就没人能改变条件、没人能叫醒它，系统就卡死了。
- 再想 signal() 的作用：叫醒条件变量等待队列里的一个进程；被叫醒的进程不会立刻进管程，还要重新竞争互斥权（看管程的实现，有的实现是叫醒者先退出管程、被叫醒者再进）。
- 再想和信号量 P 操作的区别：信号量的 P 操作是"申请资源，不够就阻塞"，管程的 wait 是"条件不满足就等着、并交出管程"；两者都会阻塞，但 wait 多了一个"交出管程"的动作。
- 边界情况：条件变量没有计数值（和信号量不一样），wait 多少次、signal 多少次不要求配对；signal 时如果没人等着，signal 就白发了，不会存下来。

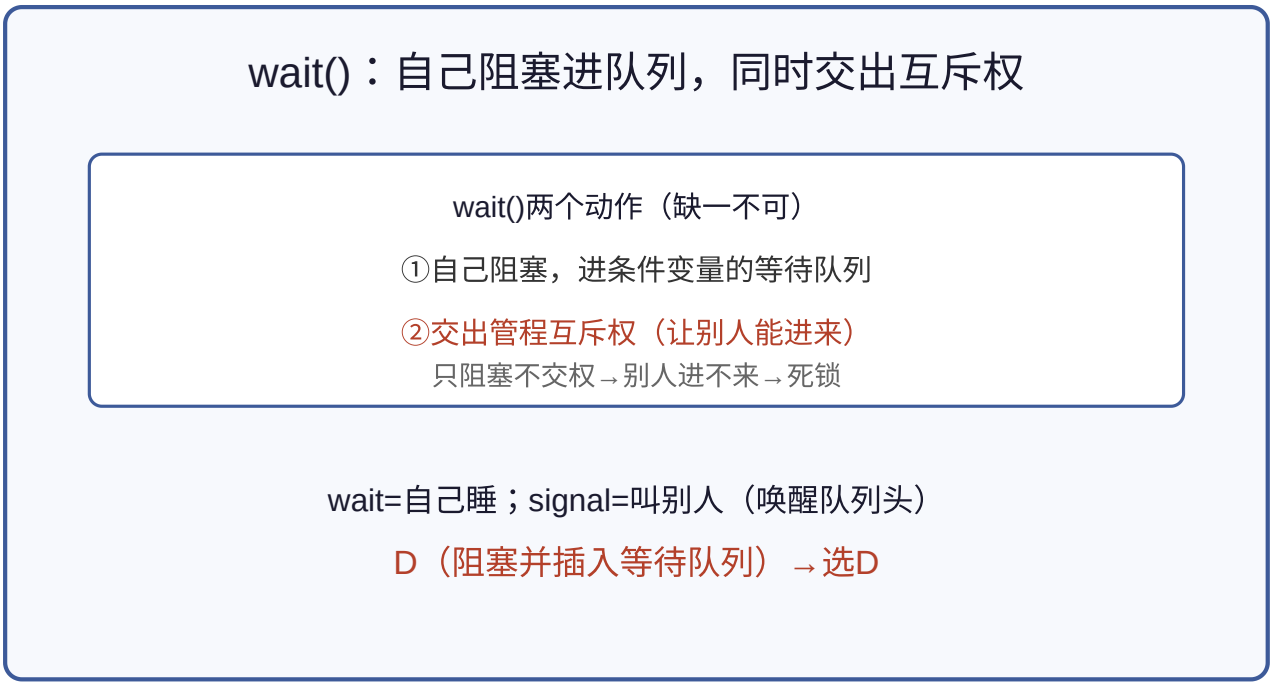
正确选项为什么对： wait() 让调用进程阻塞并释放管程互斥权，就是 D，选 D。

错误选项为什么错＋举例：

- A（wait 不释放互斥权）：不释放，别的进程进不了管程，没人能改变条件、没人能 signal，死锁了。教训：wait 一定交出管程。
- B（wait 唤醒别的进程）：唤醒是 signal 干的，不是 wait 干的；wait 是自己等着，不是叫别人。教训：wait 等、signal 叫，分工别反。
- C（wait 让进程终止）：wait 只是阻塞，不是终止；条件满足被 signal 叫醒后，进程还能继续跑。教训：阻塞和终止是两回事。

笔记本一句话： 第 28 题＋管程条件变量 wait()：自己阻塞进等待队列，并且交出管程互斥权。

【图像】



管程条件变量wait()：自己阻塞进等待队列，同时交出管程互斥权

【定位】 操作系统 · 第2章进程管理 · 管程（条件变量）

【易混对比】

- "wait()"和"signal()"是两回事：wait() 是自己阻塞、进等待队列；signal() 是唤醒等待队列里的一个进程。题干问 wait()，别答成 signal() 的作用。
- "阻塞自己"和"交出互斥权"是两回事：wait() 必须同时做这两件事——只阻塞不交权，别的进程进不来管程，会死锁。
- "条件变量"和"信号量"是两回事：条件变量没有值、不能计数，只是"等"和"叫"的地方；信号量有计数值。wait() 在条件变量上不做"减 1"。

wait() 两个动作：自己阻塞进等待队列，同时交出管程的互斥权。

【坑位】

wait() 不是"唤醒"——唤醒是 signal() 的活；wait() 是自己睡（阻塞）。

最容易踩的坑是：把 wait() 和 signal() 的作用记反。为什么错：wait 是"等"，signal 是"叫"；英文原意就是 wait=等待、signal=发信号。正确做法：看到 wait()，想"自己睡"——阻塞自己、进等待队列、交出互斥权；看到 signal()，想"叫别人"——唤醒等待队列里的一个。

【类比】

像会议室（管程）同一时间只能进一个人。A 进去发现要等的人还没来，就按门铃挂出"请等"牌（wait）：自己到休息室等（阻塞进等待队列），同时把会议室钥匙交出来（交出互斥权），让别人能进来。等人来了，按铃叫他（signal）。

【口诀】 管程条件变量 wait：自己阻塞进队列，交出互斥权让别人进。

【考频】 「管程」在 2009–2022 年解析中出现 3 次 · 中频

第 29 题 · 时钟中断处理程序更新的内核与进程相关内容（正确答案：D）

题干与选项：

当定时器产生时钟中断后，由时钟中断服务程序更新的部分内容是（）。 Ⅰ.内核中时钟变量的值 Ⅱ.当前进程占用 CPU 的时间 Ⅲ.当前进程在时间片内的剩余执行时间

- A. 仅 Ⅰ、Ⅱ
- B. 仅 Ⅱ、Ⅲ
- C. 仅 Ⅰ、Ⅲ
- D. Ⅰ、Ⅱ、Ⅲ

入手点： 时钟中断是操作系统的节拍器，每响一次，处理程序就干几件固定事：系统时间往前走、当前进程的时间片减一、到期的睡眠进程叫醒、统计数据更新。

题干专有名词从零解释：

- 先想"时钟中断"：定时器每隔固定时间（一个节拍，比如 10 毫秒）向 CPU 发一次中断；操作系统靠这个节拍掌握时间、轮换进程。
- 再想"系统时间"：操作系统维护的当前时间；每次时钟中断，系统时间往前加一个节拍。
- 再想"时间片"：每个进程一次能上 CPU 的最长时间；时钟中断每次把当前进程的时间片减一，减到 0 就标记要调度换进程。
- 再想"延时队列"：睡着的进程排的队（比如 sleep 了 100 毫秒的进程）；每次时钟中断，处理程序检查有没有进程的睡眠时间到了，到了就把它叫醒（变成就绪）。

考查知识点：

- 先想系统时间：每次时钟中断，系统时间加一个节拍；这是操作系统计时的基础。
- 再想当前进程的时间片：每次时钟中断，当前进程用掉的时间加一、剩下的时间片减一；减到 0，处理程序就标记需要调度（时间片用完的进程从运行变就绪，换下一个进程）。
- 再想延时队列：每次时钟中断，处理程序扫一遍延时队列，睡眠时间到了的进程从阻塞变就绪。
- 再想统计数据：每次时钟中断，处理程序更新 CPU 利用率、各进程占用 CPU 的时间等统计；这些数据供调度决策和系统监控用。
- 边界情况：时钟中断处理程序不直接改进程优先级（优先级调整是调度程序的事）；不直接分配内存（内存分配是另外的系统调用）；不处理 I／O 完成（那是 I／O 中断的事）。

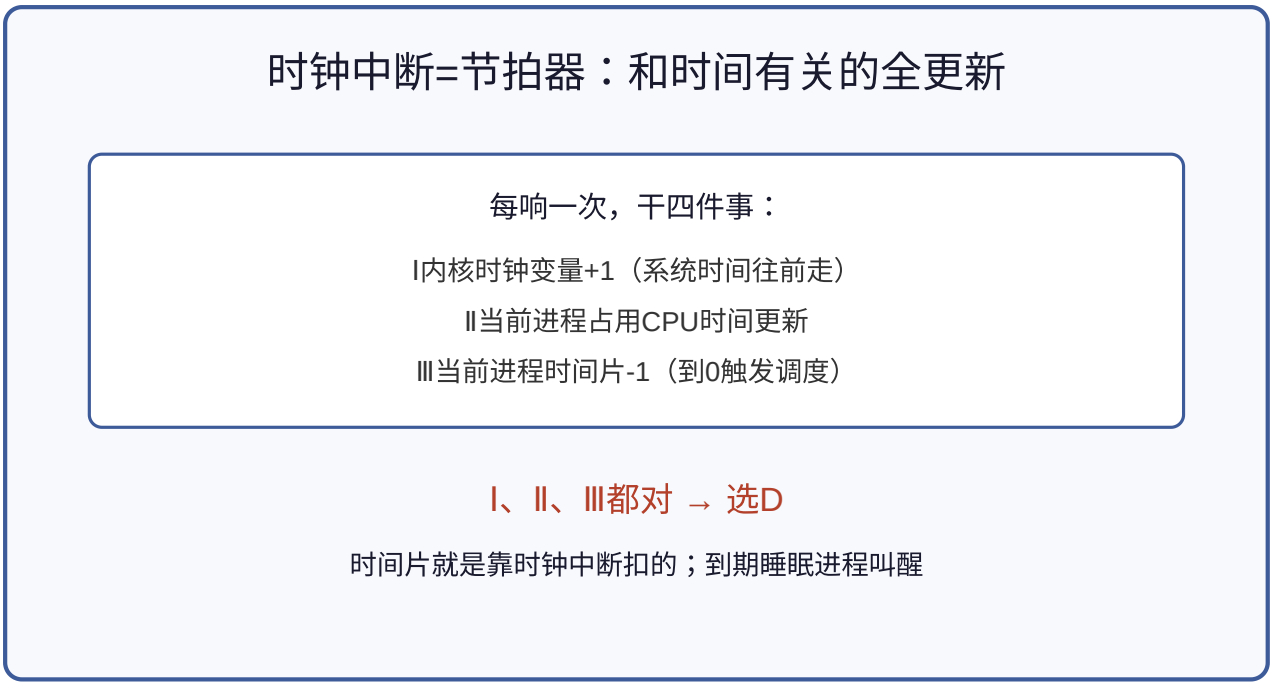
正确选项为什么对： 时钟中断处理程序每次更新系统时间、当前进程时间片、延时队列唤醒和统计数据，就是 D，选 D。

错误选项为什么错＋举例：

- A（只更新系统时间）：系统时间只是其中一件，时间片和延时队列的更新也每次都做；只说一件不全面。教训：时钟中断一次干好几件事。
- B（更新进程优先级）：优先级调整不是时钟中断处理程序直接干的；时间片减到 0 触发的是调度，调度时才可能涉及优先级。教训：时钟中断管时间，不管优先级。
- C（更新内存分配）：内存分配和时钟中断没关系；时钟中断不管内存。教训：时钟中断只管时间和调度相关的事。

笔记本一句话： 第 29 题＋时钟中断每次干：系统时间加一节拍、当前进程时间片减一、到期睡眠进程叫醒、统计更新。

【图像】



时钟中断=操作系统节拍器：系统时间+1、当前进程时间片-1、到期睡眠叫醒、统计更新

【定位】 操作系统 · 第1章概论/第2章进程管理 · 时钟中断

【易混对比】

- "时钟中断"和" I/O 中断"是两回事：时钟中断是定时器周期性产生的（节拍器），I/O 中断是设备有事才产生的。时钟中断每次都来，I/O 中断有事才来。
- "更新时间"和"切换进程"是两回事：时钟中断每次都更新时间、扣时间片；但不是每次都切换进程——只有时间片用完或有更高优先级进程时才调度。
- "内核时钟变量"和"进程时间片"是两回事：前者是系统时间（全局），后者是每个进程剩下的 CPU 时间（每个进程一个）；时钟中断两个都更新。

时钟中断是节拍器：系统时间往前走、当前进程时间片减一、到期睡眠进程叫醒、统计更新。

【坑位】

三个都对，别漏—— I （内核时钟）、 II （占用 CPU 时间）、III（时间片剩余）时钟中断都更新。

最容易踩的坑是：觉得"时间片剩余"是调度程序的事，时钟中断不管。为什么错：时间片就是靠时钟中断来扣的——每响一次，当前进程的时间片减一；减到 0 就触发调度。正确做法：时钟中断 = 和时间有关的所有事——系统时间、进程 CPU 时间、时间片剩余、到期叫醒，全都更新。

【类比】

像学校的上下课铃（时钟中断）：每响一次，教务处（内核）就干几件固定事——校钟往前拨（系统时间）、这节课剩几分钟减一（时间片）、下课睡觉的同学叫醒（到期叫醒）、考勤表更新（统计）。铃一响，这几件事全干，不落下。

【口诀】 时钟中断节拍器：系统时间加一，时间片减一，睡觉到期叫醒。

【考频】 「时钟中断」在 2009–2022 年解析中出现 20 次 · 高频

第 30 题 · 哪种磁盘调度算法不会导致磁臂黏着（正确答案：A）

题干与选项：

系统总是访问磁盘的某个磁道而不响应对其他磁道的访问请求，这种现象称为磁臂黏着。下列磁盘调度算法中，不会导致磁臂粘着的是（）。

- A. 先来先服务（FCFS）
- B. 最短寻道时间优先（SSTF）
- C. 扫描算法（SCAN）
- D. 循环扫描算法（CSCAN）

入手点： 磁臂黏着是“磁臂被某个磁道不断来的请求拽住、长时间不离开”；不会黏着的算法，一定是没有“专挑近的请求”这个机制——先来先服务按到达顺序办，谁先来办谁，不挑。

题干专有名词从零解释：

- 先想“磁盘调度”：一堆读写请求排着队，操作系统决定先服务哪个磁道的请求；不同的决定办法就是不同的调度算法。
- 再想“磁臂”：磁盘里移动的读写头；磁臂移到哪个磁道，就读写哪个磁道的数据；移动要花寻道时间。
- 再想“磁臂黏着”：磁臂长时间停留在某一磁道附近不离开——因为该磁道不断有新请求，算法总选它；其他磁道的请求长期得不到服务。
- 再想“先来先服务（FCFS）”：按请求到达的先后顺序服务，先来的先办，后来的排队；不看磁道远近。
- 再想“最短寻道时间优先（SSTF）”：每次选离当前磁道最近的请求服务；磁道越近越先办。
- 再想“扫描算法（SCAN）”和“循环扫描算法（CSCAN）”：磁臂按一个固定方向移动、服务沿途请求；SCAN 到端反向服务，CSCAN 到端直接跳回起点、不反向服务。

考查知识点：

- 先想黏着是怎么产生的：SSTF 每次选离当前磁道最近的请求；如果某个磁道不断有新请求，磁臂就一直围着它转，远处的请求长期等不到——这就是黏着。
- 再想先来先服务为什么不会黏着：它按到达顺序办，不挑磁道远近；某个磁道就算请求不断，也得排队，不能插队把磁臂拽住。
- 再想扫描和循环扫描：它们按方向推进，服务顺序由方向决定，和先来先服务的“纯按到达顺序”不是一回事；本题答案是先来先服务。
- 边界情况：先来先服务平均寻道时间可能比较长（磁臂来回跑），但“跑得多”和“黏着”是两回事；黏着的关键是“某个磁道长期霸占磁臂”，先来先服务没有这个毛病。

正确选项为什么对： 先来先服务按到达顺序服务，没有“专挑近的请求”的机制，磁臂不会被某个磁道拽住，所以不会导致磁臂黏着，就是 A，选 A。

错误选项为什么错＋举例：

- B（最短寻道时间优先）：每次选最近的请求，某个磁道请求不断来，磁臂就赖在那里——这就是黏着本身。教训：专挑近的，就会黏着。
- C（扫描算法）：按固定方向推进、到端反向服务，服务顺序由方向决定，和先来先服务的“纯按到达顺序”不是一回事，不是本题答案。教训：做题先认准算法名，不要凭“好像也行”选。

- D（循环扫描算法）：到端直接跳回起点、不反向服务，同样是方向推进的机制，不是本题答案。教训：扫描和循环扫描的区别在回卷方式，两个都不是先来先服务。

笔记本一句话： 第 30 题＋磁臂黏着：专挑近的 SSTF 会黏着；先来先服务按到达顺序、不挑，不会黏着。

【图像】



磁臂黏着：SSTF专挑近的会被拽住；FCFS按到达顺序不挑，不会黏着

【定位】 操作系统 · 第4章文件管理 · 磁盘调度

【易混对比】

- "磁臂黏着"和"饥饿"是两回事：黏着是磁臂被某个磁道不断来的请求拽住、长时间不离开；饥饿是某个请求长期得不到服务。SSTF 既可能黏着，也可能让远处的请求饥饿。
- "SSTF"和"FCFS"是两回事：SSTF 专挑最近的请求（会黏着），FCFS 按到达顺序办（不挑，不会黏着）。
- "SCAN"和"CSCAN"是两回事：SCAN 是电梯算法（到端才回头），CSCAN 是单向扫描（回头不服务）；但两者都会被"专挑近的"拽住，都可能黏着。

磁臂黏着 = 被"专挑近的"拽住；FCFS 按到达顺序、不挑近的，不会黏着。

【坑位】

SCAN 和 CSCAN 也会黏着——别以为"扫描"就不黏；只有 FCFS（不挑）不会。

最容易踩的坑是：觉得"SCAN 是来回扫，很公平，不会黏着"。为什么错：黏着的关键是"有没有专挑近的请求的机制"；SCAN 在向一个方向扫时，如果该方向不断有新请求来，磁臂就一直往那个方向走、回不了头——照样黏着。正确做法：只看"挑不挑近的"——SSTF、SCAN、CSCAN 都挑（会黏着），FCFS 按到达顺序不挑（不会）。

【套路】

- 第一步，定标准：会不会黏着，只看有没有"专挑近的请求"的机制。
- 第二步，逐个看：SSTF（专挑最近）会；SCAN（顺路挑近的）会；CSCAN（单向挑近的）会；FCFS（按到达顺序，不挑）不会。

- 第三步，选：选"不会"的那个——FCFS。

黏着看"挑不挑近的"；SSTF、SCAN、CSCAN 都挑，FCFS 不挑。

【类比】

像出租车拉客：SSTF 是"专拉最近的客人"，如果某个地方不断来客人，车就一直围着那转、不去别的地方（黏着）；FCFS 是"按叫车顺序拉"，谁先叫拉谁，不挑，车就不会被拴在一个地方。

【口诀】 磁臂黏着专挑近；SSTF、SCAN 都黏着，FCFS 按顺序不黏着。

【考频】 「磁盘调度」在 2009–2022 年解析中出现 3 次 · 中频

第 31 题 · 提高文件访问速度的优化方法（正确答案：D）

题干与选项：

下列优化方法中，可以提高文件访问速度的是（）。 Ⅰ. 提前读 Ⅱ. 为文件分配连续的簇 Ⅲ. 延迟写 Ⅳ.采用磁盘高速缓存

- A. 仅 Ⅰ、Ⅱ
- B. 仅 Ⅱ、Ⅲ
- C. 仅 Ⅰ、Ⅲ、Ⅳ
- D. Ⅰ、Ⅱ、Ⅲ、Ⅳ

入手点： 文件访问慢，慢就慢在读盘；四种方法一个一个看：只要能减少读盘次数、或者把数据提前放进内存、或者让数据块连在一起少寻道，就是有效的。

题干专有名词从零解释：

- 先想“提前读”：还没用到就先把可能要用的数据从磁盘读进内存；真正用到时，内存里已经有了，不用再读盘。
- 先想“为文件分配连续的簇”：一个文件的数据块在磁盘上连在一起排着；顺序读的时候，磁头不用大范围跳，寻道时间少了。
- 先想“延迟写”：写操作先写进内存缓存、不马上写盘；攒一批之后一次写盘，写盘的次数少了。
- 再想“磁盘高速缓存”：在内存里划一块地方当磁盘的缓存；读过的数据留在缓存里，下次再读直接命中内存，不用读盘。

考查知识点：

- 先想提前读为什么快：读文件时常常是顺序往下读，提前把后面一块读进内存，下次读就不用等磁盘了。
- 再想连续的簇为什么快：数据块连在一起，读完一块，下一块就在旁边，磁头移动的距离短，寻道时间少。
- 再想延迟写为什么快：多次小写操作攒成一次大写操作，写盘次数少了；而且有些临时数据还没来得及写盘就被删了，连写都不用写。
- 再想磁盘高速缓存为什么快：内存比磁盘快得多；读过的数据留在缓存里，重复访问直接命中，不用再读盘。
- 再想结论：四种方法都能减少读盘或者减少寻道，全选，就是 D。
- 边界情况：提前读对随机访问帮助不大（预读的数据可能用不上）；延迟写在掉电时会丢数据（还没写盘的部分就没了），但速度上是快的；题目问的是“能不能提高速度”，不是“有没有代价”。

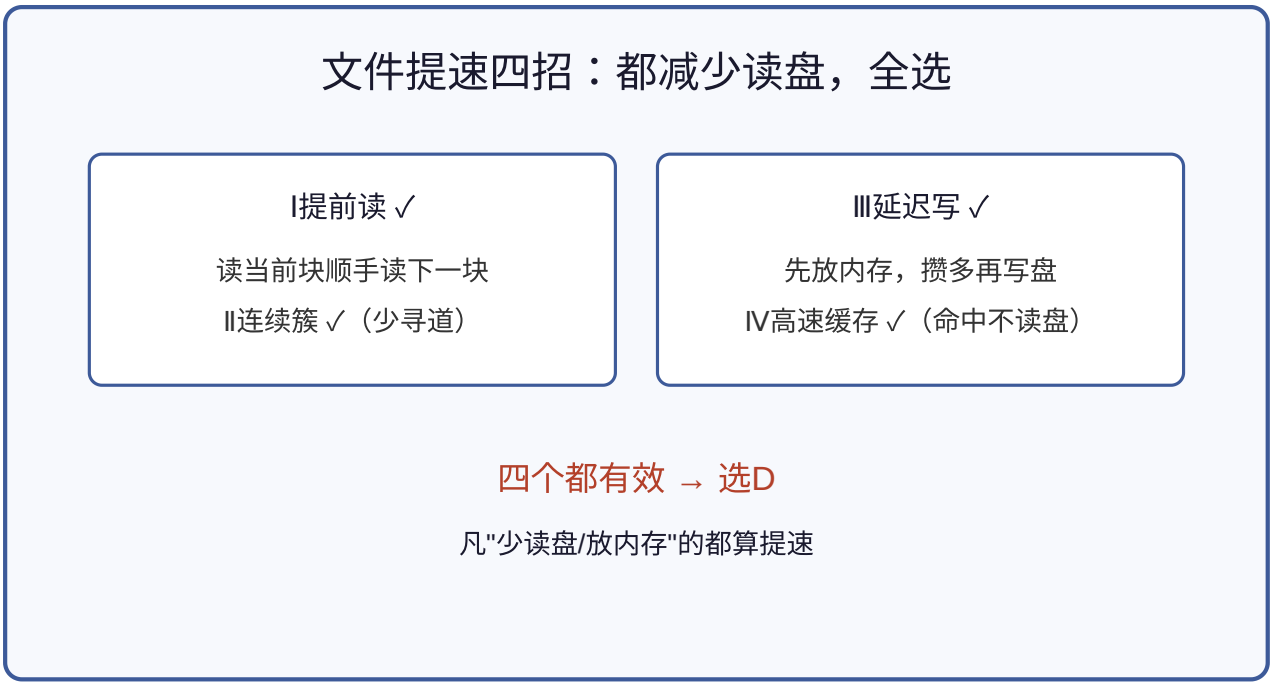
正确选项为什么对： 提前读、连续的簇、延迟写、磁盘高速缓存，四种都能提高文件访问速度，全选，就是 D，选 D。

错误选项为什么错＋举例：

- A（仅 Ⅰ、Ⅱ）：漏了延迟写和磁盘高速缓存；延迟写能合并写盘次数、高速缓存能命中内存，两个都有效。教训：四个方法一个一个过，不要漏。
- B（仅 Ⅱ、Ⅲ）：漏了提前读和磁盘高速缓存；提前读能把后面的数据先备好、高速缓存能留住读过的数据，两个都有效。教训：四个方法一个一个过，不要漏。
- C（仅 Ⅰ、Ⅲ、Ⅳ）：漏了连续的簇；数据块连在一起能少寻道，也是有效的。教训：四个方法一个一个过，不要漏。

笔记本一句话： 第 31 题＋提高文件访问速度：提前读、连续的簇、延迟写、磁盘高速缓存，四个都有效，全选。

【图像】



提高文件访问速度四招：提前读、连续簇、延迟写、磁盘高速缓存——都减少读盘

【定位】 操作系统 · 第4章文件管理 · 文件访问优化

【易混对比】

- "提前读"和"延迟写"是两回事：提前读是"读当前块时顺手把下一块也读进内存"（读优化），延迟写是"写先放内存、攒多了再写盘"（写优化）；两者都减少读盘次数。
- "连续的簇"和"离散的簇"是两回事：连续分配让文件的数据块挨在一起，读的时候磁头不用大范围寻道；离散分配要到处找，慢。
- "磁盘高速缓存"和"内存"是两回事：磁盘高速缓存是内存里专门用来缓存磁盘块的一块区域；命中缓存就不用读盘，速度快得多。

四个都有效：提前读、连续的簇、延迟写、磁盘高速缓存——全选。

【坑位】

四个全对，别漏——提前读、连续簇、延迟写、高速缓存，每个都能提高访问速度。

最容易踩的坑是：觉得"延迟写"是写优化、和"访问速度"无关，就不选Ⅲ。为什么错：延迟写把要写的数据先放内存，攒到一定时候再写盘；如果这段时间又有读请求，直接从内存读，不用读盘——读盘次数少了，访问速度就上去了。正确做法：凡"减少读盘次数"或"把数据放内存"的，都算提高访问速度。

【类比】

像图书馆借书：提前读是"借这本时把下一本也带上"；连续的簇是"要的书都在同一排，不用满馆跑"；延迟写是"还书先放前台，攒多了再上架"；磁盘高速缓存是"热门书放前台，不用进库房"。四招都是少跑路。

【口诀】 文件访问提速四招：提前读、连续簇、延迟写、高速缓存，全都要。

【考频】 「文件访问」在 2009–2022 年解析中出现 1 次 · 低频

第 32 题 · 哪种同步机制可以实现让权等待（正确答案：C）

题干与选项：

在下列同步机制中，可以实现让权等待的是（）。

- A. Peterson 方法
- B. swap 指令
- C. 信号量方法
- D. TestAndSet 指令

入手点： 让权等待就是"等不到资源就阻塞、交出 CPU"，而不是空转；看哪种机制在等的时候会阻塞进程——记录型信号量会，自旋锁不会。

题干专有名词从零解释：

- 先想"同步机制"：多个进程协调着用共享资源、不打架的办法；比如信号量、管程、自旋锁。
- 再想"让权等待"：等不到资源时，进程阻塞（停下来等），并且交出 CPU，让别的进程先跑；等资源有了再被叫醒。
- 再想"忙等"：等不到资源时，进程不交出 CPU，一直循环测试"资源有了没有"；CPU 空转，白白浪费。
- 再想"信号量"：一个计数器加一个等待队列；P 操作（申请）是计数器减 1，不够减就阻塞；V 操作（释放）是计数器加 1，不够减的进程叫醒一个。

考查知识点：

- 先想记录型信号量为什么能让权等待：P 操作时资源不够，进程进入等待队列、变成阻塞状态，CPU 交给别的进程；资源释放时，V 操作从等待队列叫醒一个进程。等的时候不占 CPU，这就是让权等待。
- 再想自旋锁为什么不能：自旋锁拿不到锁就一直循环测试，CPU 一直被它占着，不交出来；这叫忙等，不是让权等待。
- 再想 Peterson 方法为什么不能：Peterson 方法是软件办法，拿不到资源就循环检测标志位，还是忙等，不交出 CPU。
- 再想硬件原子指令（比如测试并设置）为什么不能：它只是保证"测试和设置"一步完成、不被打断；拿不到锁还是要循环测试，还是忙等。
- 边界情况：自旋锁在多 CPU、且等待时间很短时也有用（省了切换开销）；但题目问的是"能实现让权等待"，自旋锁不符合。
- 易混对子：整型信号量和记录型信号量不一样——整型信号量没有等待队列，资源不够时也是忙等；记录型信号量有等待队列，才能让权等待。

正确选项为什么对： 记录型信号量在资源不够时让进程阻塞进等待队列、交出 CPU，就是让权等待，选 C。

错误选项为什么错＋举例：

- A（Peterson 方法）：软件办法，拿不到资源就循环检测标志位，CPU 空转，不交出来；这是忙等，不是让权等待。教训：软件循环检测就是忙等，不让权。
- B（swap 指令）：硬件原子交换指令，拿不到锁就循环重试，CPU 空转，不交出来；这是忙等，不是让权等待。教训：原子指令只保证操作不被打断，不管让权。
- D（测试并设置指令）：它只保证原子操作，拿不到锁还是要循环测试，还是忙等。教训：原子指令不等于让权等待。

笔记本一句话： 第 32 题＋让权等待：记录型信号量资源不够就阻塞进等待队列、交出 CPU；自旋锁是忙等，不让权。

【图像】



让权等待：记录型信号量资源不够就阻塞交出CPU；自旋锁是忙等不让权

【定位】 操作系统 · 第2章进程管理 · 进程同步（信号量、让权等待）

【易混对比】

- "让权等待"和"忙等"是两回事：让权等待是等不到资源就阻塞、交出 CPU（记录型信号量）；忙等是等不到就空转、占着 CPU 不放（自旋锁、TestAndSet）。
- "记录型信号量"和"整型信号量"是两回事：记录型有等待队列，资源不够就阻塞；整型没有等待队列，只能忙等。
- "Peterson 算法"和"信号量"是两回事：Peterson 是软件方法，有限等待但不让权（还是轮流进临界区）；记录型信号量才让权。

让权等待 = 等不到就阻塞、交出 CPU；记录型信号量可以，自旋锁不行。

【坑位】

TestAndSet、swap、自旋锁都是忙等（不让权）；Peterson 有限等待但也不让权——只有记录型信号量让权。

最容易踩的坑是：看到"Peterson 算法满足有限等待"，就以为它也让权。为什么错：有限等待是"迟早能进临界区"，让权等待是"等的时候交出 CPU"；Peterson 是进程轮流检查标志位，等的时候还在空转（占 CPU），不叫让权。正确做法：让权只认"阻塞"——等不到就睡、交出 CPU；凡空转的（自旋、TestAndSet、swap、Peterson）都不让权。

【套路】

- 第一步，定标准：让权等待 = 等不到资源就阻塞、交出 CPU。

- 第二步，逐个看：Peterson（空转，不让权）；swap（自旋，不让权）；记录型信号量（阻塞，让权）；TestAndSet（自旋，不让权）。
- 第三步，选：选会阻塞的那个——记录型信号量。

让权只认阻塞；空转的（自旋、TS、swap、Peterson）都不让权。

【类比】

像排队买票：让权等待是"没票了就回家等，来了再叫你"（阻塞，交出 CPU）；忙等是"没票也在售票口站着，一遍遍问有票没"（占着 CPU 空转）。记录型信号量是前者，自旋锁是后者。

【口诀】 让权等待交出 CPU；记录型信号量阻塞，自旋锁忙等不让权。

【考频】 「让权等待」在 2009–2022 年解析中出现 10 次 · 高频

第 33 题 · TCP／IP 应用层协议中可使用传输层无连接服务的是哪一个（正确答案：B）

题干与选项：

下列 TCP/IP 应用层协议中，可以使用传输层无连接服务的是（）。

- A. FTP
- B. DNS
- C. SMTP
- D. HTTP

入手点： 传输层两个服务：面向连接的 TCP（可靠、开销大）和无连接的 UDP（不可靠、开销小）；应用层协议用哪个，看它要不要可靠——一次一答、丢了重查就行，就用 UDP。

题干专有名词从零解释：

- 先想"应用层协议"：应用程序之间说话的规矩，比如浏览网页、发邮件、查域名，各有各的协议。
- 再想"传输层服务"：应用层下面的服务，分两种——面向连接（先建连接、再传数据、传完拆连接，保证可靠）和无连接（不建连接、直接发，不管到没到）。
- 再想"TCP"：面向连接的服务：传之前先握手建连接，传的时候有确认、有重传，保证数据不错、不丢、不乱序；代价是开销大。
- 再想"UDP"：无连接的服务：不建连接、不确认、不重传，数据可能丢、可能乱序；好处是开销小、速度快。

考查知识点：

- 先想哪些应用层协议用 UDP：域名查询（DNS）、动态主机配置（DHCP）、简单网络管理（SNMP）、简单文件传送（TFTP）、路由信息（RIP）；它们的共同点是：一次一答、数据量小、丢了重查就行。
- 再想域名查询为什么用 UDP：查一个域名就是问一句、答一句；答丢了，查询方超时重查一次就行，不值得为一次查询建连接、握手、拆连接。
- 再想哪些应用层协议必须用 TCP：浏览网页（HTTP）、文件传送（FTP）、发邮件（SMTP）、远程登录（TELNET）；它们的共同点是：数据不能丢、不能错，必须可靠。
- 边界情况：域名查询在区域传送（主辅域名服务器之间同步整个数据库）时用 TCP，因为数据量大、必须可靠；但普通的域名查询用 UDP。题目问的是"可使用无连接服务"，普通查询用 UDP 就符合。

正确选项为什么对： 域名查询一次一答、丢了重查就行，可以用无连接的 UDP 服务，就是 B，选 B。

错误选项为什么错＋举例：

- A（HTTP）：浏览网页要可靠传输页面内容，丢了、错了都不行，必须用 TCP。教训：网页内容不能丢，用 TCP。
- C（FTP）：文件传送不能丢一个字节，必须用 TCP 保证可靠。教训：文件传送要可靠，用 TCP。
- D（SMTP）：邮件内容不能丢、不能错，必须用 TCP 保证可靠。教训：邮件要可靠，用 TCP。

笔记本一句话： 第 33 题＋应用层选传输服务：一次一答、丢了重查的用 UDP（DNS）；不能丢不能错的用 TCP（HTTP、FTP、SMTP）。

【图像】



应用层选传输服务：一次一答丢了重查用UDP(DNS)；不能丢不能错用TCP(HTTP/FTP/SMTP)

【定位】 计算机网络 · 第1章概论/第6章应用层 · 传输层服务选择

【易混对比】

- "UDP"和"TCP"是两回事：UDP 无连接、不可靠、开销小；TCP 面向连接、可靠、开销大。应用层用哪个，看它要不要可靠。
- "DNS"和"HTTP/FTP/SMTP"是两回事：DNS 一次一答、丢了重查就行，用 UDP；HTTP 传网页、FTP 传文件、SMTP 发邮件，都不能丢不能错，用 TCP。
- "无连接服务"和"不可靠"是两回事：无连接不保证可靠，但"不可靠"不等于"不能用"——DNS 丢了包，客户端超时重查就行。

一次一答、丢了重查的用 UDP（DNS）；不能丢不能错的用 TCP（HTTP、FTP、SMTP）。

【坑位】

别把"重要"和"用 TCP"划等号——DNS 也重要，但它一次一答、丢了重查，用 UDP 就够了。

最容易踩的坑是：觉得"DNS 很重要，所以得用可靠的 TCP"。为什么错：选 UDP 还是 TCP，看的是"丢了能不能接受、重查方不方便"；DNS 查询一次一答，丢了客户端超时重发一个查询就行，开销小、速度快。正确做法：看应用的特点——"一次一答、丢了重查"→UDP；"传文件/网页/邮件，不能丢"→TCP。

【类比】

像寄东西：UDP 是平信（便宜、快，丢了再寄一封），TCP 是挂号信（贵、慢，但保证到）。查个电话号码（DNS），平信丢了再寄一封就行；寄合同原件（FTP 传文件），必须用挂号信。

【口诀】 应用层选传输：一次一答用 UDP，传文件网页邮件用 TCP。

【考频】 「UDP」在 2009–2022 年解析中出现 13 次 · 高频

第 34 题 · 不属于物理层接口规范定义范畴的是（正确答案：C）

题干与选项：

下列选项中，不属于物理层接口规范定义范畴的是（）。

- A. 接口形状
- B. 引脚功能
- C. 物理地址
- D. 信号电平

入手点： 本题问"不属于"的是哪一个：先把物理层接口规范的四个特性摆出来——机械、电气、功能、规程；不在这四个特性里的，就是更高层管的事，就是答案。

题干专有名词从零解释：

- 先想"物理层"：OSI 参考模型的最底层，管的是"怎么在物理媒体上传比特流"——电压多高、接头什么样、信号怎么时序。
- 再想"接口规范"：设备和传输媒体之间接头的规矩；两台设备要连起来，得在接头上达成一致。
- 再想四个特性：机械特性（接头形状、尺寸、针脚数）、电气特性（电压范围、信号电平）、功能特性（每根线是干什么的）、规程特性（各线工作的时序、顺序）。
- 再想"不属于"：这项内容不是物理层接口规范管的，是更高层（数据链路层、网络层或者更高层）管的。

考查知识点：

- 先想机械特性：接头的形状、尺寸、针脚排列——比如插头是方的还是圆的、有几个针脚。
- 再想电气特性：信号电压的范围、传输速率、阻抗——比如高电平是多少伏、低电平是多少伏。
- 再想功能特性：每根线（引脚）的意义——哪根是发数据、哪根是收数据、哪根是地线。
- 再想规程特性：各线工作的顺序和时序——比如先发请求信号、再发数据信号，时序不能乱。
- 再想不属于的：成帧（把比特流分成帧）、差错控制、流量控制、路由选择、数据表示转换——这些都是数据链路层、网络层或者表示层管的事，不在物理层接口规范的四个特性里。
- 边界情况：传输速率是电气特性的一部分（信号传多快），属于物理层；但"选择哪条路传"（路由）是网络层的事，不属于物理层。

正确选项为什么对： C（物理地址）是数据链路层 MAC 地址的范畴，不在机械、电气、功能、规程四个特性里，所以不属于物理层接口规范的范畴，选 C。

错误选项为什么错＋举例：

- A：说的是接头形状尺寸——这是机械特性，属于物理层接口规范，不是答案。
- B：说的是电压范围——这是电气特性，属于物理层接口规范，不是答案。
- D：说的是某根线的意义或者工作时序——这是功能特性或者规程特性，属于物理层接口规范，不是答案。

笔记本一句话： 第 34 题＋物理层接口规范四个特性：机械（形状）、电气（电压）、功能（线义）、规程（时序）；更高层的功能不属于。

【图像】



物理层接口四特性：机械(形状)、电气(电压)、功能(线义)、规程(时序)；物理地址是链路层的不属于

【定位】 计算机网络 · 第2章物理层 · 接口特性

【易混对比】

- "物理层"和"数据链路层"是两回事：物理层管"线"（形状、电压、线干什么用、时序）；数据链路层管"帧"（地址、差错控制）。物理地址（MAC 地址）是链路层的，不属于物理层。
- "四个特性"和"随便列的"是两回事：物理层接口规范就四个——机械、电气、功能、规程；不在这四个里的，就是别层的。
- "接口形状"和"物理地址"是两回事：接口形状（机械特性）是插头长什么样；物理地址（MAC）是网卡的编号，链路层用。

物理层四个特性：机械（形状）、电气（电压）、功能（线义）、规程（时序）；物理地址是链路层的，不属于。

【坑位】

"物理地址"听起来像物理层的，但它是数据链路层的（MAC 地址）——名字有"物理"不等于归物理层。

最容易踩的坑是：看到"物理地址"里有"物理"两个字，就以为归物理层。为什么错：物理地址（MAC 地址）是网卡的硬件地址，用在数据链路层的帧头里，决定帧发给谁；物理层只管"线"（比特流怎么传），不管"地址"。正确做法：看到"地址"，先想链路层（MAC）或网络层（IP）；物理层没有地址的概念。

【套路】

- 第一步，背四个：机械（接口形状）、电气（信号电平）、功能（引脚功能）、规程（时序）。
- 第二步，逐个对：接口形状→机械，对；引脚功能→功能，对；信号电平→电气，对；物理地址→四个里没有，错。
- 第三步，定归属：物理地址（MAC）是数据链路层的，所以"不属于"物理层，选它。

四特性机械电气功能规程；有"地址"先想链路层/网络层，物理层没有地址。

【类比】

像修一条路：物理层是"路本身"——多宽（机械）、限速多少（电气）、车道干什么用（功能）、红绿灯时序（规程）；"车牌号"（物理地址）是交警（链路层）管的，不归修路的管。名字里有"物理"，不等于归修路的。

【口诀】 物理层四特性：机械电气功能规程；物理地址归链路层。

【考频】 「物理层」在 2009–2022 年解析中出现 38 次 · 高频

第 35 题 · IEEE 802.11 CSMA／CA 信道预约方法（正确答案：D）

题干与选项：

IEEE 802.11 无线局域网的 MAC 协议 CSMA/CA 进行信道预约的方法是（）。

- A. 发送确认帧
- B. 采用二进制指数退避
- C. 使用多个 MAC 地址
- D. 交换 RTS 与 CTS 帧

入手点： 无线很难边发边听、检测碰撞，所以 802.11 用"预约"代替"检测"：发送方先发请求、接收方回答应，周围站点听到请求或者回答就静默一段时间——这就是 RTS／CTS 加虚拟载波监听。

题干专有名词从零解释：

- 先想"IEEE 802.11"：无线局域网的标准（平时说的 Wi-Fi）；用无线电波传数据，没有网线。
- 再想"CSMA／CA"：带碰撞避免的载波监听多路访问——发送之前先听信道空不空，空了再发；发之前还要想办法避免碰撞。
- 再想"信道预约"：发送方提前告诉周围站点"我要用信道一段时间"，周围站点这段时间不发，避免撞车。
- 再想"隐藏终端"：两个站点互相听不到对方，但都能听到中间的接收方；它们同时发，接收方那里就撞了——这是无线特有的问题。

考查知识点：

- 先想为什么不用碰撞检测：无线信号发出去之后，自己的发送信号太强，听不到别人的信号，没法边发边检测碰撞；所以 802.11 不用 CSMA／CD（碰撞检测），改用 CSMA／CA（碰撞避免）。
- 再想 RTS／CTS 怎么预约：发送方先发 RTS（请求发送，里面写着"我要用信道多久"），接收方回 CTS（清除发送，也写着时长）；听到 RTS 或者 CTS 的站点，就知道这段时间信道被预约了，自己不发。
- 再想虚拟载波监听（NAV）：每个站点维护一个"网络分配向量"——听到 RTS／CTS 后，把预约时长记下来，这段时间里即使物理上听着信道是空的，也当它是忙的（虚拟的忙）；这就是虚拟载波监听。
- 再想物理载波监听：站点平时也要听信道——物理上听到有信号，就是忙；物理监听和虚拟监听两个都空闲，才能发。
- 再想隐藏终端怎么解决：两个互相听不到的站点 A 和 C，都要发给 B；A 先发 RTS，B 回 CTS，C 听到了 CTS（C 听不到 A，但听得到 B），C 就知道信道被预约了，不发——碰撞避免了。
- 边界情况：RTS／CTS 只对长帧用（短帧直接发，预约的开销不划算）；预约时长由 RTS／CTS 里的时长字段告诉周围站点。

正确选项为什么对： 802.11 的信道预约方法是 RTS／CTS 加虚拟载波监听（NAV），就是 D，选 D。

错误选项为什么错＋举例：

- A（CSMA／CD）：那是有线以太网的碰撞检测；无线边发边听做不到，802.11 不用它。教训：有线用检测，无线用避免。
- B（只用物理载波监听）：只听物理信道，解决不了隐藏终端问题——互相听不到的两个站点还是会撞。教训：物理监听不够，还要虚拟监听。
- C（令牌传递）：令牌环网的办法（轮流拿令牌才能发），不是 802.11 的办法。教训：802.11 是预约，不是令牌。

笔记本一句话： 第 35 题＋802.11 信道预约：RTS／CTS 加虚拟载波监听（NAV），解决隐藏终端问题。

【图像】



CSMA/CA信道预约：RTS/CTS握手+虚拟载波监听(NAV)；解决隐藏终端，无线不用碰撞检测

【定位】 计算机网络 · 第3章数据链路层 · 无线局域网 CSMA/CA

【易混对比】

- "CSMA/CA"和"CSMA/CD"是两回事：CA（避免碰撞）是无线的，用预约（RTS/CTS）；CD（检测碰撞）是有线的，用边发边听。无线很难边发边听，所以不用 CD。
- "RTS/CTS"和"ACK"是两回事：RTS/CTS 是发数据之前的信道预约（"我要发了，大家别出声"）；ACK 是收到数据之后的确认（"收到了"）。预约用前者。
- "二进制指数退避"和"信道预约"是两回事：退避是碰撞后的重发等待策略（CD 里用）；预约是发之前的握手（CA 里用）。题目问预约，答 RTS/CTS。

CSMA/CA 信道预约 = RTS/CTS 握手 + 虚拟载波监听（NAV）；解决隐藏终端。

【坑位】

"二进制指数退避"是 CSMA/CD 的碰撞处理，不是 CSMA/CA 的信道预约——问预约只认 RTS/CTS。

最容易踩的坑是：把"二进制指数退避"当成预约方法。为什么错：退避是"撞了之后等多久再发"，预约是"发之前先打招呼占地方"；一个管撞后，一个管发前。正确做法：看到"预约"，想"打招呼"——RTS（请求发送）、CTS（允许发送），周围站点听到就静默（NAV）。

【类比】

像开会发言：无线开会（听不清别人有没有在说）不能靠"边说边听"，得先举手（RTS 说"我要发言"），主持人点头（CTS 说"大家安静"），再发言。举手+点头就是预约；说完有人回"听到了"（ACK）是确认，不是预约。

【口诀】 无线预约不用检测撞；RTS/CTS 先握手，虚拟监听 NAV 静默。

【考频】 「CSMA/CA」在 2009–2022 年解析中出现 4 次 · 中频

第 36 题 · 停-等协议、传输速率／单向传播时延／信道利用率求数据帧长度（正确答案：D）

题干与选项：

主机甲采用停-等协议向主机乙发送数据，数据传输速率是 3 kbps，单向传播延时是 200 ms，忽略确认帧的传输延时。当信道利用率等于 40%时，数据帧的长度为（ ）。

- A. 240 比特
- B. 400 比特
- C. 480 比特
- D. 800 比特

入手点： 停-等协议一个周期是"发一帧、等确认回来"；信道利用率就是"发帧时间占整个周期的比例"；把公式列出来，解出帧长就行。

题干专有名词从零解释：

- 先想"停-等协议"：发送方发一帧就停下来，等接收方回确认，收到确认再发下一帧；一次只发一帧，不连续发。
- 再想"数据传输速率"：发送方往信道里放数据的速度（比如每秒多少比特）；帧长除以速率，就是发一帧要花的时间。
- 再想"单向传播时延"：信号从发送方走到接收方要花的时间（单程）；确认回来还要再走一程，所以一个周期里传播时延要算两次。
- 再想"信道利用率"：真正用来发数据的时间占整个周期的比例；等确认的时间里信道是空闲的，利用率就不到 1。

考查知识点：

- 先想一个周期是多长：发一帧的时间是帧长除以速率；发完之后，信号走到接收方要单向传播时延，确认再走回来又要单向传播时延；确认帧很短，发送时间忽略不计；所以一个周期等于帧发送时间加上两倍单向传播时延。
- 再想利用率公式：信道利用率等于帧发送时间除以整个周期，也就是（帧长除以速率）除以（帧长除以速率加上两倍单向传播时延）。
- 再想解出帧长：把公式变形——帧长等于利用率乘速率乘两倍单向传播时延，再除以（1 减利用率）。代入题中给的速率、单向传播时延和利用率，算出来的帧长就是 D。
- 再想题中的数：速率 3kbps，单向传播时延 200ms，利用率 0.4；帧长等于 0.4×3000×（2×0.2）÷（1−0.4）= 480÷0.6 = 800 比特，就是 D。
- 拿小数字验算：速率是 10Mbps，单向传播时延是 10 微秒，利用率是 0.5；帧长等于 0.5×10Mbps×20 微秒÷0.5，等于 200 比特。利用率越高，帧长越长；传播时延越大，同样利用率需要的帧长也越长。
- 边界情况：利用率接近 1 时，分母接近 0，帧长会变得很大——停-等协议里利用率到不了 1，因为总要等确认；确认帧的发送时间题目通常忽略不计。

正确选项为什么对： 按公式解出帧长 800 比特，就是 D，选 D。

错误选项为什么错＋举例：

- A（240 比特）：只算了单向传播时延还没除（1−利用率），0.4×3000×0.2=240；一个周期里传播要走两个单程，还要除以（1−利用率）。教训：往返是两倍单向时延，公式里的（1−利用率）不能丢。

- B（400 比特）：只算了单向传播时延（没乘 2），0.4×3000×0.2÷0.6=400；一个周期里传播要走两个单程。教训：往返是两倍单向时延。
- C（480 比特）：乘了两倍单向时延但没除（1−利用率），0.4×3000×0.4=480；利用率公式里的分母（1−利用率）不能丢。教训：解帧长要除以（1−利用率）。

笔记本一句话： 第 36 题＋停-等协议求帧长：利用率等于发送时间除以（发送时间加两倍单向传播时延），解出帧长。

【图像】



停-等协议信道利用率=发送时间/(发送时间+2×单向传播时延)；解出帧长

【定位】 计算机网络 · 第3章数据链路层 · 流量控制（停-等协议、信道利用率）

【易混对比】

- "发送时延"和"传播时延"是两回事：发送时延 = 帧长 ÷ 速率（把帧推出去要多久），传播时延 = 距离 ÷ 光速（信号在路跑多久）。利用率公式里两个都要。
- "单向传播时延"和"往返"是两回事：停-等一个周期要"发过去 + 确认回来"，传播时延要算两倍（去一趟、回一趟）。
- "忽略确认帧的传输延时"和"忽略传播时延"是两回事：确认帧很短，发送它花的时间可忽略；但确认帧在路上跑的时间（传播时延）不能忽略。

利用率 = 发送时间 ÷（发送时间 + 2×单向传播时延）；列方程解出帧长。

【坑位】

传播时延要乘 2（一去一回）；"忽略确认帧传输延时"不等于忽略它的传播时延。

最容易踩的坑是：分母只加了一倍传播时延（忘了确认回来也要跑）。为什么错：停-等协议一个周期是"发一帧→等确认回来"；确认帧回来也要跑一趟，传播时延一去一回共两倍。正确做法：周期 = 发送时延 + 2×单向传播时延（确认帧发送时延忽略，传播时延保留）。

【套路】

- 第一步，设未知：设帧长为 L 比特。
- 第二步，写发送时延：L ÷ 速率（3kbps）。
- 第三步，写周期：发送时延 + 2×单向传播时延（200ms×2，注意单位统一）。
- 第四步，列方程：利用率 = 发送时延 ÷ 周期 = 40%，解出 L。

设帧长，算发送时延；周期加两倍传播；利用率列方程解。

【类比】

像寄快递：利用率 = "打包时间"占"打包+路上来回"的比例。打包越快（帧短），路上时间占比越大，利用率越低；要利用率 40%，就得算打包（发送）要多久——列个方程解出来。

【口诀】 停-等求帧长：利用率等于发送除以周期；周期加两倍传播时延。

【考频】 「停-等协议」在 2009–2022 年解析中出现 4 次 · 中频

第 37 题 · 跨路由器交换机两主机通信、以太网帧首尾源与目的 MAC 地址变化（正确答案：D）

题干与选项：

路由器 R 通过以太网交换机 S1 和 S2 连接两个网络，R 的接口、主机 H1 和 H2 的 IP 地址与 MAC 地址如下图所示。若 H1 向 H2 发送 1 个 IP 分组 P，则 H1 发出的封装 P 的以太网帧的目的 MAC 地址、H2 收到的封装 P 的以太网帧的源 MAC 地址分别是（）。

- A. 00-a1-b2-c3-d4-62、00-1a-2b-3c-4d-52
- B. 00-a1-b2-c3-d4-62、00-a1-b2-c3-d4-61
- C. 00-1a-2b-3c-4d-51、00-1a-2b-3c-4d-52
- D. 00-1a-2b-3c-4d-51、00-a1-b2-c3-d4-61

> 注：本题是图题，路由器 R 经交换机 S1、S2 连接两个网络；图中地址按给出的可见信息写：R 左侧接口 192.168.3.1、MAC 00-1a-2b-3c-4d-51，右侧接口 192.168.4.1、MAC 00-a1-b2-c3-d4-61；H1 为 192.168.3.2、MAC 00-1a-2b-3c-4d-52；H2 为 192.168.4.2、MAC 00-a1-b2-c3-d4-62。

入手点： MAC 地址是链路本地的——每过一个路由器就换一次；IP 地址是端到端的——全程不变；交换机不改 MAC 地址，路由器转发时重新封装。

题干专有名词从零解释：

- 先想"MAC 地址"：网卡的硬件地址（比如 00-1a-2b-3c-4d-52），写在以太网帧的帧头里；MAC 地址只在同一个局域网里有效，出了局域网就换。
- 再想"IP 地址"：主机的网络层地址（比如 192.168.3.2），全程不变；从源主机到目的主机，IP 地址一直是源 IP 和目的 IP。
- 再想"路由器"：连接两个网络的设备；收到帧之后拆开帧头、看目的 IP 查路由表，再重新封装成新的帧、从另一个接口发出去。
- 再想"交换机"：同一个局域网里的转发设备；按 MAC 地址表把帧从一个口转到另一个口，不改帧头里的 MAC 地址。

考查知识点：

- 先想第一段（H1 到路由器）：H1 发帧，源 MAC 是 H1 的 MAC（00-1a-2b-3c-4d-52），目的 MAC 是路由器左侧接口的 MAC（00-1a-2b-3c-4d-51）；帧经交换机 S1 到达路由器左接口，交换机不改 MAC 地址。
- 再想路由器干什么：路由器拆开帧头，看目的 IP（192.168.4.2）查路由表，知道要从右侧接口发出去；于是重新封装：源 MAC 改成右侧接口的 MAC（00-a1-b2-c3-d4-61），目的 MAC 改成 H2 的 MAC（00-a1-b2-c3-d4-62）。
- 再想第二段（路由器到 H2）：帧经交换机 S2 到达 H2，源 MAC 是路由器右侧接口的 MAC，目的 MAC 是 H2 的 MAC；交换机同样不改 MAC 地址。
- 再想 IP 地址：全程不变——源 IP 一直是 H1 的 192.168.3.2，目的 IP 一直是 H2 的 192.168.4.2；变的是 MAC 地址，不是 IP 地址。
- 拿小例子验算：H1 给同网段的 H3 发数据（不经过路由器），MAC 地址全程不变（源是 H1、目的是 H3）；一旦跨过路由器，MAC 地址就要换一次，IP 地址还是不变。

- 边界情况：路由器有多个接口时，源 MAC 用的是"转发出去的那个接口"的 MAC，不是"收进来的那个接口"的；目的 MAC 是下一跳的 MAC（这里下一跳就是目的主机 H2）。

正确选项为什么对： 第一段源 MAC 是 H1、目的 MAC 是路由器左接口；第二段源 MAC 是路由器右接口、目的 MAC 是 H2；IP 地址全程不变，就是 D，选 D。

错误选项为什么错＋举例：

- A（MAC 地址全程不变）：MAC 地址是链路本地的，跨过路由器就要换；全程不变的是 IP 地址，不是 MAC 地址。教训：MAC 逐段换，IP 全程不变。
- B（IP 地址也变了）：IP 地址是端到端的，从源主机到目的主机一直不变；变的是 MAC 地址。教训：IP 不变，MAC 变。
- C（源目的 MAC 配错段）：把第一段的目的 MAC 写成 H2 的、第二段的源 MAC 写成 H1 的——第一段的目的地是路由器左接口，不是 H2；第二段的源头是路由器右接口，不是 H1。教训：每段的源和目的看这段的两个端点。

笔记本一句话： 第 37 题＋跨路由器通信：MAC 地址逐段换（每段看两个端点），IP 地址全程不变；交换机不改 MAC，路由器重新封装。

【图像】



跨路由器MAC逐段换、IP全程不变；交换机不改MAC，路由器重新封装

【定位】 计算机网络 · 第4章网络层/第3章数据链路层 · MAC 与 IP 地址变化

【易混对比】

- "MAC 地址"和"IP 地址"是两回事：MAC 是链路本地的，每过一个路由器就换一次；IP 是端到端的，全程不变。
- "交换机"和"路由器"是两回事：交换机不改 MAC 地址（只按 MAC 转发）；路由器转发时要重新封装，源/目的 MAC 换成新一段的两个端点。

- "目的 MAC"和"目的 IP"是两回事：H1 发出的帧，目的 IP 是 H2（全程不变），但目的 MAC 是下一跳（R 的左接口），不是 H2。

MAC 逐段换（每段看两个端点），IP 全程不变；交换机不改 MAC，路由器重新封装。

【坑位】

H1 发出的帧，目的 MAC 不是 H2——是下一跳路由器接口；源 MAC 才是 H1 自己。

最容易踩的坑是：目的 MAC 写成 H2 的 MAC（把 IP 的"端到端"套到 MAC 上）。为什么错：MAC 地址只在一段链路内有效；H1 和 H2 不在同一段链路（中间隔着路由器），H1 的帧只能发到下一跳（R 的左接口），目的 MAC 就是 R 左接口的。正确做法：一段一段看——每段的源/目的 MAC 就是这段的两个端点；IP 地址全程都是 H1→H2。

【套路】

- 第一步，分段：H1→R（左网段），R→H2（右网段），共两段。
- 第二步，定每段端点：第一段端点是 H1 和 R 左接口；第二段端点是 R 右接口和 H2。
- 第三步，写 MAC：H1 发出的帧：源=H1，目的=R 左接口；H2 收到的帧：源=R 右接口，目的=H2。
- 第四步，IP 不变：IP 分组源/目的全程是 H1→H2，不用管。

分段看端点；MAC 每段换，IP 全程不变。

【类比】

像跨国寄快递：快递单号（IP）全程不变；但每到一个国家，要换当地快递公司重新打包（路由器重新封装），面单上的"发货仓/收货仓"（MAC）就换成当地的。国内转运（交换机）不换面单，只管送。

【口诀】 跨路由器 MAC 逐段换，IP 全程不变；交换机不改，路由器重封装。

【考频】 「MAC 地址」在 2009–2022 年解析中出现 33 次 · 高频

第 38 题 · 相同下一跳四条／21 路由聚合后目的网络地址（正确答案：C）

题干与选项：

某路由表中有转发接口相同的 4 条路由表项，其目的网络地址分别为 35.230.32.0/21、35.230.40.0/21、35.230.48.0/21 和 35.230.56.0/21，将该 4 条路由聚合后的目的网络地址为（）。

- A. 35.230.0.0/19
- B. 35.230.0.0/20
- C. 35.230.32.0/19
- D. 35.230.32.0/20

入手点： 路由聚合就是找最长公共前缀：四条／21 地址连续、下一跳相同，就能合成一条；四条覆盖了第 22、23 位的四种组合，公共前缀就短 2 位，变成／19。

题干专有名词从零解释：

- 先想"路由"：路由器里的一条记录——去某个目的网络，下一跳走哪个接口；目的网络用"网络地址／前缀长度"表示。
- 再想"／21"：前缀长度是 21，意思是前 21 位是网络号、后 11 位是主机号；一条／21 路由覆盖 2048 个地址。
- 再想"下一跳"：去某个目的网络，下一步要发给哪个路由器（或者哪个接口）；下一跳相同，说明这几条路由的去向一致。
- 再想"路由聚合"：把几条地址连续、下一跳相同的路由合成一条更短的路由（前缀更短、覆盖范围更大），路由表里的表项就变少了。

考查知识点：

- 先想聚合的条件：地址要连续、对齐，下一跳要相同；缺一个条件都不能聚合。
- 再想公共前缀怎么找：四条／21 的地址，前 21 位里，前 19 位相同，第 20、21 位（也就是第 22、23 位，从 1 数）分别是 00、01、10、11 四种组合；公共前缀就是前 19 位，聚合后是／19。
- 再想目的网络地址怎么写：公共前缀后面补 0，就是聚合后的目的网络地址。
- 再想题中的数：四条是 35.230.32.0／21、35.230.40.0／21、35.230.48.0／21、35.230.56.0／21；第三字节分别是 32（00100000）、40（00101000）、48（00110000）、56（00111000），公共前缀是前 19 位（前两个字节加第三字节前 3 位 001），聚合后是 35.230.32.0／19，就是 C。
- 拿小例子验算：192.168.0.0／21、192.168.8.0／21、192.168.16.0／21、192.168.24.0／21，第三字节分别是 0（00000000）、8（00001000）、16（00010000）、24（00011000）；前 19 位公共（前两个字节加第三字节前 3 位 000），聚合后是 192.168.0.0／19，覆盖 8192 个地址，正好是四条／21 之和。
- 边界情况：四条地址不连续（比如中间缺了一条），就不能聚合成一条／19；下一跳不同也不能聚合（聚合后下一跳就说不清了）；聚合后的前缀一定比原来短，不能比原来长。

正确选项为什么对： 四条／21 地址连续、下一跳相同，公共前缀是前 19 位，聚合后是 35.230.32.0／19，就是 C，选 C。

错误选项为什么错＋举例：

- A（35.230.0.0／19）：前缀长度对了（／19），但网络地址错了（0.0 不是 32.0）；公共前缀后面补 0，第三字节前 3 位是 001，不是 000。教训：聚合后的网络地址要按公共前缀写，不能全置 0。

- B（35.230.0.0／20）：前缀和网络地址都错了；四条／21 合一条，前缀短 2 位是／19，不是／20，网络地址也不是 0.0。教训：四条合一条，前缀短 2 位，地址按公共前缀写。
- D（35.230.32.0／20）：网络地址对了（32.0），但前缀错了（／20 不是／19）；四条／21 合一条，前缀短 2 位。教训：前缀长度要算对，少短一位就漏了一半地址。

笔记本一句话： 第 38 题＋路由聚合：地址连续、下一跳相同，找最长公共前缀；四条／21 合一条／19。

【图像】



路由聚合找最长公共前缀：四条/21第三段取前3位相同，聚合成/19

【定位】 计算机网络 · 第4章网络层 · 路由聚合（最长公共前缀）

【易混对比】

- "聚合"和"分别列出"是两回事：聚合是把多条前缀合成一条（下一跳相同才能合），转发表变小；分别列出是每条都写，转发表大。
- "最长公共前缀"和"最短"是两回事：聚合取的是"最长的公共前缀"（能区分的最多位）；不是越短越好，太短会把不相关的网段也包进来。
- "/21 变 /19"和"变 /20"是两回事：四条 /21 的第三段分别是 32、40、48、56，前 3 位相同（001），第 4 位有 0 有 1，所以公共前缀是 16+3=19 位；不是 20 位。

路由聚合找最长公共前缀；四条连续的 /21，第三段前 3 位相同，聚合成 /19。

【坑位】

别数成 /20——四条地址第三段第 4 位有 0 有 1，公共前缀到第 3 位为止，是 /19。

最容易踩的坑是：看到"四条 /21 合一条"，就以为前缀减 2 位得 /19——这次蒙对了，但换题就错；或者看成 /20。为什么错：聚合的关键是"最长公共前缀"，要一位一位比；四条地址第三段（32=00100000、40=00101000、48=00110000、56=00111000）前 3 位都是 001，第 4 位有 0 有 1，所以只能取到前 3 位。正确做法：把不同的那段写成二进制，从左往右数相同的位数。

【套路】

- 第一步，定不同的段：前 16 位都相同，看第三段（32、40、48、56）。
- 第二步，写二进制：32=00100000，40=00101000，48=00110000，56=00111000。
- 第三步，数相同位：从左往右，前 3 位都是 001，第 4 位有 0 有 1→取 3 位。
- 第四步，拼前缀：16+3=19，网络地址是 35.230.32.0/19（后面位清零）。

找不同的段，写二进制，数相同位，拼前缀清零。

【类比】

像给四个门牌号找公共地址：32 号、40 号、48 号、56 号都在"第三街"，门牌号二进制前 3 位一样，所以公共地址是"第三街前 3 位相同的那片"，后面的位数清零——这就是聚合。

【口诀】 路由聚合找公共前缀；不同段写二进制，数相同位拼前缀。

【考频】 「路由聚合」在 2009–2022 年解析中出现 5 次 · 中频

第 39 题 · UDP 分用依赖的头部字段（正确答案：B）

题干与选项：

UDP 协议实现分用(demultiplexing)时所依据的头部字段是（ ）。

- A. 源端口号
- B. 目的端口号
- C. 长度
- D. 校验和

入手点： 分用就是"收到报文、交给谁"；UDP 交给谁就看目的端口号——报文要找监听这个端口的套接字，端口对上了就交给它。

题干专有名词从零解释：

- 先想"分用"：接收方收到报文后，决定把它交给哪个进程（哪个套接字）的过程；和"复用"（发送方把多个进程的报文合起来发）是一对。
- 再想"端口号"：进程在传输层的门牌号；一个进程要收报文，就得绑定一个端口号（比如域名服务用 53 号端口）。
- 再想"UDP 头部字段"：UDP 报文前面的一小段控制信息，包括源端口号、目的端口号、长度、校验和。
- 再想"套接字"：应用程序收发数据的接口；一个套接字由 IP 地址加端口号标识。

考查知识点：

- 先想 UDP 分用的依据：目的端口号。报文到达主机后，UDP 看目的端口号，找到监听这个端口的套接字，把报文交给它。
- 再想源端口号是干什么的：对方回包时用的——回包的目的端口就是原来报文的源端口；分用时不用它。
- 再想 IP 地址字段是干什么的：源 IP 地址和目的 IP 地址是 IP 层用来找主机的；IP 层分用靠协议字段（比如协议号 17 是 UDP、6 是 TCP）决定交给 UDP 还是 TCP；到了 UDP 层，分用靠目的端口号。
- 再想校验和是干什么的：检错用的——算一遍校验和，对不上就说明报文传错了、丢弃；不是用来分用的。
- 拿小例子验算：主机上域名服务监听 53 号端口、视频服务监听 5004 号端口；收到目的端口是 53 的 UDP 报文，就交给域名服务；收到目的端口是 5004 的，就交给视频服务。
- 边界情况：UDP 分用只看目的端口号，不看源端口号和源 IP 地址（和 TCP 不一样，TCP 分用要看源 IP、源端口、目的 IP、目的端口四元组）；目的端口没有进程监听，报文就丢弃（有的实现回一个不可达差错）。

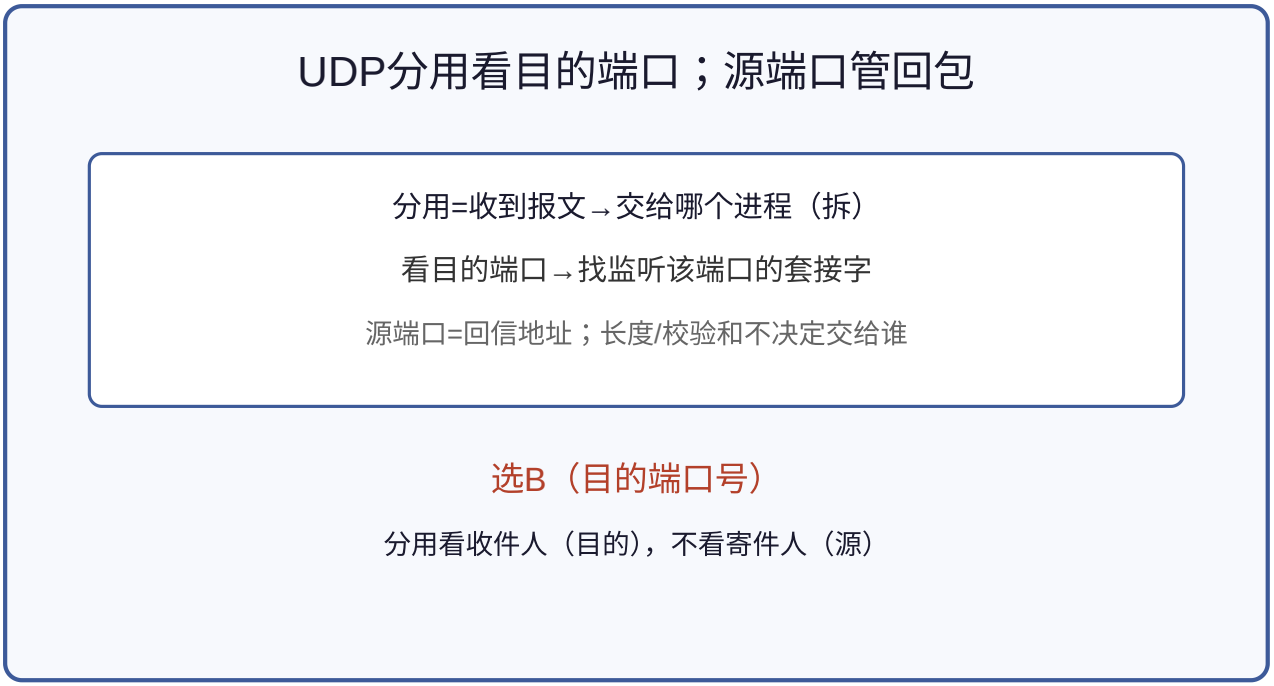
正确选项为什么对： UDP 分用靠目的端口号找到对应的套接字，就是 B，选 B。

错误选项为什么错＋举例：

- A（源端口号）：源端口是对方回包时用的，不是分用的依据；分用看目的端口。教训：分用看目的，回包看源。
- C（源 IP 地址）：源 IP 地址是找主机的，不是找进程的；到了 UDP 层，找进程靠目的端口号。教训：IP 找主机，端口找进程。
- D（校验和）：校验和是检错的，算对了才能收、算错了丢弃；它不决定交给哪个进程。教训：校验和管对错，不管分用。

笔记本一句话： 第 39 题＋UDP 分用：看目的端口号找套接字；源端口是回包用的，校验和是检错的。

【图像】



UDP分用看目的端口号找套接字；源端口是回包用的，长度校验和不决定交给谁

【定位】 计算机网络 · 第5章传输层 · UDP（分用）

【易混对比】

- "分用"和"复用"是两回事：分用是"收到报文、交给哪个进程"（拆），复用是"多个进程的报文、从一个口发出去"（合）。题目问分用，看目的端口。
- "目的端口"和"源端口"是两回事：分用看目的端口（找监听这个端口的套接字）；源端口是对方回包时用的，不决定这次交给谁。
- "端口号"和"IP 地址"是两回事：IP 地址决定到哪台主机（网络层），端口号决定到哪个进程（传输层）；分用是传输层的事，看端口。

UDP 分用看目的端口号：找监听这个端口的套接字，交给他。

【坑位】

别选源端口号——源端口是"对方回包时用"，分用（这次交给谁）看目的端口。

最容易踩的坑是：觉得"源端口也标识进程，分用也该看它"。为什么错：分用是接收方决定"这个报文给谁"；接收方只认"发给我的端口"（目的端口），源端口是发送方留的"回信地址"，这次用不上。正确做法：分用=拆快递——看收件人（目的端口），不看寄件人（源端口）。

【类比】

像公司前台分信：每封信（UDP 报文）看收件人（目的端口号），送到对应部门（进程/套接字）；寄件人（源端口）是回信时用的，分信时不看。长度（信多厚）、校验和（有没有破损）都不决定送给谁。

【口诀】 UDP 分用看目的端口；源端口管回包，长度校验不管分。

【考频】 「分用」在 2009–2022 年解析中出现 7 次 · 中频

第 40 题 · SMTP 无需转换即可直接传输的内容类型（正确答案：D）

题干与选项：

无需转换即可由 SMTP 协议直接传输的内容是（）。

- A. JPEG 图像
- B. MPEG 视频
- C. EXE 文件
- D. ASCII 文本

入手点： SMTP 是老协议，只认 7 位 ASCII 文本；凡是 7 位 ASCII 之外的东西（中文、图片、音频、二进制文件），都得先编码转成 7 位 ASCII 才能传。

题干专有名词从零解释：

- 先想"SMTP"：简单邮件传送协议——发邮件时，邮件服务器之间传邮件用的协议。
- 再想"7 位 ASCII 文本"：每个字节最高位是 0 的纯英文文本（英文字母、数字、常见标点）；这是 SMTP 唯一能直接传的内容。
- 再想"MIME"：多用途邮件扩展——给 SMTP 打的补丁，让邮件能带非 ASCII 内容；办法是把非 ASCII 内容编码成 7 位 ASCII 再传（比如 base64 编码、quoted-printable 编码）。
- 再想"转换"：把原来内容编码成 7 位 ASCII 的过程；不转换就传，SMTP 传不了。

考查知识点：

- 先想 SMTP 为什么只认 7 位 ASCII：SMTP 是很早的协议，当时只考虑英文文本；协议规定每个字节最高位必须是 0，最高位是 1 的字节传过去会被当成控制字符或者直接丢掉。
- 再想 7 位 ASCII 文本为什么不用转换：它本来就是 7 位 ASCII，符合 SMTP 的规定，直接传就行。
- 再想 8 位 ASCII 文本为什么要转换：8 位字符（比如带重音符号的西欧文字、中文字符的某种编码）最高位是 1，SMTP 传不了，得先用 MIME 编码转成 7 位 ASCII。
- 再想二进制文件为什么要转换：图片、音频、可执行文件都是二进制数据，里面什么字节都有，必须用 base64 这类编码转成 7 位 ASCII 文本再传；收件方再解码还原。
- 边界情况：MIME 编码后传的还是 7 位 ASCII 文本，SMTP 本身没变；现在的邮件系统都支持 MIME，但 SMTP 协议本身还是只认 7 位 ASCII。

正确选项为什么对： 7 位 ASCII 纯文本符合 SMTP 的规定，不用转换就能直接传，就是 D，选 D。

错误选项为什么错＋举例：

- A（8 位 ASCII 文本）：8 位字符最高位是 1，SMTP 传不了，得先用 MIME 编码转成 7 位 ASCII。教训：7 位直接传，8 位要编码。
- B（二进制文件）：二进制数据里什么字节都有，必须用 base64 编码转成 7 位 ASCII 再传。教训：二进制文件先编码、再传。
- C（图片、音频）：图片和音频都是二进制数据，同样要 MIME 编码转成 7 位 ASCII 才能传。教训：非文本内容都要编码。

笔记本一句话： 第 40 题＋SMTP 直接传：只有 7 位 ASCII 纯文本不用转换；8 位文本、二进制文件都要 MIME 编码。

【图像】



SMTP只认7位ASCII文本；图片视频二进制文件都要MIME编码转成ASCII才能传

【定位】 计算机网络 · 第6章应用层 · 电子邮件 SMTP

【易混对比】

- "7 位 ASCII"和"8 位/二进制"是两回事：SMTP 只认 7 位 ASCII（老协议）；中文、图片、视频、EXE 都是 8 位或二进制，必须先编码转成 7 位 ASCII（MIME）才能传。
- "无需转换"和"编码后能传"是两回事：图片视频编码后也能传，但那是"转换后"；题目问"无需转换"，只有 ASCII 文本。
- "SMTP"和"HTTP/FTP"是两回事：HTTP/FTP 传二进制没问题；SMTP 是老协议，只认 7 位 ASCII，这是它的历史包袱。

SMTP 只认 7 位 ASCII 文本；图片、视频、二进制文件都要先编码转换。

【坑位】

别觉得"现在邮件能发图片，所以 SMTP 能直接传图片"——那是 MIME 编码转成了 ASCII 再传的，不是"直接"。

最容易踩的坑是：想"我天天用邮件发图片，SMTP 肯定能传图片"。为什么错：你发的图片是邮件客户端先用 MIME 编码转成了 7 位 ASCII 文本，再交给 SMTP 传的；SMTP 线上跑的还是 ASCII。题目问"无需转换直接传"，图片视频 EXE 都要转，只有 ASCII 文本不用转。正确做法：看到 SMTP，想"老协议、7 位 ASCII"；看到"无需转换"，只选 ASCII 文本。

【类比】

像只收中文信的邮局（SMTP）：英文信（ASCII）直接寄；法文信（图片/视频/二进制）得先找人翻译成中文（MIME 编码），才能寄。题目问"不用翻译直接寄"，只有中文信（ASCII 文本）。

【口诀】 SMTP 老协议，只认 7 位 ASCII；图片视频要先编码，无需转换只有文本。

【考频】 「SMTP」在 2009–2022 年解析中出现 6 次 · 中频

附注

- 本册只含 2018 年 408 选择题第 1–40 题，不含综合应用题；升级版每题含完整题干与选项，可脱离原卷独立使用；讲解重心在方法、推理、易混点上。
- 每题固定 6 节：入手点 → 题干专有名词从零解释 → 考查知识点 → 正确选项为什么对 → 每个错误选项为什么错＋举例 → 笔记本一句话。
- 题干问"不符合／不是／不会／不属于"的互换题（第 4 节讲正确选项即那个符合题干要求的选项为什么符合，第 5 节讲其余选项为什么不符合）：第 2、7、30、34 题。
- 语言风格：大白话为主，每个术语第一次出现配一句大白话解释，顺序按"从具体到抽象"（先讲生活例子、小数字、具体场景，再贴术语名）；讲解比 2022 版更深，考查知识点与错误选项两节把定义、适用条件、边界情况、易混对子讲到位；每题自成体系，不依赖其他题的解析，不预设前置概念。
- 答案表（逐题值）：
- 数据结构：第 1 题 B，第 2 题 C，第 3 题 A，第 4 题 A，第 5 题 A，第 6 题 C，第 7 题 D，第 8 题 B，第 9 题 C，第 10 题 D，第 11 题 A。
- 计组：第 12 题 D，第 13 题 C，第 14 题 A，第 15 题 A，第 16 题 B，第 17 题 C，第 18 题 B，第 19 题 A，第 20 题 D，第 21 题 B，第 22 题 C。
- 操作系统：第 23 题 C，第 24 题 D，第 25 题 B，第 26 题 A，第 27 题 C，第 28 题 D，第 29 题 D，第 30 题 A，第 31 题 D，第 32 题 C。
- 计网：第 33 题 B，第 34 题 C，第 35 题 D，第 36 题 D，第 37 题 D，第 38 题 C，第 39 题 B，第 40 题 D。

核对结果

- 双源比对（答案表级）：GitHub 公开 PDF《2018年408真题》文末参考答案节与夸克版《2018年计算机408统考真题解析》第 1 页答案表逐题比对，40/40 一致，差异 0 题；答案按一致值写，无自行改动。
- 图题部分核对（2 题，图片细节未能全部看清，图中细节以原卷为准）：
- 第 6 题：二叉排序树图题，结点 x1～x5 依次以折线相连；左右孩子对应关系不够清晰。
- 第 7 题：有向图题，结点 1～6 以有向边相连；部分箭头方向不够清晰。
- 表题：第 24 题（表列为进程、等待时间、需要的 CPU 时间、优先级；P1：30μs、12μs、10；P2：15μs、24μs、30；P3：18μs、36μs、20；一次进程调度与切换的系统开销为 1μs）；第 25 题（thread1 与 thread2 各有取共享变量 x 到寄存器、寄存器加 1、把寄存器写回 x 三条指令，初始共享变量 x 为 0）。
- 图题：第 37 题（路由器 R 经交换机 S1、S2 连接两个网络；图中地址已给出：R 左侧接口 192.168.3.1、MAC 00-1a-2b-3c-4d-51，右侧接口 192.168.4.1、MAC 00-a1-b2-c3-d4-61；H1 为 192.168.3.2、MAC 00-1a-2b-3c-4d-52；H2 为 192.168.4.2、MAC 00-a1-b2-c3-d4-62）。
- 语言与结构自查：全文检索跳步词 0 处，跨题外指 0 处，教科书名 0 处，内部用语 0 处；40 题标题与答案逐题核对，与给定答案表一致；每题题意概括与 6 节各 40 处，齐全。